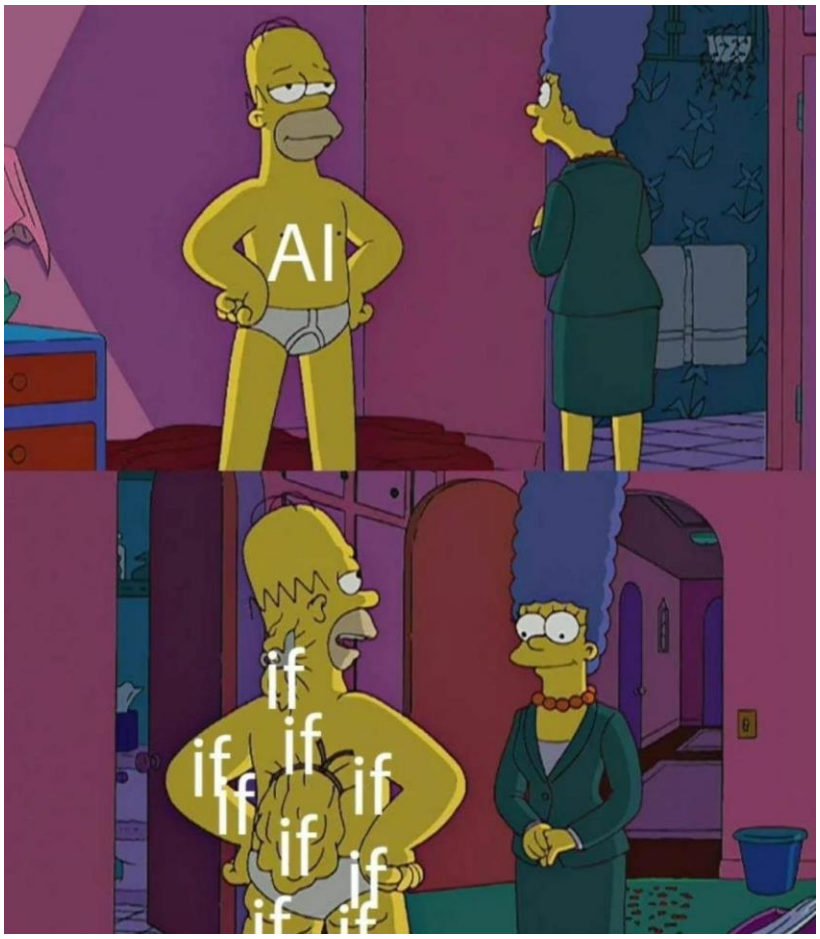
# BITÁCORAS INTELIXENCIA ARTIFICIAL

## 2021-2022

## 3º curso – 1º cuadrimestre

<u>Os seguintes apuntamentos foron recollidos en forma de bitácoras durante cada clase do curso polas seguintes persoas:</u>

Adriana Aurora Rodríguez Oreiro, Adrián Vidal Lorenzo, Andrea Solla Alfonsín, Diana Mascareñas Sande, Elena Segade Martínez, Hugo Vázquez Docampo, Javier Beiro Piñón, Manuel Cid Domínguez, Martín Campos Zamora, Nerea Freiría Alonso, Nicolás Vilela Pérez, Pablo Gil Pérez, Pablo Martínez González, Roberto de la Iglesia, Teresa Gutiérrez Blanco




**Non son apuntamentos recollidos de forma 100% formal e pode haber erros nalgúns datos ou fallos de formato. A pesar disto, pensamos que poden ser de gran axuda para os vindeiros cursos :)**







Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Inteligencia Artificial

06-09-2021

Clase 00

Temas trabajados: Presentación

# Índice

1. Información General ..... 3
a) Horario ..... 3
b) Profesores ..... 3
c) Contexto de la asignatura ..... 3
d) Objetivo general ..... 3
2. Programación: ..... 3
a) Clases expositivas ..... 3
i. Perspectiva histórica da IA ..... 3
ii. Estrategias para a resolución de problemas en IA ..... 3
iii. Aprendizaje automático ..... 5
iv. Repercusiones de la IA ..... 5
b) Clases prácticas y de seminario ..... 2
3. Evaluación. ..... 3

# 1. Información General

6 créditos
50 Horas ECTS + horas de tutoría
• 15 de clase expositiva
• 35 de clases interactivas (seminarios/presentaciones, aproximadamente 10 horas) y de laboratorio (aproximadamente 25 horas)

## a) Horario

Lunes de 9 a 10, miércoles de 9 a 11 las primeras semanas (para llegar al número de horas establecidas en el plan) con descanso en el medio, después solo de 10 a 11.
Clases en gallego, contenidos en español/ingles/gallego.

## b) Profesores

Senén Barro Ameneiro, coordinador. Clases teóricas.
Alberto Bugarín Diz
Alejandro Catalá Bolos
Yago Fontenla Seco, colaborador
Lorenzo Vaquero Otal, colaborador

## c) Contexto de la asignatura

Materia equivalente a una del plan viejo (ingeniería del conocimiento).
Ámbito con un gran protagonismo en empresas, organizaciones, vida privada, servicios, aplicaciones... que lleva décadas de vida, no es reciente como mucha gente cree. En 1956 se le da el nombre de Inteligencia Artificial, pero ya antes de que se le diese esta etiqueta existía gente del mundo de las matemáticas, física, economía, computación, ingeniería... pensando en este ámbito y generando las primeras ideas. Incluir inteligencia en las maquinas ya era algo que se planteaban los empresarios.
La historia de la IA cuenta con altibajos, pero este momento (última década) es el mejor, ni si quiera los puntos más altos de la historia de la IA se acercan al momento que estamos viviendo hoy en día. La IA es el conjunto de tecnologías más transformadoras del mundo, de la economía, la vida diaria... de las desarrolladas hasta ahora en la historia. Por eso es importante que un ingeniero informático tenga una formación en este campo. Esta materia será una primera aproximación a este mundo.

## d) Objetivo general

Obtener conocimientos básicos sobre la Inteligencia Artificial: evolución, tipos de problemas que aborda y técnicas para resolverlos, repercusiones sociales, económicas, éticas, en el empleo...

# 2. Programación:

## a) Clases expositivas

i. Perspectiva histórica da IA

ii. Estrategias para a resolución de problemas en IA

- Busca en espacios de estados

Hay problemas complejos que se pueden modelar a través de una representación denominada espacio de estados. El problema en cada momento está en un estado determinado dentro del conjunto de los posibles.

Hay que buscar como avanzar a través de los estados para llegar a un estado meta. Por ejemplo: ajedrez y otros juegos, desde el origen de la IA es un problema tipo al que los investigadores se enfrentan creando algoritmos y estrategias para intentar resolverlos computacionalmente. Una maquina capaz de tener cierta destreza resolviendo estos problemas se puede decir que tiene cierta inteligencia artificial. El ajedrez es un problema clásico modelable a través de representación de búsqueda en espacios de estados. La ubicación de las piezas en el tablero define un estado. A medida que la partida avanza y vamos moviendo las fichas el estado cambia. ¿Cuál es la estrategia para resolver de forma adecuada este problema? Pues ir haciendo que vaya cambiando el estado hasta llegar a uno de los estados meta (múltiples en este caso) definido como comerle el rey al contrincante o llegar a tablas. Cuando una persona juega no modela el juego de esta manera explícitamente, pero de manera implícita es lo que está haciendo mediante las estrategias de juego y la experiencia del jugador, va anticipando movimientos del contrincante... Hacemos una predicción anticipada de los movimientos que vamos a realizar. Pues esto se puede hacer también computacionalmente mediante búsqueda en el espacio de estados. Haremos una practica sobre esto, diseño de algoritmos y resolución de problemas a través de búsqueda en el espacio de estados.

- Basado en conocimiento

Por ejemplo, un sistema experto en medicina. Si queremos tener un sistema informático que diagnostique una enfermedad igual que lo haría un experto humano ya no contamos con un problema igual al que se puede resolver mediante búsqueda en el espacio de estados, ya que el problema en si (una persona enferma que cuenta con información a valorar para ser diagnosticada) no se puede modelar o sería muy complicado. Aquí lo interesante sería simular el comportamiento humano de este proceso. Si el experto humano empezase con la exploración y análisis (adquisición de evidencias analíticas) y esta información la va contrastando con su conocimiento y experiencia puede generar diferentes hipótesis diagnosticas para tratar de llegar a un diagnostico final, se podría hacer lo mismo a nivel computacional. Tratando de recoger este conocimiento experto y modelarlo de alguna manera se podría usar esos elementos de conocimiento médico para un razonamiento automático. Así se pueden abordar otro tipo de problemas complejos con una estrategia diferente. También haremos una practica al respecto.

- Sistemas conexionistas

Muy de moda últimamente, se habla de redes neuronales artificiales o de computación neuronal. La IA nace inspirada intentando construir modelos funcionales matemáticos a partir de las neuronas. Lógicamente, desde Ramon y Cajal, sabemos que la neurona es la pieza elemental de la inteligencia. Por tanto, si somos capaces de conocer cual es, a nivel funcional y organizativo, el elemento fundamental; si sabemos cómo se conectan las neuronas entre sí, cómo evolucionan con el aprendizaje, que tipo de respuestas ofrecen en función de los estímulos… lógicamente es razonable pensar que esta información puede dar pistas, a través de la bioinspiración, de cómo construir artificialmente sistemas que pudiesen tener algunas de las competencias de los sistemas naturales pero a nivel sintético. Esta aproximación conexionista tuvo altibajos (como todas) y estuvo básicamente abandonada, pero empezó a tomar fuerza de nuevo a finales

de los 80, sobre todo en esta última década. A partir de los años 2010, concretamente el 2012, los algoritmos que ya se habían desarrollado de computación neuronal vuelven a tomar protagonismo como nunca antes, al hilo de una mejora algorítmica, ya que la potencia de cálculo y memoria de las maquinas que siempre fue un problema para las estrategias de IA. Hasta que la arquitectura de los computadores fue lo suficientemente potente no se podían abordar problemas necesarios para aplicaciones practicas. Además, la exigencia de datos: Internet fundamentalmente, las tecnologías ligadas a ellas, el hecho de que hoy cada persona somos un generador continuo de datos, a través de los móviles y de nuestra interacción con las redes sociales y todo tipo de dispositivos somos generadores de datos y conocimientos que se vuelca en las redes, la nube, los dispositivos que pueden compartir información, la cuál es útil de forma masiva para hacer aprender los sistemas de conocimiento automático y, fundamentalmente, aquellos basados en arquitecturas de computación neuronal. Por tanto, toda esta combinación de factores, que se vio en los últimos años, dio lugar a una explosión de capacidades, y por tanto, de aplicaciones que están a la orden del día en todas partes.

iii. Aprendizaje automático

- Regresión lineal y logística
- Aprendizaje supervisado
- Aprendizaje no supervisado

Se dedicará una parte significativa de la materia a analizar las estrategias básicas de aprendizaje automático, que son las que hoy predominan y están omnipresentes en la inteligencia artificial y, desde luego, en sus aplicaciones. Fijarse que un dispositivo puede aprender a resolver un problema sin ser necesario construirlo al mínimo detalle para solucionarlo, lo que supone una ventaja inmensa. Los problemas más complejos se pueden tratar, pensando, por ejemplo, en un coche autónomo, que tiene una gran cantidad de sensores, recibe y tiene que procesar en tiempo real un volumen de información enorme. Además, las circunstancias que se pueden dar son infinitas. Ese coche debe procesar un sin fin de posibles opciones que deben ser contempladas para su correcto funcionamiento: la velocidad, carretera, destino, todo lo que lo rodea a ese coche, intensidad de la luz, objetos en la carretera… ¿Se puede realizar un programa codificado línea a línea con absolutamente todas las variables, todos los contextos, todas las posibilidades con las que se debe enfrentar para que en cada caso ya tiene definido cómo debe responder? Respuesta: NO. Por tanto, o aprende fundamentalmente a desenvolverse de un modo suficientemente seguro y útil o no será posible acometer un problema de estas características y complejidad.

iv. Repercusiones de la IA

Socioeconómicas, éticas... Debido a su gran presencia en nuestras vidas, es un ámbito tecnológico muy transversal. Esto supone un riesgo. Debería estar presente en la formación de todos los usuarios de esta tecnología y de sus desarrolladores una reflexión sobre las repercusiones éticas de la IA ya que puede poner en peligro nuestra privacidad y seguridad tanto física como psicológica. Todas las

aplicaciones deberían concebirse con criterios de carácter ético.

## b) Clases prácticas y de seminario

En relación con el tema de la ética en la IA, deberemos realizar una presentación oral con un debate de un tema, que cada uno escogerá libremente, teniendo que ver con la inteligencia artificial. Se deberá escoger cualquier cosa dónde se use la inteligencia artificial con una mirada en cómo esa aplicación, ese problema, esa forma de afrontarlo; nos puede afectar, para bien o para mal, en cualquier tipo de ámbito humano, sea individual o colectivamente. Esto tiene que ver con las clases interactivas, que son por una parte las prácticas, y por otra un conjunto de clases que se pueden entender como seminarios. Se impartirán en la clase A4. Ejemplos de lo que se verá en estos seminarios: aplicaciones que desarrollaron o se están desarrollando en el CITIUS.

Habrá un seminario de comunicación oral antes de la exposición del trabajo descrito anteriormente. Todos deberán realizar una presentación en grupos, normalmente de 2 personas y si, es necesario de 3, aunque también se puede hacer individualmente. Esta exposición será al final de la materia, aunque se puede comenzar a preparar con anterioridad, ya que, Senén no quiere que se realice a última hora, de manera acelerada y preparando cualquier cosa, debido a que tiene repercusión (mínima) en la nota. ENTRE EL APROBADO Y EL SUSPENSO PUEDE ESTAR LA DEDICACIÓN Y EL BUEN TRABAJO. Más adelante, se darán instrucciones más detalladas.

Un ejemplo evidente y complejo es el coche automático de lo que es la inteligencia artificial en la resolución de un problema al que nos enfrentamos a diario, tanto en un entorno personal como en uno profesional. Está claro que el conseguir coches autónomos es una forma de ganar en eficiencia y en seguridad. Cada año mueren en el mundo más de 1.200.000 personas por accidentes de tráfico, del cual el 95% proceden de errores humanos: despistes, negligencias, consumos de drogas, alcohol,... Si tuviésemos una forma de reducir esta tragedia, ¿quién no apostaría por ello?. Pues, esta forma será el coche autónomo. Esa sería la parte positiva, pero, seguramente también existen elementos negativos, ya que hay cuestiones éticas. Por ejemplo, si una persona se encuentra conduciendo, respetando todas las normas de tráfico en perfectas condiciones pero, de pronto, se cruza algo a 20 metros en una autovía, con casi total seguridad no podrá evitar la colisión o, instintivamente, girará el volante, evitando la colisión pero con la posibilidad de acabar chocando contra la mediana o colisionar contra otro turismo. ¿Alguien puede reprochar cualquier acción que dicho conductor realice en cuestión de milisegundos por causa de un elemento extraño que aparece de forma inesperada a poca distancia? Evidentemente no. En el mejor de los casos la acción realizada fue un acierto y, en otros casos, la peor de las decisiones. Pero, el ser humano no tiene una capacidad de reacción racional ante circunstancias de este tipo pero, una máquina sí puede, ya que, puede reaccionar en cuestión de milisegundos de un modo previsto o de acuerdo con

aquello que aprendió más lo que se diseñó de forma expresa. Aquí se abre un abanico importante de cuestiones éticas a tener en cuenta: ¿cuál es la decisión en esas circunstancias?, ¿la que salve más vidas?,¿la que salve mi vida?. Supongamos que este es un tema que un alumno elige. Este debería leer sobre el tema, reflexionar para aportar su propia opinión, abrir interrogantes para el debate.. Todo esto con el apoyo de una única página visual que puede proyectar en pizarra. NO SE PUEDE ANDAR PASANDO UNA TRAS OTRA. EN UNA ÚNICA DEBE SINTETIZAR AQUELLO QUE CONSIDERA QUE ES ÚTIL PARA ACOMPAÑAR SU DISCURSO ORAL y en 5 minutos debe ilustrar sobre lo que considera que es relevante y su opinión. Después, unos minutos de debates.

Optativamente, todos los años se realiza un concurso de redacción (individual), donde sería lo mismo pero redactando, lo que se podría entender como un artículo de divulgación para un periódico. Debe ser de una página o página y poco. Las personas que deseen participar, deben hablar con el profesor, construir su artículo, se lo envían, él lo lee y realiza recomendaciones (o no), iterando unas cuantas veces. Finalmente, entre varias personas deciden cuál es el mejor, y ese se intentará que se publique en El Correo Gallego. El resto, quedarán colgados en el campus virtual. En el campus hay algunos de años anteriores.

Con respecto a las prácticas, una será de resolución de problemas en espacio de estados, otra sobre sistemas basados en conocimiento y varias de aprendizaje automático con distintas estrategias.

Tutorías: Preferentemente por Teams, siempre con previo aviso. Horario: Jueves 15:30-16:30. Se pueden realizar en otra hora.

# 3. Evaluación.

Hay un examen de teoría, normalmente un tipo test con algunas preguntas cortas. Este vale el 40% de la nota pero, se debe aprobar por separado, de tal modo que, se debe tener por lo menos un 4 para que haga media. Si se tiene un 4 en la teoría pero el resto compensa, aprobado. Las clases interactivas igual, pero pesan un 10% y las prácticas un 50%. Se debe tener un 5 en todas las partes para aprobar.

En la segunda opción (julio), para poder ir a la segunda opción solo se puede ir con las clases prácticas aprobadas, es decir, o se tiene un 5 en clases prácticas y en las interactivas, o no se puede ir a la convocatoria de julio.

En caso de tener que repetir la asignatura al año siguiente, no se guardan partes.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grao en Enxeñaría Informática

3º curso – 1º cuadrimestre

# Intelixencia Artificial

13-09-2021

Clase 01

Temas traballados:

1. Introdución á IA e perspectiva histórica

# Índice

1. Introdución á Intelixencia Artificial ..... 3
a) Definición ..... 3
b) Definición do profesor ..... 3
c) Contexto histórico ..... 3
d) Cerebro vs Ordenador ..... 4
e) Distintos enfoques da Intelixencia Artificial ..... 4
f) Alan Turing ..... 5

# 1. Introdución á Intelixencia Artificial

## a) Definición

Para a Intelixencia artificial non existe unha definición canónica, máis unha definición dada por un grupo de expertos da comisión Europea nesta rama é a seguinte :

**A intelixencia artificial é referida a sistemas que mostran un comportamento intelixente analizando o medio tomando accións - con certo nivel de autonomía – para acadar obxectivos específicos.**

Estes Sistemas poden estar baseados puramente en software ou pode estar embebido en dispositivos hardware.

## b) Definición do profesor

**Que significa que un sistema sexa intelixente?** Podemos dicir que un sistema é intelixente baseándonos en 3 características:

- Ter a habilidade para desenvolverse de forma autónoma en entornos complexos
- Posuír unha gran capacidade de aprendizaxe
- Presentar unha gran competencia nunha área do coñecemento especializada.

## c) Contexto histórico

O campo da intelixencia artificial nace formalmente no ano 1956. **Esta disciplina é unha mestura entre ciencia e enxeñaría**:

- Por un lado é unha disciplina científica porque o que busca é avanzar e incrementar o coñecemento
- Para acadar este obxectivo fai uso das ferramentas que a enxeñaría pon á súa disposición, e a intelixencia artificial é aplicada principalmente para a resolución de problemas.

Nun artigo do ano 1958 do New York Times, xa se falaba dos avances nesta materia, fantaseando coa posibilidade de que un ordenador puidera andar, falar, ver, escribir, reproducirse e incluso ser consciente da súa existencia. Nos 4 primeiros aspectos, a día de hoxe avanzouse moito, mais nos dous últimos apenas se conseguiu mellora algunha.

**Nesta disciplina, os avances non se conseguiron de forma continua**, se non que houbo "invernos" nos que a investigación se estancou, mais cando chegan as computadoras con certo grao de capacidade computacional e se van desenvolvendo as linguaxes de programación, a intelixencia artificial comeza a dar froitos.

Hoxe en día na IA búscase a **aprendizaxe autónoma a través de grandes volumes de datos**.

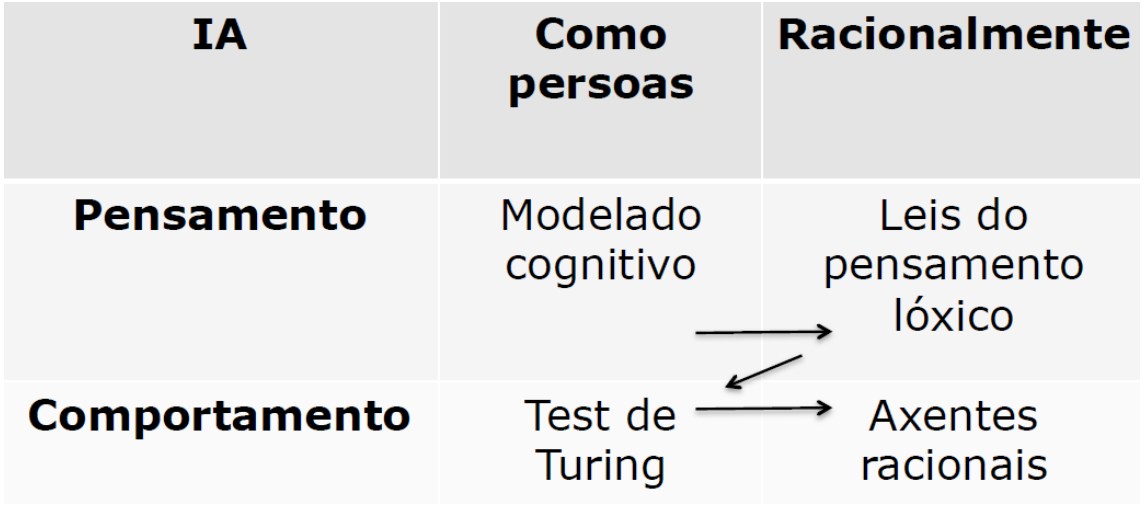
Destaca o feito de que a maior parte da investigación neste campo estase realizando por parte do sector privado.

## d) Cerebro vs Ordenador

| Elemento de comparación | Cerebro | Computador |
|---|---|---|
| Tamaño | Aproximadamente 1.500 cc | 720 m² de superficie ocupada e máis de 100 toneladas de peso (2) |
| Consumo de enerxía | 20 watts (1) | 17,6 MW -24 MW se se inclúe a unidade de refrixeración- (2) |
| Capacidade de almacenamento / memoria | 3.500 terabytes (1) | 1 petabyte de memoria asociada aos procesadores e 12,4 petabytes de capacidade de almacenamento total (2) |
| Capacidade de procesamento | 2.200 teraflops (1) | 33,86 petaflops (2) |
| Aprendizaxe | Enorme capacidade de aprendizaxe | Aprendizaxe en máquina implementadas vía software |
| Evolución | Poucos cambios nos últimos 100.000 anos | Moi rápida, sobre todo en potencia de cálculo e capacidade de almacenamento |
| Cálculos matemáticos | Moi lento | Moi rápido |
| Problemas lóxicos – formalizables algorítmicamente- | Moi lento | Moi rápido |
| Percepción e acción sobre a contorna | Moi rápido | Moi lento |
| Razoamento de sentido común | Moi rápido | Moi lento |

Nesta comparativa queda claro a grandísima capacidade que ten o cerebro humano, pero, sobre todo, a súa eficiencia enerxética. O cerebro ten un peso relativamente pequeno comparado co do resto do corpo e consume o 20% da enerxía, o cal é un gasto considerable, pero se o comparamos coas máquinas que a día de hoxe podemos construír, o consumo é insignificante.

## e) Distintos enfoques da Intelixencia Artificial




Nun primeiro momento o que inspirou a xente a traballar no campo da Intelixencia Artificial foi a idea de **modelar a través de dispositivos electrónicos o pensamento humano**, é dicir, lograr a nivel artificial máquinas que pensaran como supostamente o facemos as persoas (**modelado cognitivo**) sen necesariamente aplicacións directas onde proxectar os resultados. Esta vía non deu grandes froitos, e co paso do tempo comezouse a traballar no que serían **máquinas con capacidade de facer razoamento lóxico, non necesariamente o humano** ("unha forma de razoamento humano matematizable"). Posteriormente foi collendo interese a idea de centrarse

máis no c**omportamento** da máquina, buscando que este sexa igual ca o dunha persoa (**Test de Turing**). Un exemplo desta búsqueda nas máquinas do comportamento humano é a robótica. Ultimamente prima a Intelixencia Artificial que busca **comportamentos racionais**, é dicir, **resolver un problema sen imitar o modo no que o fan as persoas**, tanto desde o punto de vista do pensamento como o do comportamento.

### f) Alan Turing

Alan Turing foi un matemático e científico da computación con contribucións á intelixencia artificial (antes incluso de que este campo se comezara a desenvolver) asombrosas. Estas contribucións son das máis sólidas do século pasado. No ano 1936 propón a **máquina de Turing**, o modelo formal (simple) dunha máquina que computacionalmente é equivalente a un computador de propósito xeral.

No ano 1950 publica un artigo no que na primeira liña escribe a frase "Poden as máquinas pensar?". **El determina a través do Test de Turing que se unha máquina é capaz de manter unha conversa cunha persoa e a engana un 30% das veces de forma que a confunda cun humano, a máquina pensa.**

O motivo de reflexión é se realmente lle hai que recoñecer a unha máquina que teña a capacidade de falar (forma máxima da demostración de intelixencia), independentemente da forma na que este construída a máquina, que esta é intelixente.







## Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Inteligencia Artificial

*15-09-2021*

Autores:

[redacted]

**Clase 1**

Temas trabajados:
Tema 1 - Introducción a la Inteligencia Artifial

# Índice

**1. Alan Turing** **2**
1.1. ¿Qué es pensar? . . . 2
1.2. Test de Turing . . . 2
1.3. Maquina de Turing . . . 2
1.4. Maquina Enigma . . . 2

**2. La IA a lo largo de la historia** **3**

**3. Diferentes aproximaciones en el tiempo** **3**
3.1. Subsimbólica . . . 3
3.2. Simbólica . . . 5

**4. Hitos de las interligencias artificiales** **5**
4.1. Deep Blue vs Kasparov . . . 5
4.2. AlphaGo vs Lee . . . 5

**5. Paradoja de Moravev** **5**

**6. Problema de los mínimos locales** **6**

# 1. Alan Turing

Alan Turing publicó en 1950 en la revista Mind: "Computing Machinery and Intelligence" que comienza así:"I propose to consider the quiestion, Can machines think?" A la pregunta a la pregunta, ¿pueden las máquinas pensar?, Alan Turing expuso como, en caso de que nos encontremos hablando con una máquina y no seamos capaces de discernir que se trata de una computadora y no una persona, entonces sí. Esta prueba, que inicialmente Turing bautizó como "Imitation Game", es lo que posteriormente se conocerá como "test de Turing". En realidad, es suficiente con engañar a las personas en un 30 % de las ocasiones.

## 1.1. ¿Qué es pensar?

Para determinar si un ordenador que realiza traducciones es "inteligente", se propuso la siguiente hipótesi.
Si una habitación con una persona que dispone de un repositorio infinito de información, se dedica a realizar traducciones de idiomas que no conoce empleando todos los datos de los que dispone, ¿se puede decir que la persona realmente sabe esos idiomas?. No, dado que realmente no habla ninguno de ellos. Del mismo modo, las maquinas no son realmente inteligentes, ya que solo realizan asociaciones sobre la información que tienen disponible.
Existe una gran cantidad de contraargumentos a esta premisa. Por ejemplo, desde un punto de vista utilitario, esa persona es capaz de comunicarse en cualquiera de los idiomas, por lo tanto sí los sabe.

## 1.2. Test de Turing

Para la realización de este experimento se cuenta con 3 participantes: un juez, un humano y una máquina. El juez debe comunicarse con el humano y con la computadora a través de algún sistema como un chat de ordenador (de manera que no intervenga ni la voz ni la caligrafía de los participantes). Al final de la entrevista con ambos el juez debe determinar quien es el humano y quien es la máquina. Turing defiende que si la maquina consiguiera engañar al juez haciéndole creer que ella es el humano, indudablemente la maquina piensa.

## 1.3. Maquina de Turing

En el 36 idea (de forma completamente teórica) la máquina de Turing, con la misma capacidad de resolución de problemas computables que los ordenares actuales. No obstante, es importante destacar que esta capacidad de resolución no implica la misma eficiencia ni complejidad (siendo la máquina de Turing menos eficiente y más compleja).

## 1.4. Maquina Enigma

Durante la Segunda Guerra Mundial el ejercito nazi utilizaba una maquina para encriptar todas sus comunicaciones haciéndolas incomprensibles para el ejercito Aliado. Esta fue la conocida "Máquina Enigma", que constaba de 3 o 4 (depende de la versión) cilindros los cuales van girando con cada tecleo y cifrando cada letra en función de su posición (cuando el primer cilindro da una vuelta completa el siguiente empieza a girar). De esta manera la única forma de desencriptar los mensajes sería conociendo la posición inicial de los cilindros.
Fue el ejército inglés el que formo una división encargada de intentar desencriptar los mensajes. Esta estaba formada por Turing y muchos otros brillantes criptólogos y lingüistas. Gracias a la

construcción de la primera supercalculadora (no se llega a considerar aún computadora) por Turing, denominada "The Bombe"se logró poder averiguar la disposición de los mensajes alemanes. Gracias a esto se calcula que la Guerra se logró acortar 2 años salvando miles de vidas humanas.

# 2. La IA a lo largo de la historia

A lo largo de la historia de la inteligencia artificial han ocurrido los denominados "inviernos", estas son épocas donde la investigación y el avance en este campo se ha escaseado.
En un primer momento, se desarrollaron inteligencias artificiales bioinspiradas (subsimbólicas), no obstantes las limitaciones de la época llevaron a este primer invierno. Posteriormente, se comenzó a recuperar el interés con el desarrollo de, también, IA simbólicas. No obstante, la falta de avance y el fracaso de varios proyectos multimillonarios en superordenadores que permitiesen un mayor desarrollo del campo por parte de países como China, llevaron a la llegada del segundo invierno. Finalmente, hasta la actualidad, este campo ha sufrido un gran desarrollo, sobre todo por parte de las multinacionales, regresando de nuevo el foco en gran parte a las IA subsimbolicas bioinspiradas.

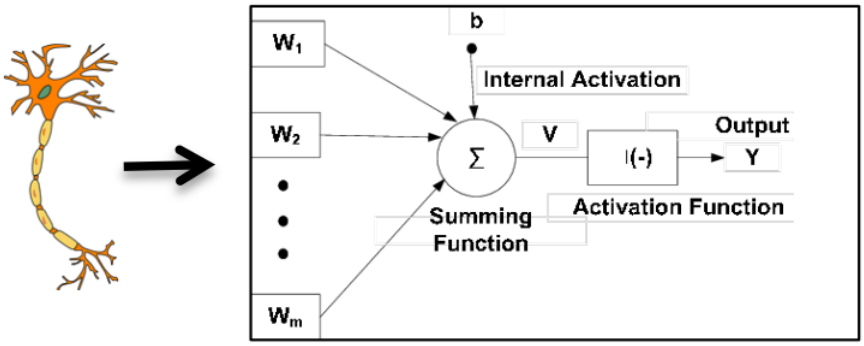
| Período | Descripción |
|---|---|
| 1940 - 1955 | Primeiros pasos prometedores |
| 1956 - 1973 | Arranque moi optimista |
| 1974 - 1980 | Declive –primeiro "inverno"- |
| 1981 - 1987 | Recuperación |
| 1988 - 1993 | Estancamento –segundo "inverno"- |
| 1994 - 2010 | Relanzamento |
| 2010 - | Ambición e realismo |

# 3. Diferentes aproximaciones en el tiempo

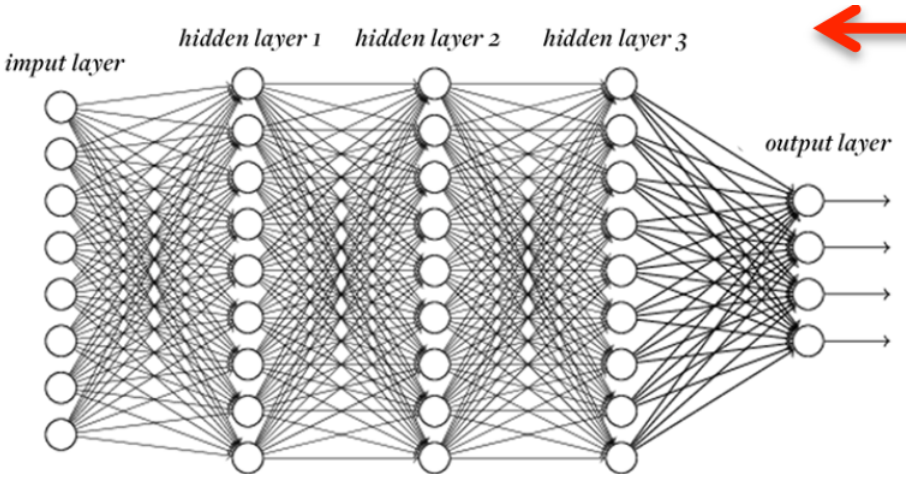
Para lograr alcanzar esta inteligencia artificial se han planteado 2 modelos:

## 3.1. Subsimbólica

La inteligencia artificial nace a través de la bioinspiración, esto es, utilizar modelos imitando las neuronas y los circuitos neuronales.
Ramón y Cajal fue el científico español que logró encontrar el mecanismo mínimo del cerebro, las neuronas. Estas son integradores espacio-temporales que cambian entre 2 estados: activa o no activa y mandan esta señal al resto de neuronas formando un circuito neuronal.
Cuando se quieres recrear estas neuronas de manera artificial se aplica el siguiente modelo: En primer lugar tenemos diferentes entradas ponderadas (son la parte sobre la que actúan los algoritmos), una función de suma de todas estas entradas, un función de activación y una salida.




Cuando se combinan varias neuronas se consigue formal las redes neuronales las cuales se conforman de distintas capas:

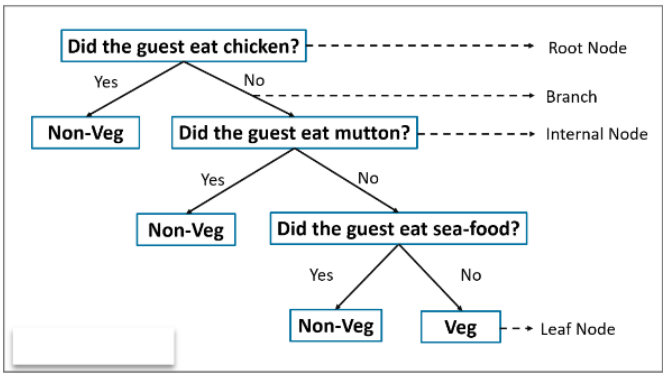



# Subsimbólica

La primera de ellas es la capa de entrada, la cual podemos manejar a nuestro antojo, podría ser, por ejemplo los píxeles que conforma una radiografía. La última capa es la capa de salida que nos da una respuesta codificada del problema, por ejemplo, el decir si en la radiografía hay algún tipo de tumor. El resto de capas intermedias son una caja negra, es decir, que no podemos entender a simple vista que implican. Esto es un problema para intentar entender los patrones que las IAs pueden encontrar y nosotros no.
Existe un campo denominado Explicabilidad que intenta entender que es lo que estas capas significan.
Para el entrenamiento de estas redes se parte de unas entradas con pesos aleatorios y a través de datos etiquetados se realizan pruebas aplicando ciertos criterios hasta lograr un modelo que pueda resolver el problema. Esto se denomina Problema Supervisado.

### 3.2. Simbólica

Al contrario que las inteligencias artificiales subsimbólicas, las simbólicas se basan no en la replicación de la estructura neuronal, si no en la propia funcionalidad del cerebro. Es decir, el objetivo es crear un procesador **simbólico**. De este modo, para su creación se realiza una




programación estándar en cualquier lenguaje. La red simbólica, recibirá unos estímulos (en forma de símbolos a los que nosotros daremos significado propio) y nos encargaremos de especificarle a través del lenguaje escogido la forma en la que debe actuar.

## 4. Hitos de las interligencias artificiales

### 4.1. Deep Blue vs Kasparov

Esta inteligencia artificial simbólica desarrollada por IBM fue capaz de ganar al que se considera como el más grande maestro de ajedrez de la historia, Kasparov.
Esta noticia generó en la gente una sensación de asombro, haciéndoles pensar que las IAs estaban cerca de alcanzar a la inteligencia humana, sin embargo, esta hazaña no tiene nada que ver con la superación ni igualdad de la inteligencia humana.

Actualmente DeepMind ha creado una red neuronal subsimbolica, la cual con menos de 40 horas de entrenamiento ha conseguido desbancar a Deep Blue como el mejor software jugador de ajedrez.

### 4.2. AlphaGo vs Lee

DeepMind ha entrenado a lo largo de los años a diferentes redes neuronales basadas en inteligencia artificiale subsimbolica para que aprendiesen a jugar a diferentes juegos clasicos de ATARI. Una de las principales conclusiones que pueden obtenerse de estas redes entrenadas es que no son extraporables, pues una red con grandes resultados en la realización de una función expecífica, con gran seguridad no poseerá niguna utilidad en cualquier otro propósito para el que no haya sido entrenada.

## 5. Paradoja de Moravev

La paradoja de Moravec es el descubrimiento en el campo de la inteligencia artificial (IA) y robótica de que, de forma antiintuitiva, el pensamiento razonado humano (el pensamiento

inteligente y racional) requiere relativamente de poca computación, mientras que las habilidades sensoriales y motoras, no conscientes y compartidas con otros muchos animales, requieren de grandes esfuerzos computacionales.

# 6. Problema de los mínimos locales

Actualmente las redes neuronales subsimbólicas se entrenan empleando la función derivada del error que obtienen frente a resultados óptimos. En este sentido se llega a un problema, cuando la derivada llega a 0, entonces la función a alcanzado un mínimo local, no obstante no es posible actualmente determinar si existe otro mínimo menor (un mínimo absoluto) que reduzca aún más el error cometido y que justifique una continuación del entrenamiento.

Se ahondará en este problema en poteriores temas.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Computación Distribuida

20-09-2021

Clase 03

Temas trabajados:

1. Introducción a la Inteligencia Artificial
2. Búsqueda en espacio de estados

# Índice

1. Tema 1: Introducción a la Inteligencia Artificial ..... 2
a) Coche autónomo ..... 2
b) Proyectos de inteligencia artificial ..... 2
2. Tema 2: Búsqueda en espacio de estados ..... 2
c) Búsqueda de soluciones ..... 3
d) Búsqueda en espacio de estados ..... 3

# 1. Tema 1: Introducción a la Inteligencia Artificial

## a) Coche autónomo

A pesar de la gran evolución de la inteligencia artificial y la gran cantidad de avances en el campo, todavía existen ciertas situaciones que no se pueden predecir. En el caso de los coches autónomos, tienen una gran cantidad de sensores que les permiten identificar muchos más obstáculos de los que los seres humanos son capaces, sin embargo, no pueden predecir situaciones potencialmente peligrosas que nosotros sí podemos intuir (por ejemplo, unos niños jugando al balón que pueden meterse delante de repente o un coche que permanece oculto detrás de otro vehículo).

Un claro ejemplo de la superioridad humana en ciertos aspectos se identifica en circunstancias en las que se cubre un objeto. Para el ser humano, es obvio que el objeto se encuentra ahí, pese a estar oculto. Sin embargo, la inteligencia artificial interpreta que ese objeto ha desaparecido del entorno.

Con el paso del tiempo, los sensores en este tipo de vehículos han ido reduciendo considerablemente su tamaño, volviéndose a su vez más funcionales.

## b) Proyectos de inteligencia artificial

- El CITIUS, mediante el uso de Inteligencia Artificial está colaborando con el proyecto NÓS, que trabaja para incorporar el gallego como una lengua más en el uso de las TIC.
- Otro avance importante logrado gracias a la IA es la generación automática de texto descriptivo acerca del clima a partir de los datos obtenidos.
- El SITUM es otro gran proyecto que utiliza la IA. Se trata de una SpinOff del CITIUS que está trabajando en el desarrollo de un software que permite geolocalizar los teléfonos dentro de los edificios. Para ello emplea balizas, giroscopios, magnetófonos... de los dispositivos para obtener una ubicación precisa. Esto permite lidiar con el problema de los satélites, ya que estos no funcionan en el interior de los edificios.

# 2. Tema 2: Búsqueda en espacio de estados

Vamos a tratar:

- Problemas resolubles representándolos en un espacio de estados, moviéndonos con operadores a través de estrategias que la IA fue desarrollando. Ejemplo: ajedrez; no existe criterio de optimización ya que el único objetivo es ganar. Existen demasiados estados y computacionalmente hablando es imposible representar todos los estados. Más bien aplicaremos estrategias no exhaustivas que solo exploren las "mejores" opciones.
- Estrategias que buscan a ciegas (sin poseer información del problema). Normalmente se realiza una búsqueda con heurística ya que da mejores resultados que la búsqueda a ciegas.

## c) Búsqueda de soluciones

Existen tres métodos para realizar la búsqueda de soluciones:

**Preciso:** Aporta una solución óptima empleando algoritmos ya definidos. (invertir una matriz, multiplicarla...)

**Heurístico:** Utiliza un método de resolución de problemas que aplica conocimiento específico del problema para encontrar una solución aproximada. Se utiliza cuando el espacio de estados es tan grande que no es posible analizar todas las posibilidades o cuando no es necesaria encontrar la mejor solución, si no que con una buena es suficiente.

**Metaheurístico: [NO SE DA EN LA ASIGNATURA].**

## d) Búsqueda en espacio de estados




1. 1-Tenemos un problema
2. 2-Creemos que podemos modelarlo en un espacio de estados
3. 3- Hay un estado inicial.
4. 4- Queremos llegar a un estado final (a veces, ya que en juegos como el ajedrez existen muchos estados de victoria).
5. 5- Existen muchos operadores que permiten evolucionar a un estado final.

En el ajedrez, en cada movimiento, el factor de ramificación es de 30 (por cada movimiento existen de media 30 posibles movimientos). Esto desemboca en un crecimiento exponencial y explorar todas las posibilidades sería inviable. Por ello, se sacrifican "posibles" mejores vías para poder computarlo; es decir, en lugar de representar todo el grafo, solo se exploran algunas ramas que parecen las mejores a priori. Este método suele ser bastante efectivo.

De este modo, construir el grafo entero tiene como ventaja obtener la solución óptima. Sin embargo, posee la desventaja de tener un alto coste computacional.

Por ejemplo, en el caso del problema del viajante de comercio, la complejidad es O(n!). Es decir, que para 100 ciudades las posibilidades serían superiores al número de partículas del universo. Por ello, se emplean algoritmos voraces; para cada conjunto de opciones, se escoge siempre la

mejor. Esto proporciona buenos resultados (no siempre los mejores) y reduce significativamente la complejidad del problema.

Ejemplo en donde escoger la opción de menor coste no siempre proporciona el mejor resultado. En el problema explicado anteriormente, si escogemos siempre la mejor opción (la ciudad más cercana a la actual) podemos no llegar a la mejor solución. Si la ruta parte en Santiago y finaliza en Moscú, de forma que las ciudades a recorrer España son todo ciudades a lo largo de la costa cantábrica y Sevilla, la ciudad más cercana nunca será Sevilla, por lo que se recorrerán todas las ciudades de España y se continuará hacia Moscú sin haber pasado por Sevilla así que, una vez llegado a Moscú, habría que volver hasta España (por lo que no sería la mejor solución).

**https://www.youtube.com/watch?v=0lzOenTOwQU**







**Grado en Ingeniería Informática**

3º CURSO – 1º CUATRIMESTRE

# Inteligencia Artificial

*15-09-2021*

Autores:

[redacted]

**Clase 3**

Temas trabajados:
Tema 2

# Índice

**1. Búsqueda de espacios de estados** **2**
1.1. En el caso del n-puzzle . . . . . 2

**2. Búsqueda a ciegas** **3**
2.1. Caracterización de las propiedades de las estrategias . . . . . 5

**3. Busca heurística** **7**
3.1. Función de evaluación . . . . . 7
3.2. Ejemplo . . . . . 7
Algoritmo A* . . . . . 7
Función de evaluación . . . . . 8
Búsqueda avara . . . . . 10
Concluisiones . . . . . 10
3.3. Heuristicas . . . . . 11

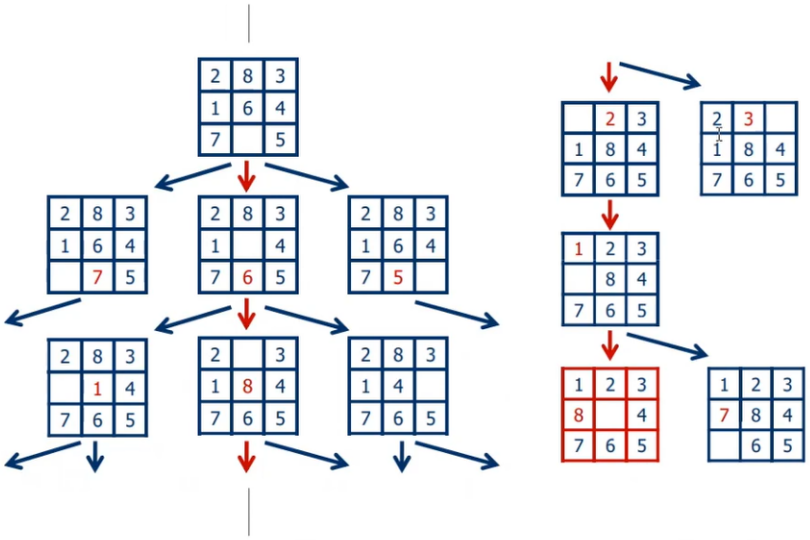
# 1. Búsqueda de espacios de estados

- **Espacios de estados:** estados a los que se puede llegar desde el **estado inicial** mediante cualquier secuencia de operadores.
- El proceso de **búsqueda** trata de encontrar una meta o solución al problema, donde a veces esta solución puede ser la **ruta** seguida (en el ajedrez por ejemplo no nos importa).
- **Prueba de meta:** permite saber si un estado dado es meta o solución del problema.
- **Condición de parada**, parcial o total: Criterio/s para detener la búsqueda. Lo ideal es que se de cuando se llega a la solución, pero en ocasiones también es necesaria para, por ejemplo, no entrar en un bucle infinito.

## 1.1. En el caso del n-puzzle

- **Estados:**Disposición de las piezas en un momento dado.
- **Operadores:** movimiento del espacio vacío $\wedge, \vee, <, >$
- **Prueba de meta:** ¿Se llegó al estado final? (una cierta disposición de los números)
- **Coste de la ruta:** Número de movimientos que realizados desde el estado inicial a la meta.

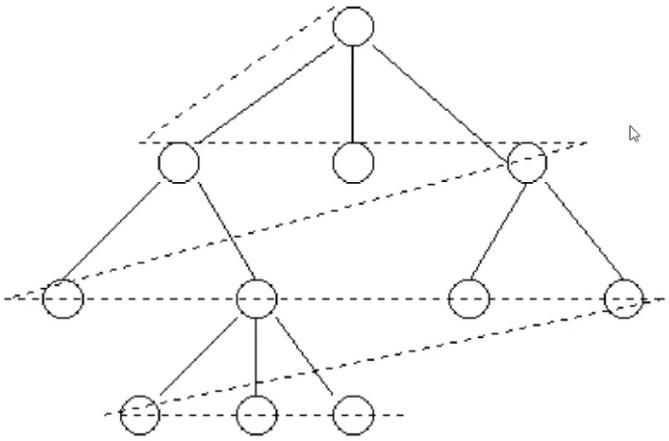
**NOTA:**El espacio de estados de este tipo de problema tienden a alcanzar enormes tamaños al recorrerse



Si se recorre el siguiente árbol de estados en amplitud (donde en ocasiones la ramificación es 4,3 o 2) se encontrará la solución óptima, ya que la primera solución que aparecerá, será la que menos movimientos requiera.
Con este recorrido, se contemplan todos los estados a los que se puede llegar, sin embargo esto en ocasiones no es posible por la cantidad de memoria que sería necesaria.
Sin embargo, si se utiliza un recorrido en profundidad, puede que no se encuentre la solución óptima o que incluso no se encuentre solución.

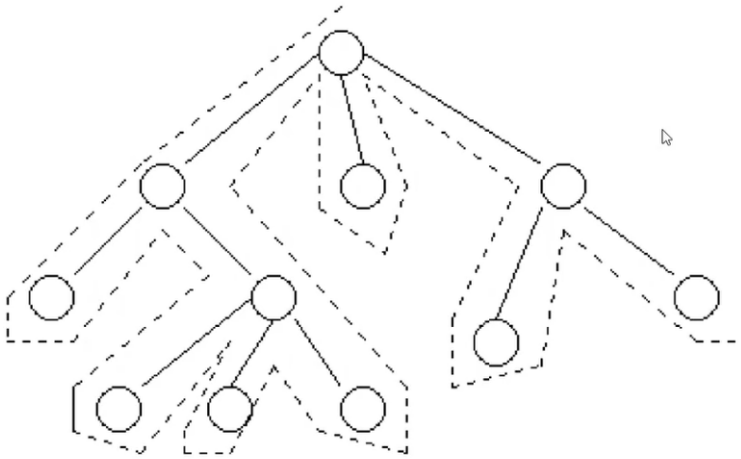
- Imposibilidad de explorar todas las alternativas en problemas complejos (no podemos valorar todas las posibilidades en el ajedrez por falta de memoria)
- Si existen múltiples soluciones, normalmente es suficiente con buscar una solución aceptable (EL MEJOR es enemigo de EL BUENO)
- **Heurísticas**: Son criterios para seleccionar un operador o una acción prometedora.

## 2. Búsqueda a ciegas



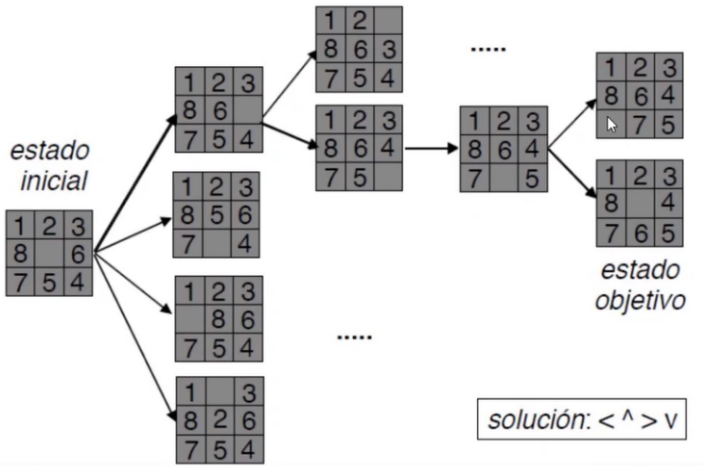
Busca en amplitude

Se va construyendo de forma exhaustiva y completa el grafo. Desde el estado inicial se ve quienes son los hijos que se derivan a partir de la realización de una operación.



Busca en profundidade

Se parte del estado inicial y se construyen sus hijos, se va a uno de ellos y se construyen sus hijos y así sucesivamente. Mientras se pueda si va por una rama hacia abajo.




8-quebracabezas en profundidade

Se parte del estado inicial, se selecciona uno de sus hijos y se desenvuelve, y así sucesivamente con sus hijos mientras se comprueba si el estado es solución o no. Finalmente se llega a un estado solución, pero no podemos asegurar que sea la solución óptima en cuanto número de pasos. Pero se logró ahorrar bastante memoria.

## 2.1. Caracterización de las propiedades de las estrategias

- Búsqueda **completa**: si existe una solución, la encuentra.
- Búsqueda **óptima**: encuentra la mejor solución, si existe.
- **Complejidad**(en términos relativos):
  - **Temporal**: Número de nodos explorados
  - **Espacial**: Máximo número de nodos en memoria de forma simultánea.
- **Terminología:**
  - **Factor de ramificación (r)**: Número medio de sucesores de los nodos
  - **Profundidad de la solución (p)**

| | **Amplitude** | **Profundidade** |
|---|---|---|
| **Busca completa** | Sí | Non, agás en espazos finitos e sen bucles |
| **Busca óptima** | Sí | Non |
| **Complexidade temporal** | $O(r^p)$ | $O(r^m)$ |
| **Complexidade espacial** | $O(r^p)$ | $O(r*m)$ |

Avaliación de estratexias de busca a cegas

r: factor de ramificación; p: profundidade da solución;
m: profundidade máxima da árbore de busca

La única ventaja que ofrece el recorrido en profundidad frente al recorrido en amplitud es la complejidad espacial, ya que no requiere una cantidad tan grande de memoria debido a que no guardamos el árbol entero.
En muchas ocasiones se debe sacrificar el obtener la mejor solución (búsqueda en amplitud) para poder resolver un problema, ya que no se dispone de memoria suficiente.

| | Preferente por amplitude | Profundidade | Limitada en profundidade | Profundización iterativa | Bidireccional (cando é posible) |
|---|---|---|---|---|---|
| Tempo | $r^p$ | $r^m$ | $r^l$ | $r^p$ | $r^{p/2}$ |
| Espazo | $r^p$ | r*m | r*l | r*p | $r^{p/2}$ |
| Óptima? | Si | Non | Non | Si | Si |
| Completa? | Si | Non | Si, cando l>p | Si | Si |

## Avaliación de estratexias de busca a cegas

r: factor de ramificación; p: profundidade da solución; m: profundidade máxima da árbore de busca; l: límite da profundidade

- **Limitada en profundidad**:Es necesario fijar un límite de profundidad. Este límite es hasta la capa a la que se puede llegar al realizar la búsqueda. Es por esto que solo se puede conseguir encontrar una solución si está se encuentra en un nivel superior a este límite.
- **Profundización iterativa**: Cada rama va creciendo de forma iterativa de 1 en 1. De esta manera no se almacena el grafo completo en la memoria.
- **Bidireccional**: Se va buscando de forma simultánea desde el estado inicial al estado solución y del estado solución al estado inicial. De esta manera cuando, los 2 arboles se conecten, significa que se ha encontrado la solución. Esta búsqueda no se puede utilizar, por ejemplo, en el ajedrez ya que no conocemos de antemano el estado solución.

# 3. Busca heurística

Este tipo de búsqueda podemos decir que ya forma parte del terreno de la inteligencia artificial. Busca con información potencialmente útil [1].

Se emplea en el algoritmo de exploración, generado a partir de información relevante del problema que abordado, que puede no ofrecer una solución óptima en todas las ocasiones pero sí una estadísticamente "buena". De este modo, de forma general podemos evaluar los nodos a través de una función de evaluación:

$$f(n) = g(n) + h(n) \tag{1}$$

- g(n) es el coste del camino desde el nodo de inicio.
- h(n) es una función heurística que asigna a cada nodo un valor de utilidad estimada en la búsqueda de una solución (en general una distancia a un nodo solución).

Pueden existir varios criterios optimización. Por ejemplo: en el caso de la búsqueda de la ruta entre dos ciudades puedo emplear la distancia, el consumo de combustible, el tiempo...

## 3.1. Función de evaluación

Si desgranamos la función de evaluación observamos como contiene todos los tipos de busquedas vistos hasat ahora:

- f(n) = g(n): si no empleamos ninguna heurística, empleamos una función de coste uniforme que únicamente tiene en cuenta el coste desde el nodo inicio hasta el actual.
- f(n) = h(n): si únicamente empleamos la heurística, estamos ante un algoritmo primero el mejor (o el más prometedor). Que solo tiene en cuenta el menor coste estimado entre el nodo más prometedor y el nodo final. Sin tener en cuenta si ese estado intermedio lleva consumido un coste mucho mayor que otras alternativas, que pueden tener un coste mayor a la hora de llegar a la solución pero que consumirían mucho menos en llegar al punto en el que se encuentran.
- f(n) = g(n) + h(n): es el algoritmo $A$ que unifica el coste real con la estimación calculada.

## 3.2. Ejemplo

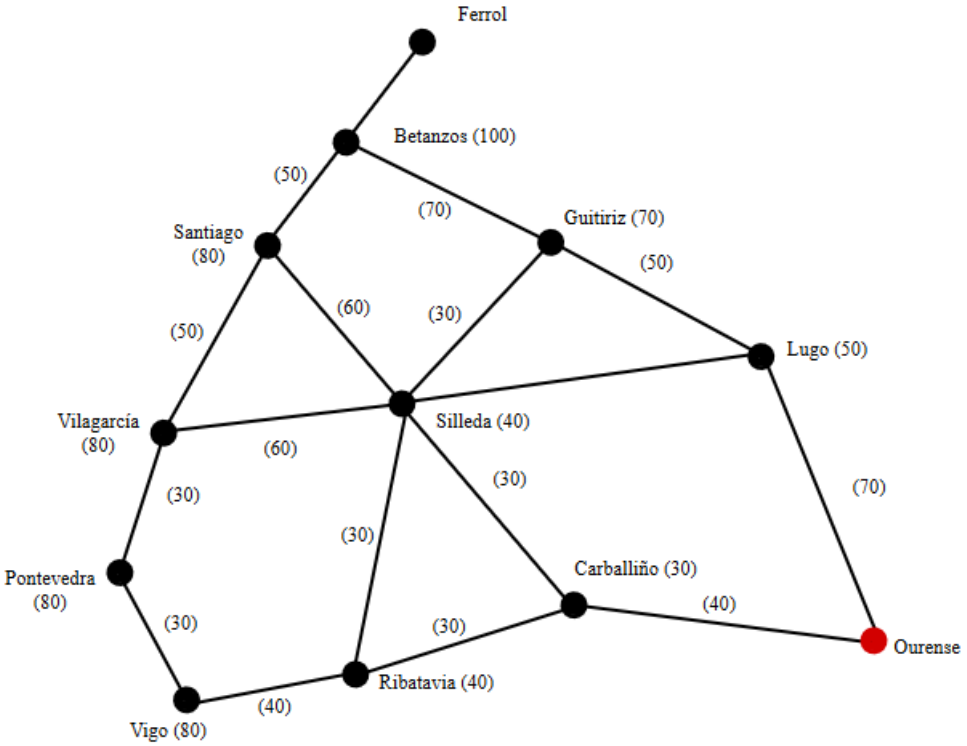
Ir de Ferrol a Ourense minimizando la distancia total de la ruta. Como estamos hablando de minimizar la distancia sabemos que la solución es única.

### Algoritmo A*

Para ello poseemos dos datos la distancia real entre ciudades (indicada en las aristas) y la distancia desde cada ciudad a Ourense en linea resta (indicada en cada vértice)[2].

[1] La primera práctica estará relacionada con la implementación de búsqueda en espacio de estados (A*), la teoría para la práctica se expondrá en estas páginas.

[2] Es importante destacar como esta es una heurística optimista pues, sabemos que en el mejor de los caso la distancia mínima será una linea resta, por lo que los recorridos serán siempre mayores o iguales a esta distancia mínima




**Función de evaluación** Dado que empleamos tanto datos desde la raíz hasta el nodo como estimaciones desde el nodo a la solucón la funcíon de evalución tiene la siguiente forma.

$$f(n) = g(n) + h(n)$$

Donde:

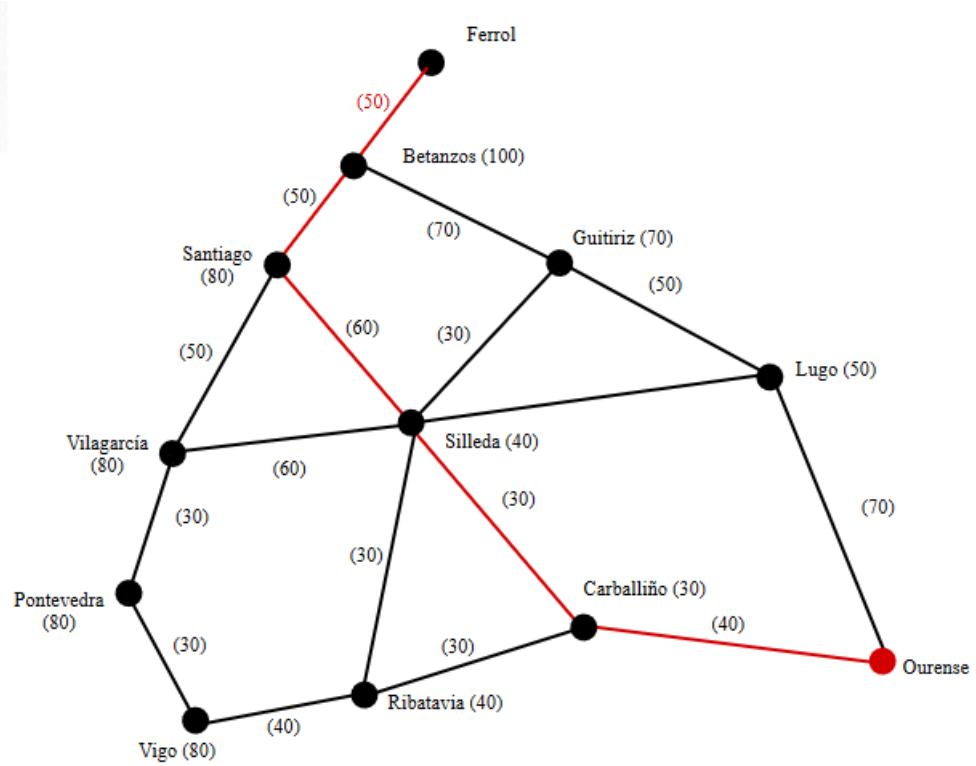
- g(n): es la distancia real de la ciudad a Ferrol.
- h(n): es la estimación de la distancia de la ciudad hasta Ourense. En este caso, se emplea la distancia en linea recta.

De este modo para cada nodo calculamos el valor de f(n). Por ejemplo:

$$f(Guitiriz) = g(Guitiriz) + h(Guitiriz) = (70 + 50) + (70) = 190$$

$$f(Santiago) = g(Santiago) + h(Santiago) = (50 + 50) + (80) = 180$$

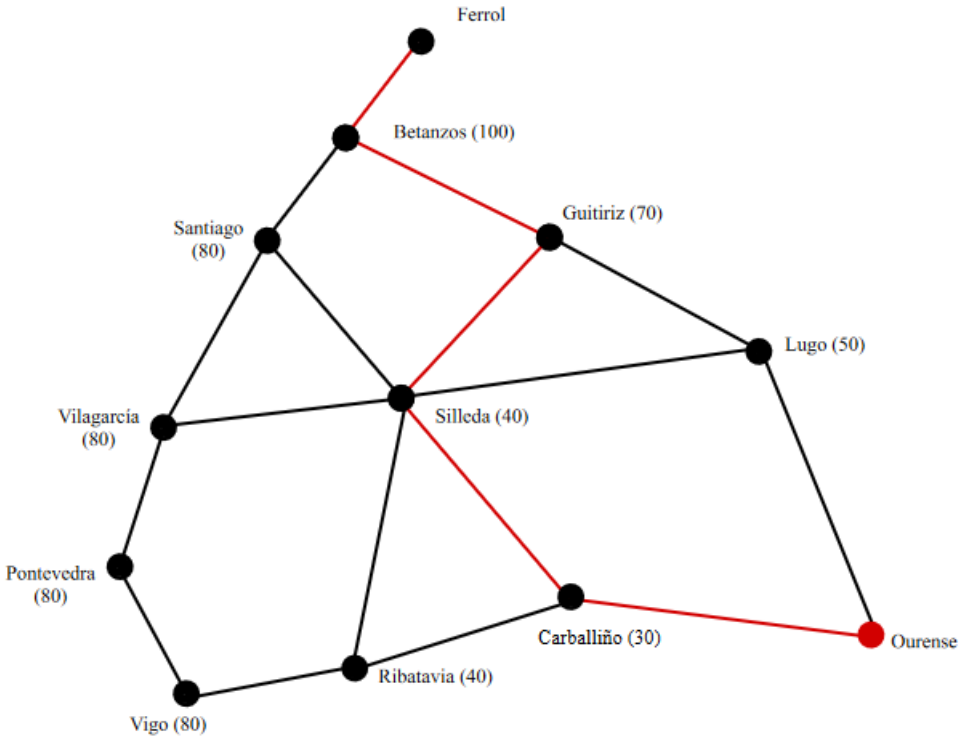
Así, entre la elección de ir Santiago o a Guitiriz desde Betanzos, elegimos Santiago. Repitiendo este mismo esquema en todas las elecciones de nodos obtenemos el siguiente camino:




Obteniendo una distancia real de: $50 + 50 + 60 + 30 + 40 = 230$

### Búsqueda avara

En este caso solo tenemos en cuenta la estimación heurística, no el coste previo de la ruta. De esta forma en casa decisión escogemos el nodo cuya distancia en linea recta con Ourense sea menor. De manera que el camino obtenido es el siguiente:




Obteniendo una distancia real de: $50 + 70 + 50 + 30 + 40 = 240$ (Info[3])

### Concluisiones

**Por lo general** (no siempre) el algoritmo A* ofrece mejores resultados que la búsqueda avara.

[3]Las distancias en entre vertices no se muestran en este grafo para recalcar que no se tienen en cuenta en la elección del recorrido, no obstante, sí se emplean en el cálculo de la distancia real. Los valores se muestran en el grafo que explica el algoritmo A*.

### 3.3. Heuristicas




Para un mismo problema pueden existir varias heurísticas válidas, en nuestro caso nos interesan las heurísticas optimistas donde el valor de la estimación es siempre mejor que el valor real. Su interés radica en que, existen casos donde aplicando heurísticas admisibles (optimistas) se puede demostrar formalmente que se llega a soluciones óptimas (nosotros no veremos la demostración).
A continuación mostraremos dos heurísticas optimistas para la estimación del coste de resolución del "8-quebracabezas".
Para calcular el número de movimientos necesarios para colocar las piezas en su lugar podemos contar:

1. Nº de piezas fuera de sitio: contamos el número de piezas que debemos recolocar y estimamos este como el número de movimientos necesarios para la reorganización.

2. Distancia Manhattan: contamos la distancia entre cada pieza y su posición siguiendo los caminos permitidos, sin tener en cuenta el resto de piezas a la hora de realizar los movimientos.

En caso de disponer de dos o más heurísticas óptimas parecidas, es posible emplear ambas en conjunto de forma que para cada nodo escogemos el valor más pesimista. Ya que, al poseer ambas siempre, un valor mejor al real, el peor de los valores será el más cercano a la realidad.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grao en Enxeñaría Informática

3º curso – 1º cuadrimestre

# Intelixencia Artificial

27-09-2021

Clase 05

Temas traballados: ningún

# Índice

1. Presentación oral ..... 3
   a) Tema ..... 3
   b) Grupos ..... 3
   c) Diapositivas ..... 3
   d) Opinión ..... 3
   e) Duración ..... 3
   f) Data ..... 3
2. Consellos e repaso ..... 3
   a) Primeira práctica ..... 3
   b) Repaso ..... 3
   c) Avance do próximo tema ..... 4

# 1. Presentación oral

## a) Tema

É recomendable ir pensando xa o tema para ir iniciando a investigación. Como non se poden repetir temas, as persoas que antes o digan terán a vantaxe de ter un abanico de opcións maior.

**Non se trata de expoñer aspectos tecnolóxicos, se non que a presentación debe tratar sobre os aspectos sociais, éticos, económicos, lexislativos... de aplicar a intelixencia artificial a algún ámbito**. A intelixencia artificial é unha tecnoloxía moi transformadora e que vai ser utilizada en todos os ámbitos da vida, polo que o noso traballo é medir os niveis do impacto.

## b) Grupos

Traballarase principalmente en parellas. Excepcionalmente, os grupos tamén poderán ser de unha persoa ou de tres persoas. **"Catro xa é como un botellón"** chégase a afirmar. Todas as persoas do grupo deben expor oralmente. Tamén se asegura que as partes dos distintos integrantes deben estar ben fiadas e non ser partes independentes.

## c) Diapositivas

**Cada presentación debe ter unha única diapositiva de apoio, non se poderán utilizar máis**. O contido que se pon na presentación ó final está competindo co presentador e se non está ben escollido pode xogar en contra. En absoluto se debe abusar de contido escrito, pois isto provoca que as persoas que están vendo a presentación se poñan a ler e perdan o fío do que está dicindo a persoa que presenta. Cada diapositiva non debe ter máis texto que un tweet.

## d) Opinión

O traballo pode abordarse dende unha perspectiva de divulgación, pero **o que se valora principalmente é a parte de pensamento, especulación, opinión...** Todo de forma fundada e defendendo unha perspectiva.

## e) Duración

**A duración da presentación debe estar entre os 5 e os 7 minutos. Despois de que esta remate, deixaranse entre 3 e 5 minutos de debate**, no que participarán tanto o profesor como o alumnado que estea na clase.

## f) Data

As presentacións terán lugar no momento no que rematemos as clases expositivas, que será a finais de novembro/comezos de decembro.

# 2. Consellos e repaso

## a) Primeira práctica

A primeira práctica vai estar baseada en A*. Polo que se recomenda que vexamos as notas e que interioricemos os conceptos para facelas.

## b) Repaso

Para poder aplicar algoritmos de busca precisamos movernos nun espazo de estados. Os máis fáciles de entender son os xogos coma o xadrez, pero tamén existen problemas reais nos que podemos aplicar técnicas baseadas en espazos de estados coma o do viaxante.

**Hai casos nos que a solución é meramente un estado, mentres que noutros o importante é a ruta que seguimos para chegar ó estado final.**

**O algoritmo A* valora cada estado mediante unha función, coa finalidade de non ter que percorrer todo o espazo**. Por un lado mídese o custo "seguro", de ir dende o espazo inicial ó estado que se avalía (Ex.: distancia dende a primeira cidade ata a cidade que se avalía no problema do viaxante). Por outro lado realízase unha estimación a través dunha ou varias heurísticas sobre o custo (ou beneficio) relativo ós estados que faltan por percorrer antes de chegar ó final. Se as heurísticas son admisibles, pódese demostrar que o algoritmo A* pode dar mellor solución. Estas heurísticas son optimistas, é dicir, sempre van supor casos nos que a realidade sexa un escenario peor que a estimación. (Ex. de estimación optimista: valorar a distancia en liña recta no caso do problema do viaxante). Estas heurísticas, ademais de ser optimistas, deben ser próximas á realidade. **Atopar boas heurísticas é unha parte complicada e complexa da resolución do problema.**

## c) Avance do próximo tema

O próximo tema vai sobre problemas que non son modelables mediante espazos de estados. Estes problemas son moi complexos e non existen algoritmos para resolvelos. Normalmente están ligados con situacións que na realidade son resoltas por expertos e especialistas, como por exemplo a detección de certas enfermidades por parte de un médico.

Pódese chegar a plantexar a pregunta (en realidade non sei que clase de persoa a pode plantexar vamos a ver) de por que queremos que as máquinas resolvan estes problemas se xa o fan os humanos? Pois ben, ademais de que ás veces simplemente funcionan como un apoio para facilitar o traballo, hai moitas máis causas:

- O rendemento proporcionado polos humanos depende do seu entorno e do seu estado nun momento determinado.
- Non todo o mundo pode acudir ós mellores especialistas.
- Hai tarefas nas que o rendemento das máquinas supera ó humano.
- Hai tarefas que son demasiado tediosas/perigosas como para ser desenvoltas por persoas.
- ...

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

# Grado en Ingeniería Informática

## 3º curso – 1º cuatrimestre

# Inteligencia Artificial

29-09-2021

Clase 06

Temas trabajados:

Tema 3

# Índice

1. Sistemas baseados en coñecemento ..... 3
a) Sistemas expertos = sistemas basados en coñecemento ..... 3
i. Primeiros sistemas expertos ..... 3
ii. Exemplos de dominios ..... 3
iii. Cando usar un sistema experto? ..... 4
iv. Como se deseña? ..... 4
v. Como funciona? ..... 4
vi. Exemplo ..... 5
vii. Estructura dun sistema experto ..... 5
viii. Sistemas baseados en regras ..... 6
ix. Cadro comparativo: Coñecemento vs feitos ..... 13
x. Cadro comparativo: Programación convencional vs Sistemas Expertos ..... 14
xi. CLIPS ..... 14
xii. Lóxicas ..... 14
xiii. Dúbidas ..... 15

# 1. Sistemas baseados en coñecemento

## a) Sistemas expertos = sistemas basados en coñecemento

- A denominación de sistemas basados en coñecemento ten un ámbito máis xeral.
- Nivel de competencia equivalente ou superior a un experto humano nun dominio concreto do saber (por exemplo un médico no ámbito da medicina).
- Representación e utilización – razoamento- de coñecemento explícito.

Ter representado o coñencemento non é suficiente, necesitase ademais un mecanismo sistemático de aplicación do coñecemento (razoamento automático). Este mecanismo sistemático dependerá de cómo se represente o coñecemento.

### i. Primeiros sistemas expertos

Os primeiros sistemas expertos son dos anos 60, 70 e 80. Nos anos 80, como xa había certo éxito comezaron a nacer empresas de bases tecnolóxicas orientadas a aportar este tipo de solucións aos problemas pero houbo un estancamento a nivel da investigación.

Algúns dos primeiros sistemas expertos máis coñecidos foron:

- **Dendral**, universidade de Stanford, 1965-1975; identificación de compostos orgánicos sober espesctroscopía de masas. É dicir, a espectroscopía de masas danos información da relación masa/ carga (unha información) da que se pode derivar a composición certa substancia á analizar.
- **Mycin**, Stanford Research Institute, anos 70; identificación de bacterias causantes de infeccións.

### ii. Exemplos de dominios

- Diagnóstico médico
- Análise financeiro: saber donde invertir un capital cun determinado interés,e cunhas determinadas condicións(sen risco, certo interés…)
- Control e monitorización de procesos
- Educación -titorización intelixente…-: como aprender mellor?En que cousas non temos solvencia?...
- Diagnóstico de fallos e mantemento de máquinas

### iii. Cando usar un sistema experto?

- Problemas en dominios complexos e especializados: expertos en un tipo de problemas. Hay persoas que, tras unha experiencia permite ser moi competente resolvendo certo tipo de problemas. Estes sistemas axudarán as persoas que carezan deste dominio do problema.
- Non existe unha aproximación algorítmica ou unha forma de aprender a resolver o problema mediante aprendizaxe automática. Se teño un problema que se pode transformar en diagrama de estados entonces non é o máis óptimo.
- Existe coñecemento para a súa solución, tanto documentado como aportable por expertos humanos

### iv. Cómo se deseña?

Nos anos 80, a investigación no sistemas basados en coñecemento deu lugar a creación dalgúns programas con éxito que permitiron incormporar esta tecnoloxía no uso cotiá polo que empezou a haber certa actividade economica e profesional neste ámbito, os profesionais que se dedicaban a construir os sistemas expertos chamábaselle enxeñeiros do coñecemento.

- Obtención do coñecemento potencialmente útil sobre o problema e a súa solución por parte dun "enxeñeiro do coñecemento"
- Representación do coñecemento dun modo computacionalmente tratable para o seu uso en procesos de "razoamento" en máquinas

Un dos principais problemas dun sistema basado en coñecemento é o mantemento do mesmo, xa que o coñecemento avanza e mellorase; e neste caso, habría que cambiar e modificar o sistema experto.

### v. Cómo funciona?

Un sistema basado en coñecemento ten base nos seguintes tres bloques:

- **Base de feitos**: o que se sabe sobre o caso que esté sendo considerado dentro da tipoloxía de problemas abordados polo sistema experto.Feitos coñecidos sobre un determinado problema nun determinado momento.
- **Base de coñecemento:** coñecemento potencialmente útil para a resolución de problemas no dominio específico considerado. Método

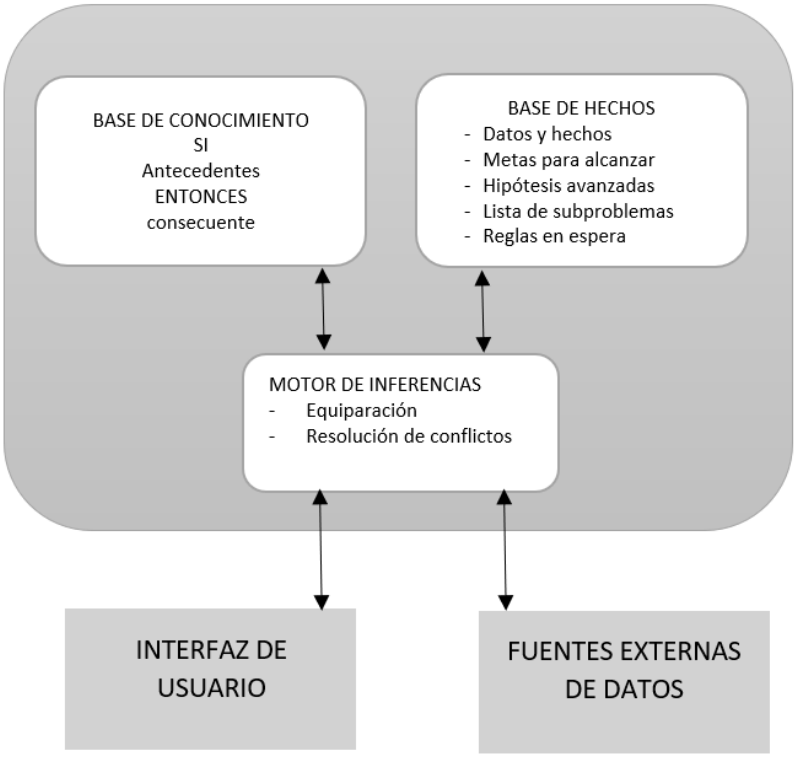
sistemático que contrasta os datos do caso particular e coñecemento sobre o problema, en base a esa aplicación, suponse que ese mecanismo de razoamento permitirá chegar a unha conclusión.

- **Mecanismos de razoamento**: aplicar o coñecemento aos feitos para obter unha solución ao problema abordado.

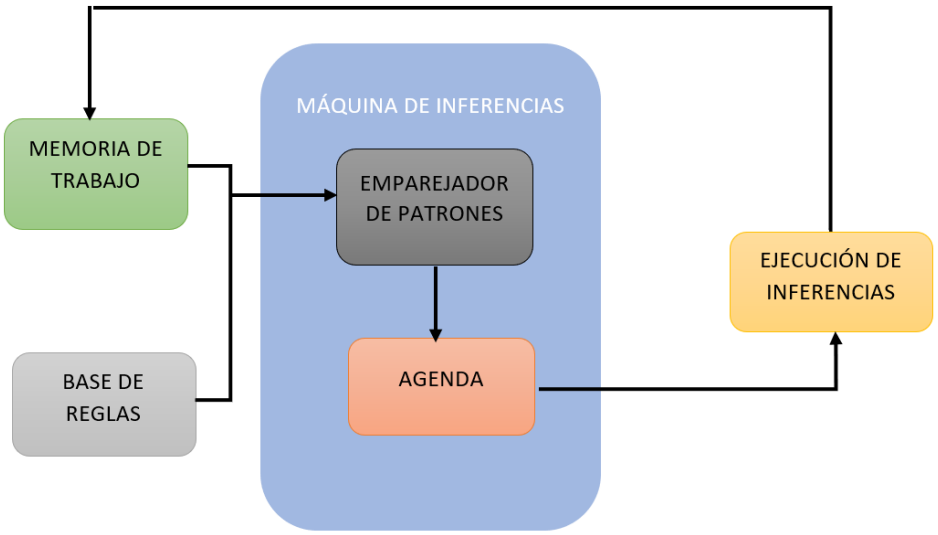
### vi. Exemplo

- Coñecemento: se a tempperatura corporal é superior a 37 graos, entonces conclúo que o paciente ten febre
- Feito: temperatura do paciente é de 38.3 graos
- Conclusión: "o paciente ten febre", se o mecanismo de razoamento automático aplica a regra e contrastaa co feito, entonces verifícase a parte condición da regra, enton conclúese a parte consecuente da regra.
- A ter en conta: os feitos son dinámicos, e tamén pode selo o coñecemento; dependen do momento no que se determinen os feitos.

### vii. Estructura dun sistema experto



- **Compoñentes principais**




## viii. Sistemas baseados en regras

- **Regra de producción:**

  SE situación ENTÓN acción

As regras de produción é a forma máis común de representar de coñecemento, é simple de explicar, de entender….

- **Situación:** condicións a satisfacer
- **Accións típicas:**

  As conclusións poden ser de calquera tipo, pode ser que conclúa nun valor para unha certa variable, pode ser que suprima algo da memoria de traballo(donde se atopan os feitos coñecidos e inferidos sobre o caso concreto que se analiza), imprimir un elemento, pedir un dato…

  - Engadir algún feito á memoria de traballo
  - Suprimir algún feito da memoria de traballo
  - Executar algún procedemento
- **Encadeamento cara adiante:**

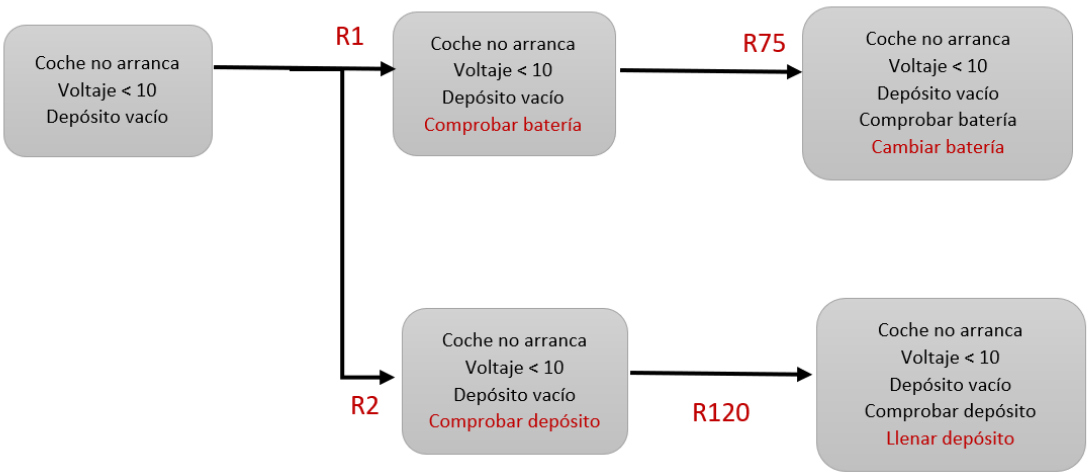
  As regras son pezas de coñecemento independentes, estan desacopladas e pódense aplicar sempre que a súa parte condición se verifique, da igual a orde na que se introduzan as mesmas.

  Ainda que sexan independentes poden ter unha relación entre elas, se unha regra conlúe algo e outra a ten na súa parte condición estarán relacionadas.

As regras de descripción (producción) diferenciase das regras da lóxica, en que nesta non falamos de regras de producción aínda que se pareza na forma de estructurar o coñecemento nunha lóxica. Manipúlanse feitos sobre os que necesariamente o seu valor é verdadeiro ou falso ao igual que as suas conclusións.

- Pártese dos feitos na memoria de traballo
- Emparéllanse os feitos cos antecedentes das regras
- Aplícanse regras até que se acada un obxectivo ou non se pode seguir
- Aplicación de criterios de selección das regras aplicables dentro do "conxunto conflicto"
- Principio de refracción. Non aplicación reiterada dunha regra aplicable
- Exemplo 1:

```
Regla 1. IF coche no arranca,
         THEN comprobar batería
Regla 2. IF coche no arranca
         THEN comprobar combustible
...
Regla 75. IF comprobar batería
          AND voltaje batería < 10V
          THEN cambiar batería
...
Regla 120. IF comprobar combustible
           AND depósito de combustible vacío
           THEN llenar depósito.
```



- Exemplo 2:

```
Regla 0. IF hay placas (puntos blancos) en la garganta
         THEN diagnóstico: posible infección de garganta
Regla 1. IF garganta inflamada
         AND sospechamos infección bacteriana
         THEN diagnóstico: posible infección de garganta
Regla 2. IF temperatura paciente > 39
         THEN paciente tiene fiebre
Regla 3. IF paciente enfermo más de una semana
         AND paciente tiene fiebre
         THEN sospechamos infección bacteriana
```

  - Memoria de traballo inicial
    - Temperatura= 40
    - Enfermo desde hay dúas semanas
    - Garganta inflamada

| M.T | C.C. | Resolución |
|---|---|---|
| Temperatura=40<br>Enfermo dos semanas<br>Garganta inflamada | R2 | R2 |
| Temperatura = 40<br>Enfermo dos semanas<br>Garganta inflamada<br>Fiebre | R3 | R3 |
| Temperatura = 40<br>Enfermo dos semanas<br>Garganta inflamada<br>Fiebre<br>Posible infección bacteriana | R1 | R1 |

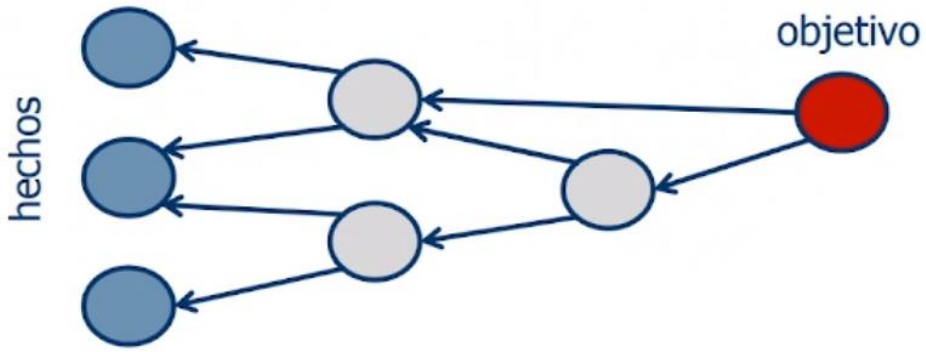
Para o primeiro caso, aplicamos as regras (conxunto conflicto: todas as regras que se poden aplicar nese momento).

No primeiro caso aplicamos a R2-> o usuario ten febre

Sobre a nova fase de feitos pódense apricar outras regras (non as xa aplicadas, xa que non se aplican de forma reiterada), neste caso ao levar o paciente enfermo dúas semanas e ter febre-> infección bacteriana.

Se non é concluinte, teranse que realizar máis probas ben seleccionadas, non escollidas ao azar para determinar a enfermidade do paciente.

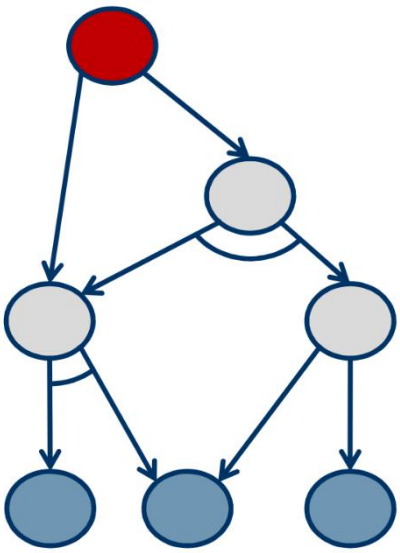
- **Encadeamento cara atrás:**




É bastante común a forma de estar conectadas as regras se debuxen como un grafo.

Un proceso de encadeamento cara atrá deríbase normalmente de que hai unha pregunta dun usuario a un sistema experto, por exemplo *"ten o usuario infección de garganta?"*.

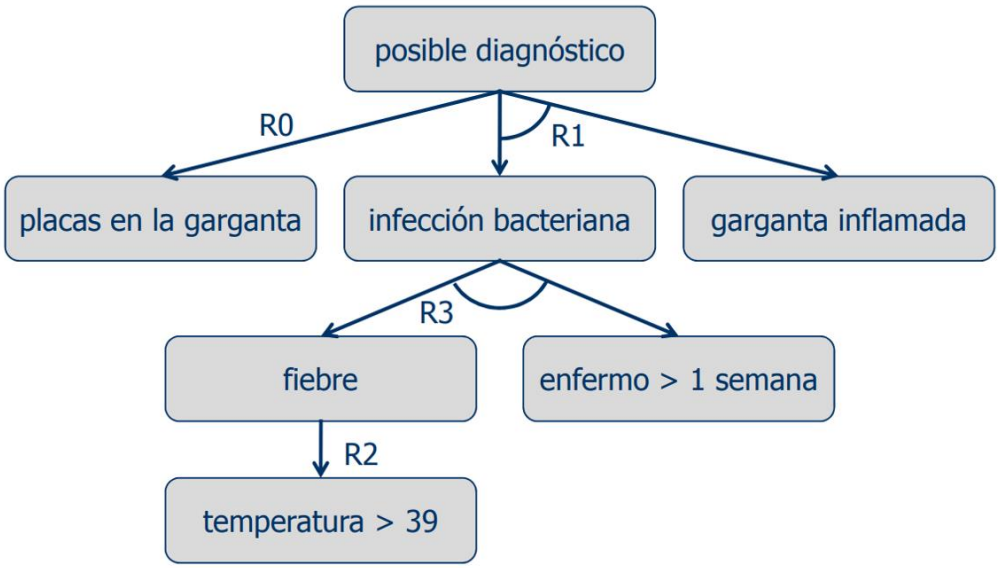
O razoamento cara diante e cara atrás son as dúas formas usuais de sistematizar o razoamento automático sobre as regras. O razoamento cara adiante é dirixido polos datos mentres que o cara atrás é dirixido polas preguntas dos usuarios ou especulacións.



Cando se usa a representación de grafo, se poñemos un arco entre duas conexións quere dicir que é un 'and', 'y' ou conexión; entonces ambas se teñen que dar. Se non aparece quere dicir que é un OR ou suma, é dicir, non se teñen que dar as dúas de forma síncrona.

Loxicamente tamén se pode utilizar operadores de negación combinados cos operadores anteriormente mencionados (and e or).

Para ilustrar mellor o que é o proceso de encadeamento cara atrás, supoñamos que temos nunha estructura de grafos unha parte das regras de producción dun sistema de diagnóstico de enfermidades infecciosas en persoas.



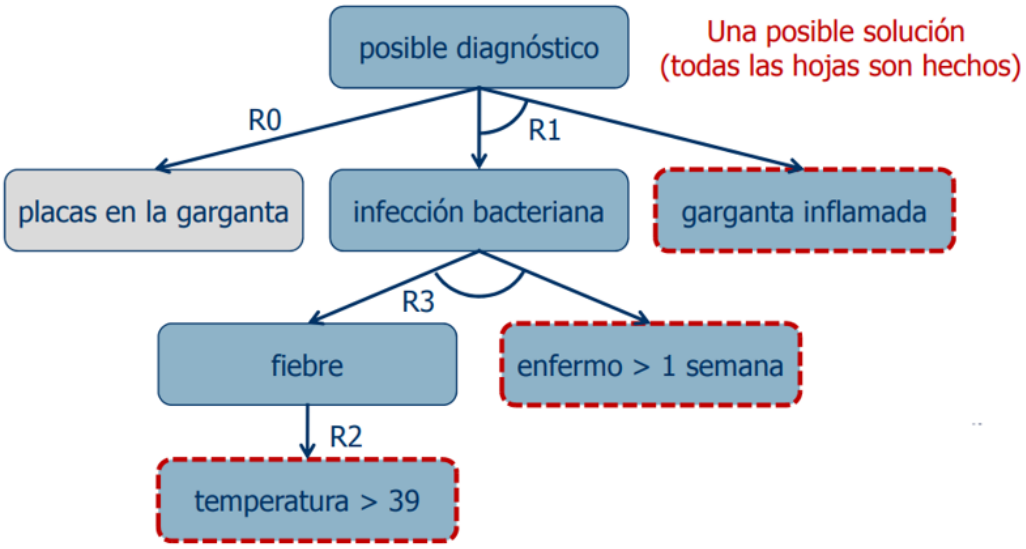

No anterior debuxo temos representado unha serie de regras que son de fácil lectura, por exemplo R2 dí que se a temperatura é maior que 39 entonces ten febre.

Aquí o razoamento cara atrás significaría que quero chegar a un diagnóstico e por tanto comezo a ver como se pode afirmar algo en función da estructura de coñecemento. Poderíase facer a traves de R0 se sabemos que cumple a condición,ou a través de R1 se as suas regras son cumplidas de forma simultánea.

Polo tanto, o que se intentaría, será tratar de verificar que alfunha das condicións é certas, ben R0 ou R1 e R2 e R3.

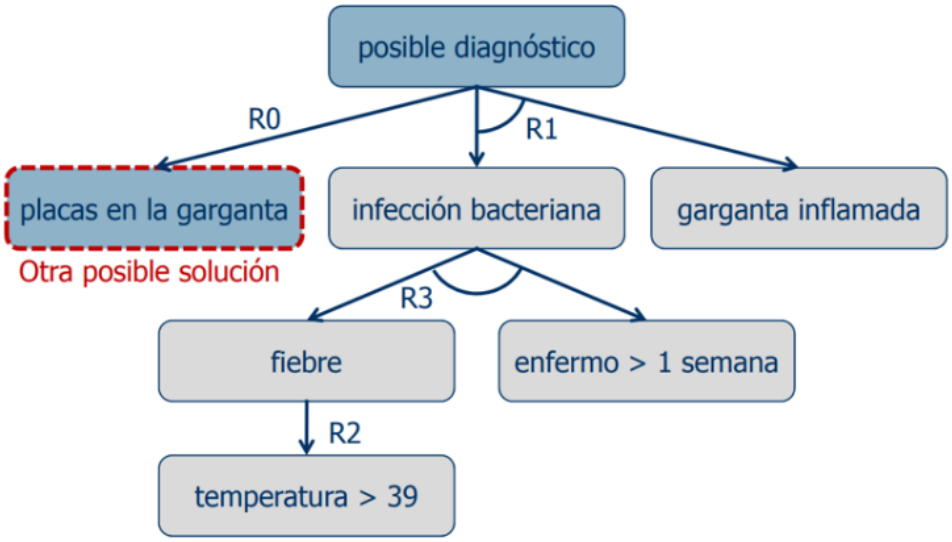
As posibles solucións serían:

1-




Para que esta solución sexa válida os cadrados vermellos terán que cumplirse de forma simultánea.

2-




O sistema de razoamento automático vai explorando cunha sistemática que en principio nos permite chegar a unha conclusión válida.

**Pseudocódigos:**

- Método básico de razoamento

```
1: BH = HechosIniciales;
2: mientras NoVerificaCondiciónFinalización(BH) o NoseEjecutaAcciónde Parada hacer
3:    ConjuntoConflicto = Equiparar(BC,BH);
4:    R=Resolver(ConjuntoConflicto);
5:    NuevosHechos = Aplicar(R,BH);
6:    Actualizar(BH,NuevosHechos);
7: fin mientras
```

- Verificación de regras

```
1: Verificado=Falso;
2: si Contenida (Meta,BH) entonces
3:    devolver Verdadero;
4: si no
5:    ConjuntoConflicto = Equiparar(Consecuentes(BC),Meta);
6:    mientras NoVacío(ConjuntoConflicto) y No(Verificado) hacer
7:       R=Resolver(ConjuntoConflicto);
8:       Eliminar(R,ConjuntoConflicto);
9:       NuevasMetas=ExtraerAntecedentes(R), Verificado=Verdadero;
10:      mientras NoVacío(NuevasMetas) y Verificado hacer
11:         Meta=SeleccionarMeta(NuevasMetas);
12:         Eliminar(Meta, NuevasMetas);
13:         Verificado=Verificar(Meta,BH);
14:         si Verificado entonces
15:            Añadir(Meta,BH);
16:         fin si
17:      fin mientras
18:   fin mientras
19:   devolver(Verificado);
20: fin si
```

- Encadeamento cara adiante

```
1: BH = HechosIniciales, ConjuntoConflicto = ExtraeCualquierRegla(BC);
2: mientras NoContenida(Meta,BH) y NoVacío(ConjuntoConflicto) hacer
3:    ConjuntoConflicto = Equiparar(Antecedentes(BC),BH);
4:    si NoVacío(ConjuntoConflicto) entonces
5:       R=Resolver(ConjuntoConflicto);
6:       NuevosHechos = Aplicar(R,BH);
7:       Actualizar(BH,NuevosHechos);
8:    fin si
9: fin mientras
10: si Contenida(Meta,BH) entonces
11:    devolver "éxito";
12: fin si
```

- Encadeamento cara atrás

```
1: BH = HechosIniciales;
2: si Verificar (Meta,BH) entonces
3:     devolver "éxito";
4: si no
5:     devolver "fracaso";
6: fin si
```

O que interesa é que se entendan os conceptos e o modo de aplicalos, non é necesario saber o pseudocódigo.

### ix. Cadro comparativo: Coñecemento vs feitos

| Feitos | Coñecemento |
|---|---|
| Específicos -do problema concreto- que se intenta resolver. | De carácter xeral – do dominio no que se esta operando e na tipoloxía dos problemas- |
| Dinámicos, a partir duns feitos de partida podense evidenciar outros distintos | Relativamente estáticos, pode cambiar no tempo porque se aporta novo coñecemento |
| Aumentan durante a resolución do problema | En xeral non aumenta durante a resolución do problema |
| Necesidade de almacenamento e recuperación eficientes | Necesidade de razoamento eficiente |
| Búscase que os feitos obtidos directamente do problema abordado sexan precisos e certos | Pode ser impreciso e incerto; polo tanto, tamén os feitos inferidos |

### x. Cadro comparativo: Programación convencional vs Sistemas Expertos

| Programación convencional | Sistemas expertos |
|---|---|
| programación imperativa, instruccións nun orde, cada liña de código é algo imperactivo, un mandato. | Programación declarativa, conxunto de regras potencialmente útil, pero non ten que avaliarse todas nin nun orde determinado. |
| Modificación por reprogramación, reprogramase e recompilase. | Modificación de base de coñecemento, hay unha base de datos singular, e cambianse as regras non se recompila. |
| Solución algoritmica. | Solución por razoamento basado en coñecemento. Problemas complexos |
| Normalmente solución precisa e certa -solución óptima- | Normalmente imprecisa e con garos de certidumbre -solución probable, posible…- |
| Execución guiada polo gruxo de execución do código. | Execución guiada polo motor de inferencia ou mecanismo de razoamento automático. |

### xi. CLIPS

É unha platafoma de carácter didáctico para deseñar sistemas basados en coñecemento, xeralmente sistemas basados en regras, e usarase na seguinte práctica a de A*.

- C Language Integrated Production System
- NASA Jhonson Space Center, 1985-
- Gary Riley
- http://clipsrules.sourceforge.net/

### xii. Lóxicas

- Representación formal das relacións existentes entre conceptos, obxectos, propiedades, valores…
  - Lóxica proposicional
  - Lóxica de predicados
  - Lóxica difusa
  - Lóxicas temporais
  - Lóxicas non monótonas
  - …

## xiii. Dúbidas

- **A técnica de razoamento cara atrás non é máis ineficiente que o razoamento cara adiante?**

  Non, é ao revés, o razoamento cara atrás normalmente implica explorar menos ramas do grafo.

  Co razoamento cara adiante hay que construir a partir de todos os datos os grafos correspondentes que derivan deles.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Asignatura

04-10-2021

## Clase 7

Temas trabajados: Lógicas y sistemas conexionistas

# Índice

1. Lógicas ..... 3
2. Lógica proposicional ..... 4
3. Lógica de predicados ..... 4
4. Redes semánticas ..... 5
5. Sistemas conexionistas ..... 5

# 1. Lógicas

Esta parte la vemos para ver alguna referencia al desenvolvimiento histórico de la IA en lo que tiene que ver con la representación del conocimiento, es decir, sistemas que intentan reproducir mecanismos de razonamiento humano bajo la idea de la **hipótesis de símbolos físicos:** nuestro cerebro es una máquina capaz de manejar conceptos abstractos, símbolos, que tienen un significado construido, en general, de manera colectiva a través de la historia, la experiencia…

Los lenguajes naturales que son una representación simbólica de información, de conocimiento, de hechos. Son una construcción humana, una herramienta, probablemente la más importante que tenemos, porque a través de él somos capaces de avanzar en el conocimiento, de transmitirlo, de un modo que sería impensable sin el lenguaje. Un lenguaje natural es un ejemplo muy evidente de lo que es representar a través de símbolos una serie de palabras de manera que signifique algo, que tenga sentido.

A partir de la hipótesis de símbolos físicos podemos también decir que otras máquinas, como las computadoras, pueden manejar sistemas donde representamos a través de símbolos cualquier cosa y, a través de una manipulación acertada de esos símbolos, podemos resolver problemas o razonar de forma similar a como razonamos las personas.

Una computadora es un sistema de manipulación de símbolos, al final cualquier símbolo, cualquier pieza de información se puede representar en 0s y 1s. Es un medio para darle un soporte físico a procesos de manipulación simbólica que tienen una utilidad real. Todas las aproximaciones dentro del contexto de los sistemas basados en conocimiento parten de esta idea: representamos conocimiento a través de símbolos.

No sabemos cómo computar el lenguaje natural y por ello tenemos que elaborar lenguajes menos expresivos pero tratables computacionalmente. Las reglas de producción son una forma de hacer esto, las lógicas son otra, pero no son equivalentes, aunque se parezcan, ya que en el ámbito de las lógicas hablamos de una formalización de todos los conceptos, sus relaciones y los procesos de decisión a partir de ellos. Cuando hablamos de reglas de producción, las manejamos de un modo más laxo. Por ejemplo, aplicar el razonamiento hacia delante no es en sentido estricto equivalente al modus ponens como proceso deductivo sobre una lógica.

Cuando los sistemas expertos empezaron a tener cierto éxito, se pretendía además de usar reglas de producción usar lógicas. Sin embargo, la lógica proposicional tiene una muy mala capacidad de representación. Por ello, se fueron desenvolviendo lógicas que trataron de ampliar la cobertura de lo representable a través de la lógica. Por lo tanto, ahora algo que podamos representar con una lógica es relativamente fácil pasarlo al dominio computacional, igual que algo que podemos representar matemáticamente es relativamente fácil tratarlo computacionalmente. Podemos decir que la lógica es una matematización de los procesos de representación de conocimiento y racionamiento.

La principal diferencia entre la lógica proposicional y la **lógica de predicados** es que cuando trabajamos con una lógica de predicados podemos representar categorías o clases de elementos, en la proposicional no, tenemos que hablar sobre un individuo en concreto.

La **lógica difusa** (1965, “Fuzzy sets”) es una lógica que trata con conjuntos borrosos. En un conjunto clásico, los elementos pertenecen o no. En un conjunto difuso puedo definir grados de pertenencia, se puede pertenecer en plenitud a un conjunto o se puede pertenecer en un grado que puedo cuantificar, por ejemplo, entre 0 y 1. El principal interés en esta lógica deriva de la

facilidad que tiene de representar términos borrosos, muy comunes en el lenguaje natural. Por ejemplo, decir: "Fulanito es muy alto". ¿Qué significa "muy alto"? En el lenguaje natural no haríamos una distinción drástica como decir que muy alto es medir más de 1,90m, pero 1,89m no es ser muy alto. Nos sentiríamos más cómodos con una transición más gradual, una curva de transición, de manera que hay alturas que consideramos que claramente cumplen con la etiqueta de muy alto y hay valores que no la cumplen en absoluto, y entre los dos, toda una transición suave de valores que, a medida que aumenta la altura, van cumpliendo en mayor grado esa etiqueta. Si somos capaces de matematizar esto, tendremos una forma mejor de trasladar al ámbito computacional expresiones muy propias del lenguaje humano. Si solo pudiésemos trabajar con etiquetas drásticas, tendríamos una peor representación de conceptos como el anterior.

**Las lógicas temporales** permiten una manipulación a nivel de representación y razonamiento con hechos temporales.

Las **lógicas no monótonas** son aquellas que permiten "desdecirse". Uno de los problemas que puede haber cuando tenemos un sistema basado en conocimiento es que, al realizar inferencias y obtener nuevos hechos que vamos metiendo en la base de hechos, puede pasar que, por nuevas evidencias, el sistema pueda negar algo que fue afirmado anteriormente. Un hecho que antes era considerado como cierto, puede llegar a negarse. Resolver esto no es tan fácil como eliminar el hecho de la BH, porque a partir de su introducción en la BH se pudieron haber derivado muchas otras cosas a partir de él, a su vez teniendo consecuencias en otros hechos, y habría que deshacer todo esto. Las lógicas no monótonas permiten negar lo que previamente fue afirmado, o al revés, manteniendo la coherencia del sistema.

# 2. Lógica proposicional

Manejamos hechos que pueden ser verdaderos o falsos. No podemos trabajar con conjuntos, solo con individuos o conceptos, por lo que es muy poco expresiva. Utiliza operadores lógicos: conjunción (y), disyunción (o), negación (no), implicación lógica (relación de consecuencia). Los mecanismos de razonamiento deductivo más común son el modus ponens y el modus tollens.

Modus ponens: si p y p implica q, entonces q.

Modus tollens: si no q y p implica q, entonces no p.

# 3. Lógica de predicados

Permite manejar constantes (lo que serían las proposiciones en la lógica clásica) y predicados. Los predicados son etiquetas que categorizan a conjuntos de constantes. Incluso podemos establecer relaciones entre dos o más elementos. Permite otros operadores como los cuantificadores, por ejemplo, para todo, existe….

Podemos construir piezas de conocimiento, por ejemplo:

$$\forall x\ \forall y\ \forall z$$
$$\text{es-padre}(x,y) \wedge \text{es-padre}(y,z) \rightarrow \text{es-abuelo}(x,z)$$

El que asigna un significado a este conjunto de símbolos es el diseñador del sistema. A partir de estos significados sabemos cómo podemos manipular estos símbolos.

# 4. Redes semánticas

En ellas la información se relaciona entre sí, de manera que tenemos procesos de recuperación más eficientes y de razonamiento sobre el conjunto de información que manejamos, por ejemplo, se permite con mucha facilidad la herencia de propiedades, estableciendo que una subcategoría forma parte de una categoría de ámbito más general, y que estas subcategorías heredan todas o algunas de las características de la categoría superior. Esto es útil desde el punto de vista del razonamiento ya que significa que no tengo que representar absolutamente todo de cada elemento, sino que puedo representar conjuntos y relaciones entre ellos, herencia de propiedades entre ellos, y por lo tanto tengo mecanismos de representación que soportan un razonamiento mucho más ágil y rico.

# 5. Sistemas conexionistas

Se busca la bioinspiración: buscar crear máquinas cada vez más inteligentes inspirándose en la biología. De ahí surge la computación neuronal, los algoritmos genéticos, los algoritmos de optimización como el de colonia de hormigas (tratar de imitar el comportamiento de sociedades que individualmente son elementales pero colectivamente tienen una muy buena cooperación).

También se aplica la tecnoinspiración: la potencia de cálculo de las máquinas y su propio funcionamiento podrían resultar, no solo como instrumento, sino como inspiración o modelado de sistemas de inteligencia humana, como mecanismos de razonamiento. Se les llamaba a las computadoras “cerebros electrónicos”.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

## Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Inteligencia Artificial

*11-10-2021*

Autores:

[redacted]

**Clase 9**

Temas trabajados:
Tema 4 - Sistemas Conexionistas

# Índice

**1. Neurona Natural** **2**

**2. Neurona artificial** **2**

2.1. Modelo más simple de una neurona artificial . . . . . 2

2.2. Neurona binaria con umbral . . . . . 3

2.3. Neurona Lineal Rectificada . . . . . 3

2.4. Neurona Sigmoidal . . . . . 3

**3. Representación común de una Neurona Artificial** **4**

**4. Marcas temporales importantes** **4**

**5. Sistemas conexionistas** **5**

**6. Nota** **5**

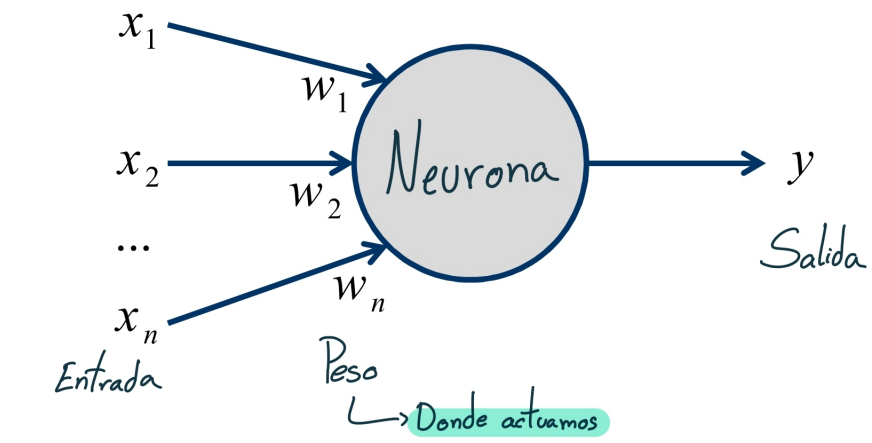
# 1. Neurona Natural

Una neurona natural funciona como una función de integración espacio-sensorial que recibe estímulos externos de otra neurona a través de neurotransmisores. El impulso eléctrico se transmite a una velocidad de 30 m/s (una velocidad muy lenta comparada con la que podemos conseguir en un circuito integrado).

El potencial de acción de una neurona comienza a un bajo nivel (Umbral de estimulación) y aumenta al llegar un estímulo (Pico de potencial de acción), después, se empieza a reducir (repolarización) hasta llegar a un nivel inferior que el inicial (Hiperpolarización).

# 2. Neurona artificial

## 2.1. Modelo más simple de una neurona artificial

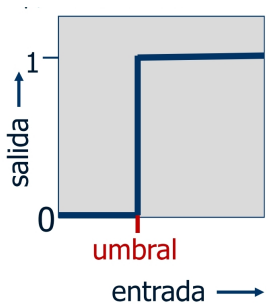



$$y = \sum_i x_i w_i = x_1 w_1 + x_2 w_2 + \ldots + x_n w_n$$

Suma de las entradas ponderadas

## 2.2. Neurona binaria con umbral

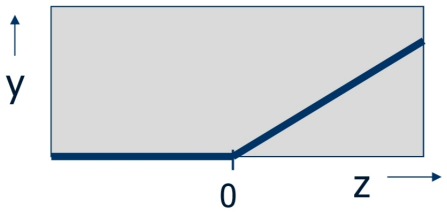
Definida por McCulloc & Pitts en 1943. La ecuación es de la forma $z = b + \sum_i x_i w_i$.
Donde Umbral $\theta = -b$ y de tal forma que si el valor de z es mayor o igual que 0 el valor de $y$ es 1 y en los demás es 0 (Función Escalón)




Gracias a esto se puede implementar ciertas funciones lógicas. Sin embargo tiene un gran inconveniente y es que esta función no es derivable en todos los puntos, lo cual es un problema para los algoritmos de aprendizaje.

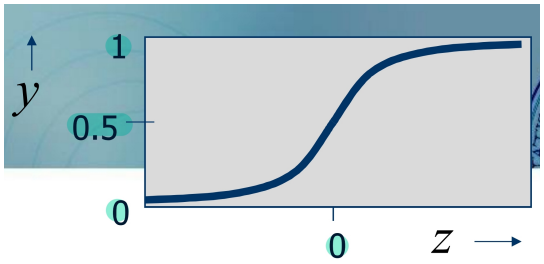
## 2.3. Neurona Lineal Rectificada

$z = \sum_i x_i w_i$, de tal forma que si $z \geq 0$ entonces $y = z$ y en otro caso $y = 0$.
Asumiendo $x_0 = 1$ y $w_0 = b$ (umbral $\theta = -b$)



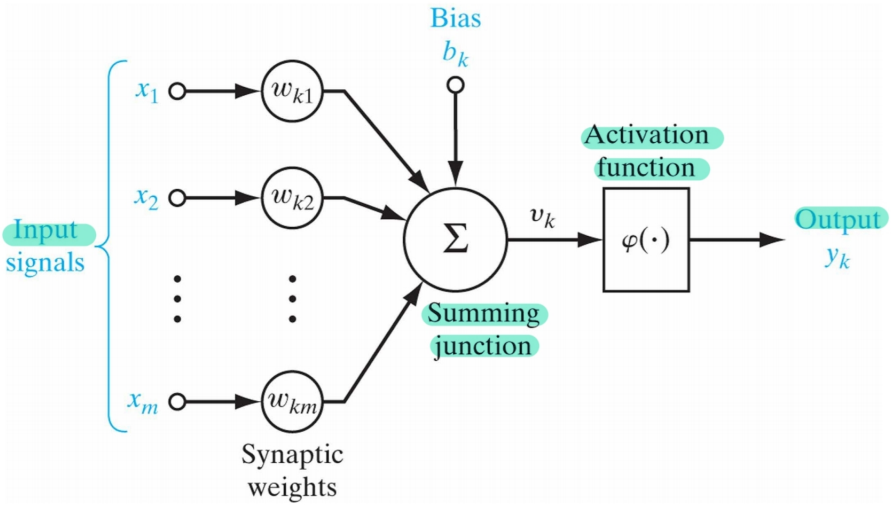

## 2.4. Neurona Sigmoidal

$z = \sum_i x_i w_i \qquad y = \frac{1}{1+e^{-z}}$

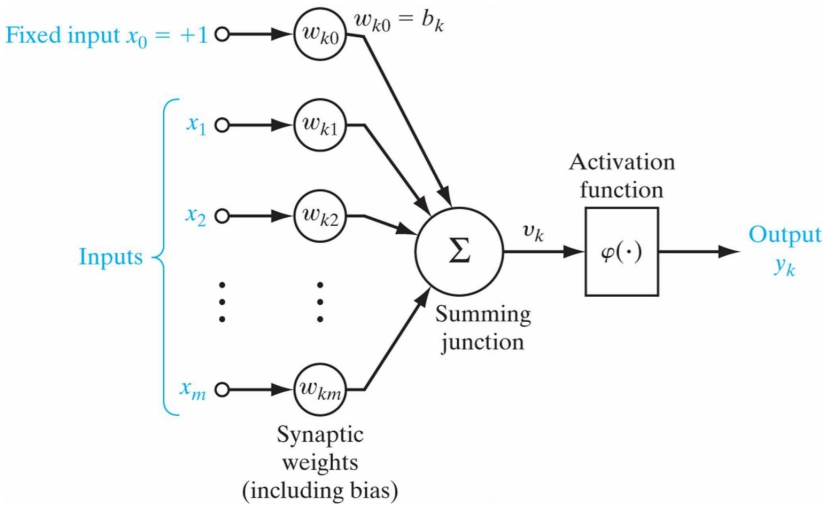



Es el modelo de neurona más utilizado ya que es una función derivable en todos los puntos.

## 3. Representación común de una Neurona Artificial




Representación común de una Neurona Artificial




Neurona Artificial con *bias* integrado como un peso

## 4. Marcas temporales importantes

- 1943: McCulloch y Pitt definicion formal de neurona.
- 1957: Frank Rosenblatt, desarrolla el algoritmo de perceptrones que permite obtener los valores de los pesos en las entradas de la neutra para obtener la salida deseada (Para una única capa de neuronas).
- 1969: publicación del libro "Perceptrones", Minshy y Papert

# 5. Sistemas conexionistas

Métodos de aprendizaje de un RNA con *Perceptrones*

1. **Aprendizaje supervisada**: es aquella donde se opera con pares de valores dato-salida deseada para ese dato.
   De este modo, si se quisiera diseñar un sistema para reconocer gatos en fotografías y se poseerá una base de datos representativa y suficientemente amplia con imágenes de ejemplo. Sería posible codificar una red con 100 neuronas en las que las salidas de todas a 0 significarían el reconocimiento de un gato. De este modo, es necesario encontrar los valores para las ponderaciones de las entradas que establezcan un valor cercano a 0 para todo el conjunto de entrenamiento. Así se emplean los siguientes elementos:

   - Conjunto de entrenamiento: formado por pares (entrada, salida deseada), suficientemente representativo del problema a resolver.
   - Ajuste de pesos: el entreno de las redes se realiza ajuntando los pesos de las entradas de los perceptrones para que en la medida de lo posible la red responda con la salida deseada ante cada ejemplo del conjunto de entrenamiento.

   De esta manera funciona a partir de ejemplos (conjunto de entrenamiento)

2. **Aprendizaje no supervisado**: no se le comunica a la red como debe responder al conjunto de datos de entrenamiento. De esta forma, la red se encargará de hacer clases (clusters) que agrupan datos suficientemente parecidos entre sí. De esta forma, al algoritmo se le indica el número de clusters a crear.

3. **Aprendizaje por refuerzo**: se suele utilizada en aquellos problemas que tienen la variable tiempo asociada

# 6. Nota

No dimos mucho temario, Senén dio discurso.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Inteligencia artificial

13-10-2021

Clase 10

Temas trabajados:

5. Sistemas conexionistas
6. Aprendizaje automático

# Índice

1.....Tema 5: Sistemas conexionistas .....3

a).....Funcionamiento de una neurona .....3

b) .....Algoritmo Perceptrón .....3

2.....Tema 6: Aprendizaje automático .....4

c).....Definición del aprendizaje automático de Tom Mitchell .....5

d) .....Estrategias de aprendizaje automático .....5

i.....Problema de regresión .....5

ii.....Problema de clasificación .....6

e) .....Regresión lineal .....6

iii..... Error cuadrático medio .....7

# 1. Tema 5: Sistemas conexionistas

Para finalizar con la introducción a la computación neuronal, quedan 2 cosas muy importantes:

- → Entender cuál es el fundamento, desde el punto de vista de resolución de problemas, de una neurona formal (artificial).
- → Algoritmo Perceptrón: propuesto a finales de los 50, es muy interesante ya que en aquel momento era una de las primeras aportaciones al campo del aprendizaje automático, a través de un algoritmo un proceso sistemático que opera sobre redes de neuronas en una estructura monocapa (gran limitación del perceptrón). Fue una propuesta muy interesante al permitir la resolución de problemas partiendo de datos sin necesidad de codificar exactamente la solución (implementar un algoritmo conocido).

## a) Funcionamiento de una neurona

Para ver el funcionamiento de una neurona en un proceso de clasificación o discriminación entre categorías se parte de la neurona que ya conocemos, con dos entradas (suponemos que opera en un espacio bidimensional, se puede generalizar sin problema a un espacio multidimensional). Se parte de dos clases que interesa diferenciar, y de una información (datos que pertenezcan a una y otra clase), de forma que cuando se introduzcan nuevos datos, un clasificador indique a cuál de las dos clases pertenece. Las entradas serían los valores de la variable, es decir, en el espacio bidimensional en el que nos encontramos una entrada es la coordenada x del punto y la otra es la coordenada y. También se puede incidir en los pesos, que se pueden fijar a través de algún algoritmo de aprendizaje. La salida de la neurona pasa por una función de tipo signo que construye propiamente la salida final de la neurona.

¿Qué implementa esa neurona? Divide en dos clases el espacio x1 y x2.

Una vez fijados los pesos, la neurona va a discriminar cualquier valor de entradas en una clase u otra. Falta ahora un algoritmo que nos ayude a fijar el valor de w1, w2 y b (algoritmos de aprendizaje).

## b) Algoritmo Perceptrón

El perceptrón es un método que nos permite fijar los valores de cualquier número de pesos que queramos fijar, pero solo para estructuras monocapa, es decir, para una neurona o un conjunto de neuronas no enlazadas (las entradas de una neurona no dependen de las salidas de otras).

Permite construir un clasificador óptimo sobre el conjunto de datos de entrenamiento (conjunto de datos dados). Que el clasificador funcione bien dependerá de la calidad del conjunto de datos dado, por lo que es importante que se proporcione un conjunto de datos representativo del problema. Para llegar a una solución óptima con este algoritmo también es importante que los datos sean linealmente separables, si esto no es así puede haber problemas a la hora de llegar a la mejor solución.

El cálculo del peso se produce de la siguiente forma: $w(n+1) = w(n) + \eta\,[d(n) - y(n)]X(n)$

Donde *d(n)* vale +1 si *X(n)* pertenece a la clase 1 y –1 si pertenece a la clase 2.

En el instante de tiempo n+1 el valor del peso (w) será el que ya tenía más una cantidad dada por:

- η es el coeficiente de aprendizaje, suele ser un valor constante y pequeño (normalmente muy pequeño) entre 0 y 1. Su función es ponderar lo que sigue de tal modo que al final los cambios que se producen sobre los pesos en cada ciclo sean pequeños.
- A continuación, se mide la diferencia entre la salida producida (*y(n)*) y la salida que se quiere que de la neurona (*d(n)*). Si esta diferencia es igual a 0 (no hay diferencia) el valor de w en el instante *n+1* es igual a su valor en el instante anterior (*n*). Si hay diferencia, se modificará el peso para conseguir que la salida sea la esperada, aumentándolo o disminuyéndolo dependiendo de la clase a la que pertenezca la entrada introducida.

Siempre se va a partir de un conjunto de entrenamiento formado por pares de valores: una entrada y la salida deseada para ella. De esta forma se resolverá el problema cuando, para el conjunto de entradas del conjunto de entrenamiento, la salida sea la indicada, de forma que, si ese conjunto es representativo, cuando procese nuevas entradas el sistema funcionará bien (se determinará la validez del funcionamiento en función de unos criterios de rendimiento).

El algoritmo del perceptrón parte de unos pesos aleatorios y va pasando los datos de entrenamiento una y otra vez por la entrada de la neurona para ir ajustando los pesos hasta disminuir de forma adecuada el error. Si las clases son linealmente separables finalizará cuando el error llega a 0, mientras que si no lo son se intentará llegar al error mínimo alcanzable. Algunos algoritmos de aprendizaje ajustan los pesos calculando el error acumulado para un conjunto de datos o incluso para todo el conjunto en lugar de para cada dato, ya que esto puede producir oscilaciones indeseadas.

*Un *epoch* es una pasada del conjunto de entrenamiento completo por la entrada de la neurona.

Minsky y Papert publicaron el libro “Perceptrón” en el año 1969, dedicado al análisis de este algoritmo de aprendizaje automático sobre neuronas formales, analizando sobre todo sus limitaciones. La limitación fundamental es el hecho de que solo funciona para neuronas en una capa, y hay muchos problemas que necesitan dos capas para resolverse. Minsky y Papert evidenciaron que no había ningún algoritmo que pudiera calcular los pesos para una estructura de varias capas y sostenían que no llegaría a haberlo (se equivocaron).

Por ejemplo: la función lógica xor (or exclusiva) se puede representar como un problema de clasificación donde existen dos dimensiones (valores de entrada de la función) y cuatro valores posibles que la función puede tomar. Aunque es un caso tan simple que no tiene sentido construir un clasificador por aprendizaje automático (es una tabla), se podría plantear como ejercicio teórico ya que es un caso muy simple que se debería poder resolver con una neurona con 2 entradas y una salida. Esto no es posible resolverlo con una única neurona ya que el problema no es linealmente separable, mientras que con 3 neuronas (2 en la primera capa y 1 en la segunda) se resuelve. Por lo tanto, el problema se puede resolver, pero son necesarias 2 capas para ello.

# 2. Tema 6: Aprendizaje automático

En este capítulo se deja atrás la computación neuronal, pero seguimos dentro del marco del aprendizaje automático, con una serie de estrategias conocidas como represores. Podría decirse que son “matemáticas que aprenden”, son algoritmos basados en fundamentos matemáticos, pero con la capacidad de aprender a resolver cierto tipo de problemas que se ajustan a la capacidad de aprendizaje de los mismos. Veremos dos tipos de regresores: el regresor lineal y el regresor logístico.

## c) Definición del aprendizaje automático de Tom Mitchell

Un programa informático aprende de la experiencia E en relación a una tarea T, utilizando una medida de rendimiento P, si mejora sus prestaciones, medidas mediante P, en la realización a la tarea T a través de la experiencia E."

Ejemplo: juego de damas

- → E = experiencia adquirida mediante el juego de muchas partidas (pueden ser contra una persona, contra una máquina o contra si misma).
- → T = jugar a las damas, el propio juego.
- → P = probabilidad de que el programa gane la próxima vez, medida de rendimiento.

## d) Estrategias de aprendizaje automático

El aprendizaje automático se clasifica fundamentalmente en tres clases, aunque algunas estrategias no responden exactamente, o no desde un modo canónico, a una de ellas.

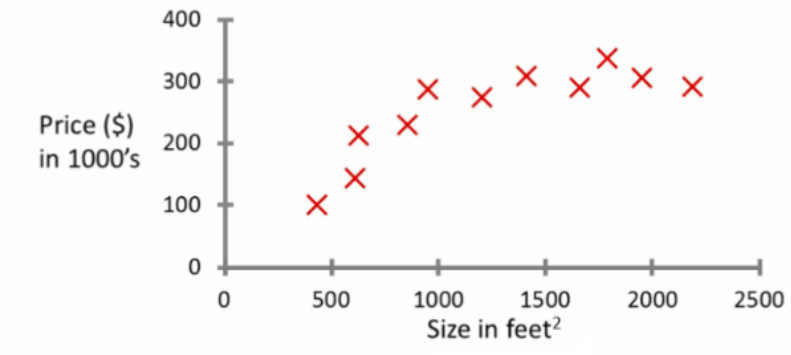
- **Aprendizaje supervisado**: "Durante la experiencia E se indica cómo se ha de realizar la tarea T". La mayoría de los problemas resueltos por aprendizaje automático son de este tipo. Es muy interesante, funciona muy bien, pero requiere que dispongamos de un conjunto de datos anotados, esto es un conjunto suficientemente grande y representativo de datos perfectamente identificados con la salida que queramos que esa máquina aprenda a asociar.
- **Aprendizaje no supervisado**: "Durante la experiencia E no se indica cómo se ha de realizar la tarea T". En este caso también se parte de un conjunto de datos pero en este caso no están anotados, por lo que se agrupan en categorías. El algoritmo de aprendizaje opera agrupando los datos introducidos intentando maximizar las similitudes entre datos de una misma categoría y la diferencia entre datos de categorías distintas.
- **Aprendizaje por refuerzo**: "Durante la experiencia E se dan indicios de si se está realizando bien/mal la tarea T". Aquí no se anotan los datos, sino que se identifica el comportamiento del sistema que se está entrenando para indicarle si lo está haciendo bien o no. Si lo hace bien, se refuerza aquello que interviene en las decisiones que llevan a que lo haga bien y si no, se debilita el papel de esos parámetros. Se usa mucho en problemas donde la variable temporal es significativa en el proceso de actuación o sensorización. Por ejemplo, en el entrenamiento de un robot, se penaliza que se choque contra una pared o se refuerza aquello que haga que el robot mantenga una distancia de seguridad más o menos constante.

### i. Problema de regresión

En este tema veremos cómo resolver problemas mediante la regresión lineal. Se introduce a continuación un problema de ejemplo que se irá utilizando durante las explicaciones, se trata de un problema trivial pero realista, en su versión más simple posible:

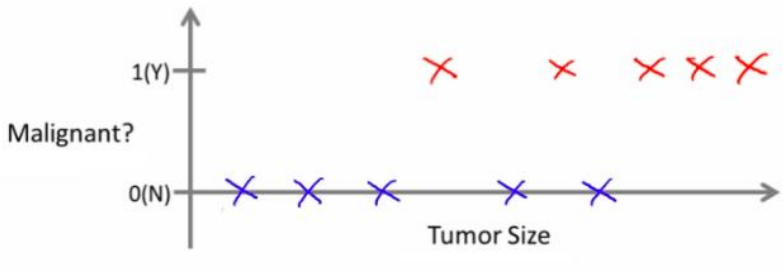
A una inmobiliaria que durante su historia vendió una serie de casas y pisos, le gustaría tener un sistema informático que, cuando llegue un cliente que quiera poner a la venta un inmueble, permita poder anticipar lo que sería el precio razonable o de referencia. En este caso, por simplicidad, se considera que lo realmente relevante para poner precio al inmueble es su superficie (aunque en un caso real influirían muchos otros parámetros).

La forma más simple de llegar a una solución sería construir una recta a partir de los datos de los inmuebles vendidos y, a partir de la misma, calcular el precio para nuevos inmuebles.




### ii. Problema de clasificación

Pongamos el ejemplo de un clasificador de tumores, que debe discriminar el carácter del tumor (benigno o maligno) en función de un único parámetro que es su tamaño.




Podemos ver también que es importante establecer unas medidas de rendimiento en función del problema concreto a tratar. En este ejemplo sería importante localizar siempre los tumores malignos, asumiendo el riesgo de clasificar como maligno algún tumor benigno y no al revés.

## e) Regresión lineal

Es un método de aprendizaje supervisado, por lo que se partirá de un conjunto de aprendizaje anotado.

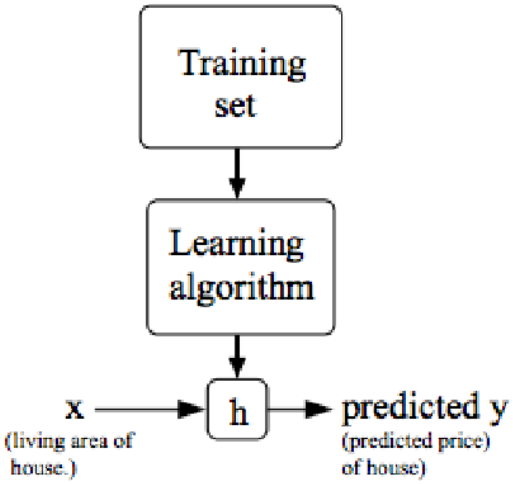
Se utilizará la siguiente notación:

- m = número de ejemplos de entrenamiento (tamaño del conjunto de entrenamiento)
- x = valores de entrada / carácterísticas
- y = valor de salida / respuesta
- El conjunto de entrenamiento tendrá la siguiente forma:

(x, y) – par entrada-salida genérico --> (xi, yi) - par entrada-salida iésimo

Construir un regresor lineal es como construir una función. Para las neuronas había que definir los valores de los pesos, que lo que hacían era definir donde se situaba una recta o hiperplano de división entre dos categorías. En este caso es lo mismo, pero no tenemos la idea de elemento neuronal formal, sino directamente una función. Se quiere llegar a construir una función *h()* que aproxime los puntos del conjunto de entrenamiento. Esta función tendrá unos parámetros libres sobre los que yo puedo actuar y, en la medida en que se le asignen valores adecuados, esta función aproximará mejor o peor el conjunto de datos. A esta función *h()* se le llama en algunos casos hipótesis.

Desde un punto de vista funcional es muy simple: tengo unos datos de entrenamiento, la incógnita es la función *h()* que, a partir de estos datos, es definida mediante un algoritmo de entrenamiento. Una vez construida la función y con un rendimiento considerado como suficiente, para cualquier nuevo valor de *x* se puede calcular su valor de *y* asociado a través de *h()*.




Esta función, en su forma más simple, es una recta que se caracteriza por su valor en el origen y la pendiente, de la siguiente forma:

$$y = h_\theta(x) = (\theta_0 + \theta_1 x)$$

En el caso de la inmobiliaria, en la coordenada x se representaría el tamaño de la casa y en la coordenada y el precio de la misma.

El algoritmo de entrenamiento funcionará tratando de minimizar el error cometido, que es la distancia entre los puntos de entrenamiento y la recta que los aproxima. Hay múltiples funciones para calcular el error, pero una muy útil y bastante utilizada es el error cuadrático medio.

### iii. Error cuadrático medio

El error cuadrático medio consiste en medir la distancias entre la función hipótesis y cada uno de los datos del conjunto de entrenamiento elevada al cuadrado, sumarlas todas y dividirlas entre el número de elementos del conjunto de entrenamiento.

$$\frac{1}{2m}\Sigma_{i=1}^{m}\left(h_\Theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grao en Enxeñaría Informática

3º curso – 1º cuadrimestre

# Intelixencia Artificial

20-10-2021

Clase 12

Temas traballados:

Aprendizaxe Automática.

Fundamentos e regresión lineal

# Índice

1. Introdución ..... 3
2. Regresión lineal ..... 3
   a) Exemplo de $J(\theta_0, \theta_1)$ para un problema dado ..... 4
3. Exemplo práctico e xeneralización da regresión lineal ..... 6
4. Regresión polinómica ..... 8
5. Regresión loxística ..... 9

# 1. Introdución

"Un programa informático aprende da experiencia E en relación a unha tarefa T utilizando unha medida de rendemento P, se mellora as súas prestacións, medidas mediante P, en relación á tarefa T a través da experiencia E"

Por exemplo, se falamos de xogar ás damas:

- E = experiencia adquirida mediante o xogo de moitas partidas
- T = xogar ás damas
- P = probabilidade de que o programa gane a próxima vez

Existen basicamente 3 estratexias de aprendizaxe automático:

- **Aprendizaxe supervisada**: durante a experiencia E indícase como se ten que realizar a tarefa T. Utilízanse conxuntos de adestramento os cales teñen que ser representativos dos casos nos que se vai aplicar a IA. Pódese utilizar en dous tipos de problemas:
    - **Problemas de regresión**: aprender a función que mellor represente os datos de adestramento.
    - **Problema de clasificación**: aprender o decisor que mellor discrimine os datos de adestramento. A parte, ademais de discriminar os datos en dúas clases, hai que ter en conta factores como que ante a dúbida, é preferible diagnosticar un falso positivo dunha enfermidade que un falso negativo.
- **Aprendizaxe non supervisada**: durante a experiencia E non se indica como se ten que realizar a tarefa T. É o máis utilizado despois do supervisado. Neste caso aprende sobre datos que non están previamente etiquetados. Apréndese polo tanto a agrupalos en función de distintos criterios de semellanza ou outros criterios de categorización dos datos.
- **Aprendizaxe por reforzo**: durante a experiencia E danse indicios de se se está a realizar ben ou mal a tarefa T. Este tipo de aprendizaxe utilízase moito en sistemas nos que a variable temporal ten moito peso.

Ás veces, as decisións humanas están moi influídas polo ruído, polo que a IA pode aportarlle luz ó asunto. Mais tamén hai que ter en conta que o desenvolvemento da IA pode resultar ser totalmente distinto ó esperado. Exemplo: para terminar coa pandemia, asasinar a todos os humanos.

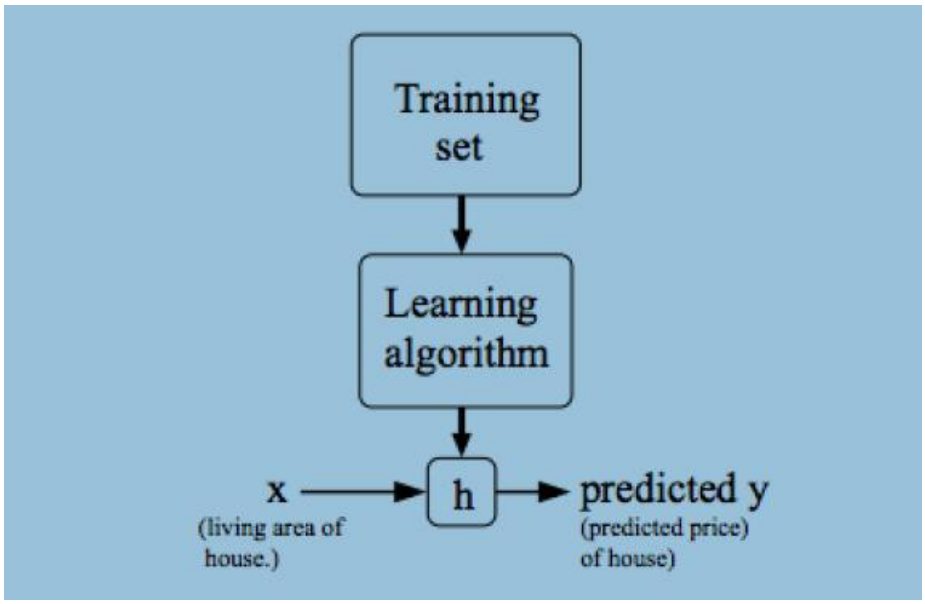
# 2. Regresión lineal

**É un tipo de aprendizaxe supervisada** e polo tanto trabállase cun conxunto de datos previamente etiquetado. A este conxunto de adestramento aplícaselle un algoritmo de aprendizaxe. **O resultado do algoritmo é unha función h() (Hipótese).** A aprendizaxe consiste en construír h() de tal maneira que aporte o valor de saída *y* asociado á entrada *x* de acordo co especificado no conxunto de adestramento (par (x, y) entrada saída).

O máis fácil é que a función sexa unha recta, polo que teríamos dúas variables (ordenada na orixe e pendente). Os datos cos que traballamos xa nos dan o valor da variable *x*, que é a entrada, polo que o que temos que buscar son os valores $\theta_0, \theta_1$tal que a saída sexa a que queremos.

Se en lugar dunha variable temos máis, en vez dunha recta teremos un hiperplano, pero dá igual, podemos traballar de igual maneira.

Cantos máis datos teñamos, de maior orde vai ter que ser a nosa función para achegarse a eles.




A regresión lineal son "as matemáticas que aprenden", un elemento preditor no mundo da aprendizaxe automática.

A hipótese $h$ represéntase como unha ecuación $y = h_\theta(x) = (\theta_0 + \theta_1 x)$ que será unha función lineal de $x$. $\theta_i$ son os parámetros que definen a recta. Neste caso:

- $\theta_0$ é o seu valor na orixe
- $\theta_1$ é a súa pendente (gradiente)

Agora hai que definir a **función de custo que guíe ao algoritmo de aprendizaxe** ao axuste dos parámetros $\theta_j$ que definen a recta que, á súa vez, teñen que aproximar o mellor posible os exemplos do conxunto de adestramento. Trátase de que $h_\theta(x)$ se aproxime o posible a $y$, se o par $(x, y)$ é un exemplo de comportamento na resolución do problema. Basicamente $h_\theta(x)$ é un "imitador de $y$". Ademais debemos avaliar en que medida o é para poder guiar o proceso de aprendizaxe.

Esta guía consiste nun **problema de minimización**, pois búscase minimizar $(h_\theta(x) - y)^2$, por exemplo, para cada par de valores de entrenamento: $\frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$

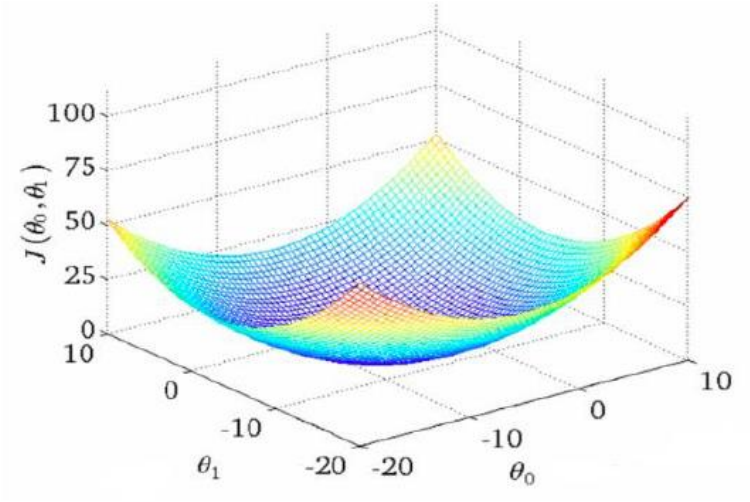
Polo tanto, a función de custo, neste caso o **erro cuadrático medio**, será:

$$J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

Cabe apuntar que no erro cuadrático medio como tal, o ½ sobra, está por outros motivos (derivación).

## a) Exemplo de $J(\theta_0, \theta_1)$ para un problema dado

Neste caso trátase dunha función convexa por como está definido $J$. Esta sería unha situación ideal que poucas veces se dá pois pode converxer no mínimo facilmente. **Normalmente non ocorre isto, se non que a función ten multitude de mínimos locais nos que o algoritmo de minimización se atopa con problemas**. Este suceso pódese evitar de diversas maneiras.




Algoritmo para minimizar o valor de $J(\theta_1, \theta_2)$:

1) Comézase por un valor dado de $\theta_1$ e $\theta_2$.
2) Variamos estes valores para que se reduza $J(\theta_1, \theta_2)$.
3) Repetimos ata que se chegue ao mínimo (pode ser local).

Formalmente trátase o que segue ata que non se poida continuar:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1), \text{ para } j = 0 \text{ e } j = 1$$

$$temp0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

$$temp1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

$$\theta_0 := temp0;\ \theta_1 := temp1$$

Método de descenso do gradiente:

$$\frac{\partial}{\partial \theta_j} J(\Theta_0, \Theta_1) = \frac{\partial}{\partial \theta_j} \cdot \frac{1}{2m} \Sigma_{i=1}^{m} \left( h_\Theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$
$$= \frac{\partial}{\partial \theta_j} \cdot \frac{1}{2m} \Sigma_{i=1}^{m} \left( \theta_0 + \theta_1 x^{(i)} \quad - y^{(i)} \right)^2$$

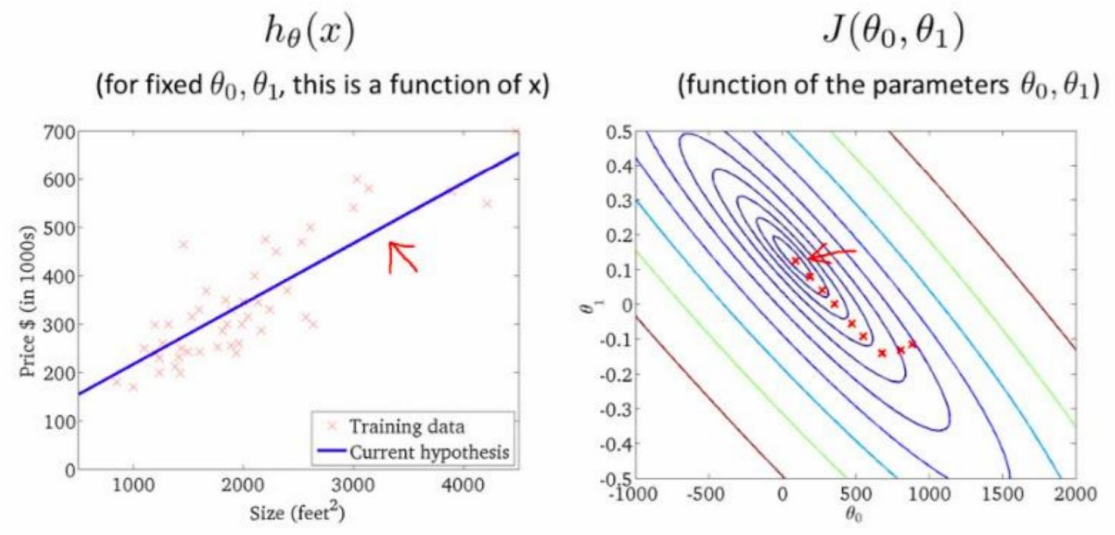
Derivación:

$$j = 0: \frac{\partial}{\partial \theta_0} J(\Theta_0, \Theta_1) = \frac{1}{m} \Sigma_{i=1}^{m} \left( h_\Theta\left(x^{(i)}\right) - y^{(i)} \right)$$

$$j = 1: \frac{\partial}{\partial \theta_1} J(\Theta_0, \Theta_1) = \frac{1}{m} \Sigma_{i=1}^{m} \left( h_\Theta\left(x^{(i)}\right) - y^{(i)} \right) . x^{(i)}$$

Con estas funcións xa podemos aplicar o algoritmo anterior e por en marcha a aprendizaxe mediante a regresión lineal. Deste xeito conseguiríamos uns valores $\theta_1$ e $\theta_2$ finais. Habería que estudar a posteriori se o grao de erro conseguido é o suficiente como para atacar o noso problema.

Con só unha pasada polo conxunto de adestramento non adoita ser suficiente, se non que se requiren varias. Normalmente este repítese ata que se cumpre unha condición de parada (certo número de ciclos, ter alcanzado un límite de erro, etc.)




A gráfica da esquerda móstranos un exemplo de función de aproximación mediante regresión lineal mentres que a da dereita mostra a evolución do erro cuadrático medio nas distintas iteracións. Neste último caso, as liñas mostran a unión de puntos de igual erro.

# 3. Exemplo práctico e xeneralización da regresión lineal

Intentamos aplicar a regresión lineal para crear un preditor do prezo dunha casa en función de distintas características súas:

$$y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 172 \end{bmatrix} \quad X = \begin{bmatrix} 2104 & 5 & 1 & 45 \\ 1416 & 3 & 2 & 40 \\ 1534 & 3 & 2 & 30 \\ 852 & 2 & 1 & 36 \end{bmatrix}$$

Os valores de y son os prezos de venta e os da dereita son [Tamaño en pés, número de habitacións, pisos, anos dende a construción].

Neste caso só estamos traballando con 4 casas, pero nun exemplo real teríamos moitas máis. Tamén teríamos moitos máis parámetros.

Se xeralizamos o problema:

- n é o número de características
- m é o número de exemplos de adestramento
- $x^i$ é o vector de características para o exemplo i-ésimo
- $x_j^i$ é a característica j-ésima do exemplo i-ésimo

Polo que a **hipótese h** agora ten a forma:

$$h_0(x) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \cdots + \theta_n x_n$$

$$h_0(x) = \theta^T X$$

Sendo $\theta^T$ o vector trasposto de $\theta$ e $x_0$= 1 por conveniencia. (Na ordenada no orixe pódese considerar que $x_0$= 1 pero non se pon xa, mais se queremos ter o mesmo subíndice en $\theta$ e en $x$ quedaría así).

Se xeralizamos o descenso do gradiente para múltiples variables, quedaríanos a seguinte función de custo:

$$J(\Theta_0, \Theta_1, \dots, \theta_n) = \frac{1}{2m} \Sigma_{i=1}^{m} \left( h_\Theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

Axustamos os parámetros da hipótese:

$$Repetir \left\{ \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\Theta_0, \dots, \Theta_n) \right\},$$
$$actualizando\ simultáneamente\ los\ valores\ para\ j = 0, \dots, n$$
$$Repetir \left\{ \theta_j := \theta_j - \alpha \frac{1}{m} \Sigma_{i=1}^{m} \left( h_\Theta\left(x^{(i)}\right) - y^{(i)} \right). x_j^{(i)} \right\},$$
$$actualizando\ simultáneamente\ los\ valores\ para\ j = 0, \dots, n$$

Algunhas consideracións que hai que facer:

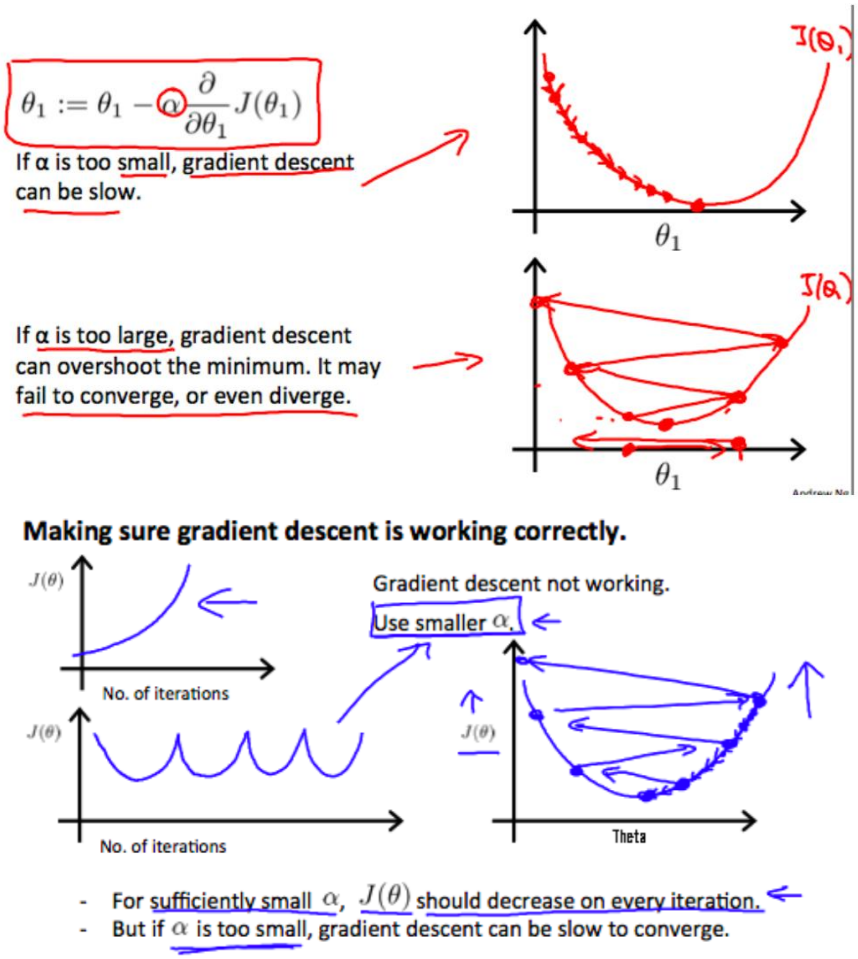
- Se as variables de entrada toman valores de rangos moi distintos pode complicarse o proceso de converxencia, polo que se poden normalizar os valores do seguinte modo:

$$x_j^i := \frac{x_j^i - \mu_j}{s_j}, i = 1, \dots, m;\ j = 1, \dots, n$$

$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} x_j^i, \qquad s_j = \sqrt{\frac{1}{m} \sum_{i=1}^{m} \left( x_j^i - \mu_j \right)^2}$$

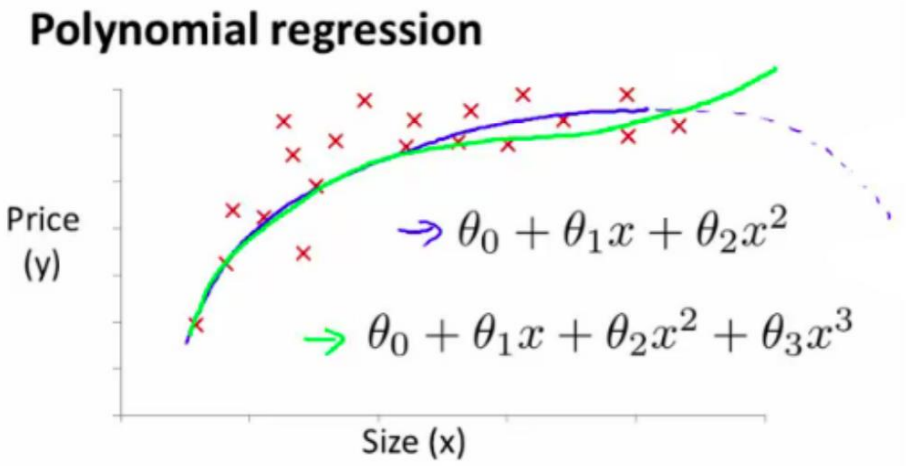
  Isto é, restarlle á característica j-ésima de cada un dos *m* valores do conxunto de adestramento o valor medio dos mesmos e dividilo entre a súa desviación estándar.

- O valor que pondera a modificación de $\theta$ en cada iteración (**coeficiente de aprendizaxe α**) ten moita importancia na converxencia do algoritmo. Se ten un valor moi pequeno, a converxencia pode ser moi lenta. Se pola contra o valor é moi grande, o algoritmo pode non converxer. Coa representación gráfica este feito vese moi ben:




# 4. Regresión polinómica

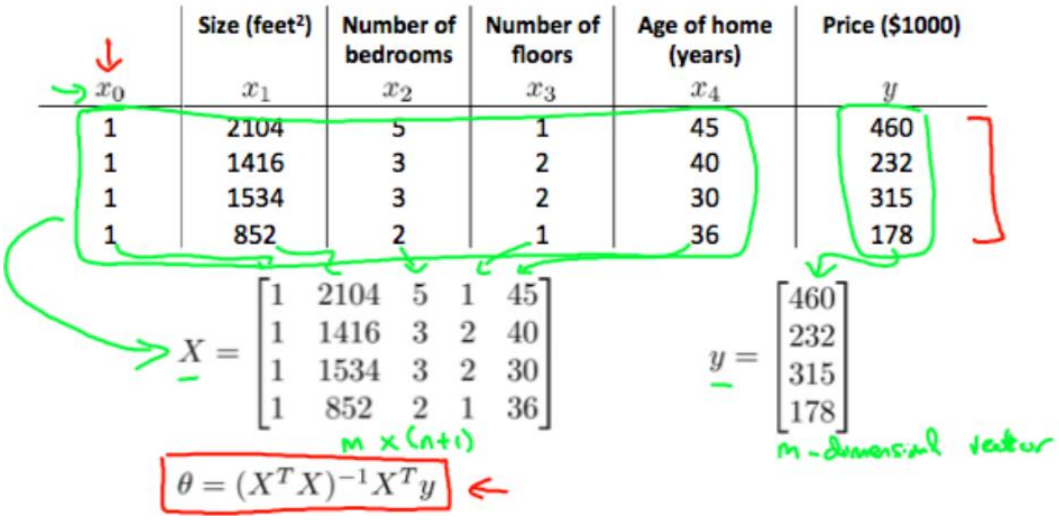
Unha recta non sempre nos vai servir para aproximar funcións, ás veces precisamos funcións de maior orde, por exemplo:




Mais para realizar este tipo de aproximación podemos utilizar a mesma estratexia. Neste gráfica, como só temos unha variable (o tamaño da casa), só aparece x, máis poderíamos ter máis variables de distintos graos, e o grao de cada unha xa será determinado a través do algoritmo de

aprendizaxe. As variables que teñan valores preponderantes terán graos máis altos, mentres que aquelas máis insignificantes teranos máis pequenos.

Hai certos casos nos que podemos calcular de xeito analítico o valor de $\theta$. **Pódese facer sempre e cando a matriz de características sexa invertible**:




Se comparamos o cálculo de $\theta$ desta maneira co seu cálculo mediante o descenso de gradiente, obtemos a seguinte táboa:

| Descenso de gradiente | Ecuación normal |
|---|---|
| Necesidad de definir a | No es necesario definir a |
| Se recomienda escalar las características de entrada | No es necesario escalarlas |
| Muchas iteraciones | Sin iteraciones |
| $O(kn^2)$ | $O(n^3)$, necesidad de calcular la inversa de $X^TX$ |
| Funciona bien cuando **n** es grande | Es lento cuando **n** es muy grande |
| No tiene este problema | En ocasiones $X^TX$ no es invertible |
| -- | No es aplicable en general a otros algoritmos de aprendizaje |

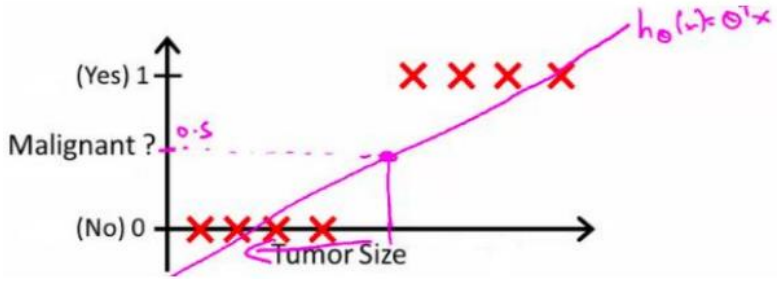
Normalmente, se temos moitas variables pero non moitos datos, seguramente non sexa invertible a matriz $X^TX$.

Co visto ata agora, podemos ser capaces de crear preditores que nos dean unha aproximación ante un caso de entrada en función dos casos estudados no conxunto de adestramento. Mais tamén cabe a posibilidade de que en lugar de aproximar, queiramos clasificar datos, como o faríamos?

# 5. Regresión loxística

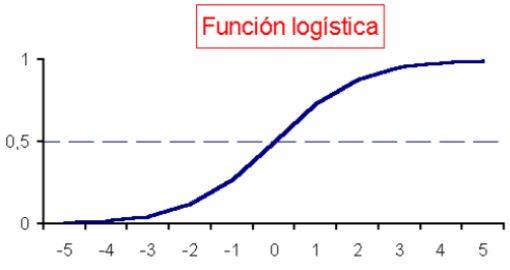
Serve o método anterior para discriminar elementos de dúas ou máis clases? Poderíamos probar a fixarlle un umbral a $h_\theta$, mais hai varios inconvenientes:

- $h_\theta(x)$ non está acotado de partida entre 1 e 0
- En xeral a aproximación lineal con umbrais non é un bo clasificador



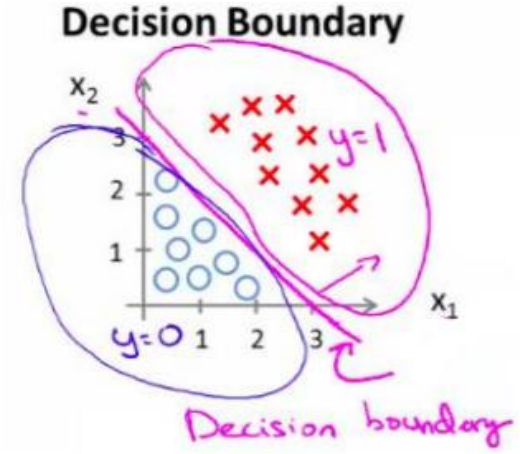

**Para solventar o problema podemos facer uso da función sigmoide ou loxística** (que ademais é tamén derivable e evoluciona de forma suave) para representar *h*:

$$h_\Theta(X) = \frac{1}{1+e^{-\theta^T X}}$$

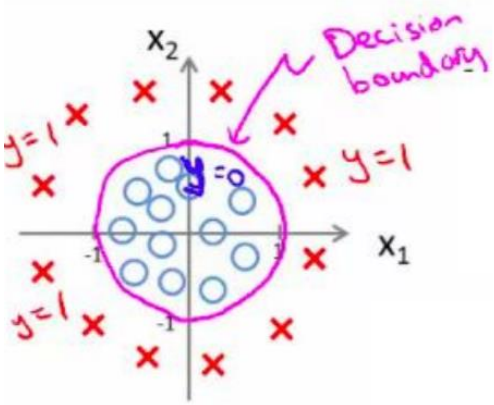



$h_\theta(x)$ é unha "estimación" da probabilidade de que y=1 ante unha entrada dada x. Esta función xa a poderíamos utilizar para a introdución dun umbral con máis precisión. Mais pódese utilizar este método se tivésemos máis variables? Si, se tivésemos 2 por exemplo, en lugar de traballar cunha recta fariámolo con hiperplanos.

Outro problema que podemos ter é que ás veces podemos separar as clases linealmente, por exemplo:




Pero noutros casos as clases non van ser separables:




Mais de igual forma que podemos aprender funcións como a da primeira gráfica, tamén podemos aprender unha expresión que defina o espazo da segunda.

$X_1^2 + X_2^2 >= 1$ > Ecuación dun círculo de radio unha unidade.

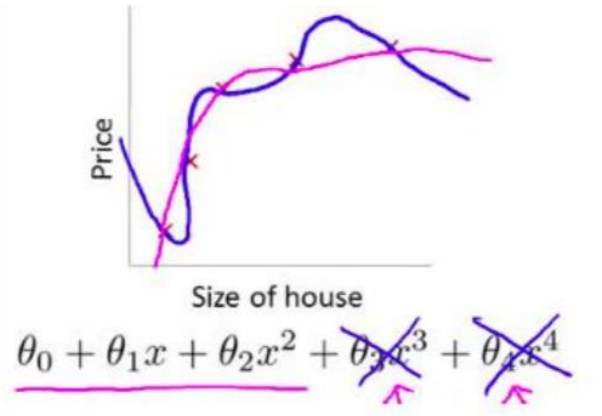
Cal debe ser o grao de cada variable?

Ir probando manualmente é moi custoso, pero podemos incluír na función de custo valores de $\theta_j$ ó cadrado, cubo..., ponderados cun valor λ, de valor alto. Deste xeito só os valores de $\theta_j$ máis relevantes, os que conseguen reducir o erro, prevalecen.

$$J(\theta) = \frac{1}{2m}\left[\Sigma_{i=1}^{m}\left(h_\Theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\Sigma_{j=1}^{m}\theta_j^2\right], con\ \lambda \gg$$

Se o grao 2 xa serve para reducir o erro, un grao maior sería descartado.

Hai que ter en conta que canto maior sexa o grao, maior flutuación se produce e polo tanto, ter un erro nulo para os casos do conxunto de proba non sempre mellora a capacidade da IA, hai que ter en conta outros factores.




No caso da imaxe, a función morada ter erro 0, mais na maioría de casos a función rosa ofrece unha maior predición.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Intelixencia Artificial

25-10-2021

Clase 13

Temas trabajados:

Aprendizaxe automático

# Índice

1. Cousiñas extra ..... 3
2. Regresión logística ..... 3
   a) Función de coste: ..... 3
      i. Función sigmoide ..... 3
      ii. Funcións alternativas ..... 4
   b) División en clases ..... 5
   c) Sobreaprendizaxe ..... 6
   d) ¿Como aborda-lo sobreaprendizaxe? ..... 7
   e) Regularización ..... 7
   f) Figura que se saltara de Regresión Loxística ..... 8
3. Aprendizaxe non supervisada ..... 8
   g) Exemplo ..... 9

# 1. Cousiñas extra

- Temas abordados para presentacións: no cv hai bastantes temas de interés.
- Senén falou das súas clases maxistrais en Madrid como exemplos para as nosas presentacións. Dixo de incluir: perspectiva histórica, cultural, impactos (emprego, sociedade).
- Falouse de Vega Sicilia, viños caros.
- Optativo: realizar un artigo: a súa presencia en medio de comunicacións.
- Para a revisión deste artigo: entrégase a versión, el vaina ler, dirá algúns comentarios, revisase, vólvese entregar e repetir ata que diga OK.
- Hai que aprobar presentacións para poder aprobar a materia.

# 2. Regresión logística

Usar regresores, usar función que aproximan a puntos, a datos dun conxunto de entrenamento pero para construir apoximadores, no soamente para aproximar esos puntos e ter por tanto una función de predicción dunha saída fronte a novos puntos de datos de entrada senon deseñar clasificadores, que me permitan a través dun proceso supervisado construir un sistema que me dirá a que clase pertenecen una vez que eu definin esas clases con claridade durante o proceso de diseño do sistema

## a) Función de coste:

Tanto a función de erro como a de aproximación pode cambiar. Normalmente optase pola que xa vimos: erro cadrático medio.

$$J(\Theta_0, \Theta_1) = \frac{1}{2m}\Sigma_{i=1}^{m}\left(h_\Theta\left(x^{(i)}\right) - y^{(i)}\right)^2$$

Esta función dinos a discrepancia entre a saída real e a ideal, con isto poderemos ter una noción de se se está facendo ben ou mal (se se vai modificando ata a solución desexada).

Medir o erro é imprescidible para determinar se o sistema está finalizando coa solución desexada. Ademáis disto, é de interese minimizar a función de erro; o algoritmo que decide como axustar os parámetros derívase matematicamente de forma que se reduza o erro. Isto ocurre nas redes neuronais e nas regresións.

Na regresión en lugar de axustar os pesos dunhas conexións entre neuronas axústase uns parámetros dunhas funcións, pero é fundamentalmente o mesmo. Dixemos que unha función que normalmente me sirve bastante ben é a función sigmoide.

### i. Función sigmoide

Esta función é útil en procesos de clasificación xa que é moi útil para disinguir entre dúas categorías debido a que está acotada entre 0 e 1. Ademáis é derivable en todo o punto.

Aqueles puntos nos que a función tome valores próximos a 0 será unha categoría e naqueles que tome valores próximos a 1 será outra diferente. Por tanto é un discriminador bastente obvio entre dúas categorías. En caso de que haxa máis de dúas categorías deberán convinarse varías funcións para acotar as distintas opcións.

ii. Funcións alternativas

Tamén se pode utilizar funcións de aproximación alternativas, como pode ser:

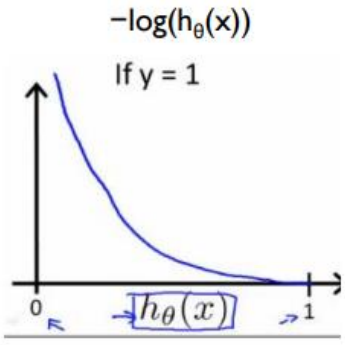
$$\text{Cost}(h_\theta(x),y) = -\log(h_\theta(x)), \text{ si } y=1$$

$$\text{Cost}(h_\theta(x),y) = -\log(1-h_\theta(x)), \text{ si } y=0$$

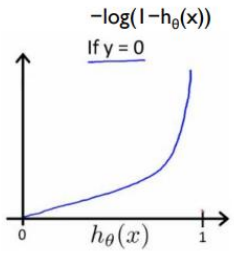
$$\text{Es decir: Cost}(h_\theta(x),y) = -y.\log(h_\theta(x)) - (1-y).\log(1-h_\theta(x))$$

Esta función está composta por dúas, a primeira que aparece operaría cando desexo que a saída teña valor 1 e a segunda valor 0. A última sería a función de coste combinando as dúas.

Se a saída toma valor 1, o segundo termo da función de coste anúlase polo que quedaría só: $-\log(h_\Theta(x))$. Isto quere dicir que, nese caso, para valores da funcio´n próximos a 1 o valor da función de coste ou de erro é 0, pero para valores da función próximos a 0 o valor da funición é exponencial, penalizando enormemente o proceso de aprendizaxe.




Se o proceso se repetise para o caso contrario, se a saída esperada é 0, entonces anularíase o primeiro termo quedando o seguinte:




De acordo coa función de coste, cando a saída debería ser 0, se a función de erro vale 0, enton no se penalizaría. Se o valor da función fose próximo a 1 enton a penalización sería maior.

Un proceso de entrenamento que funcione adecuadamenten con esa función de coste o que vai a facer é aproximar  a función de erro para que os seus valores sexas os desexados conforme ao conxunto de datos de entrenamento.

Con esta nova función de coste, o axuste dos parámetros da función de erro é a mesma(mediante descenso de gradiente).

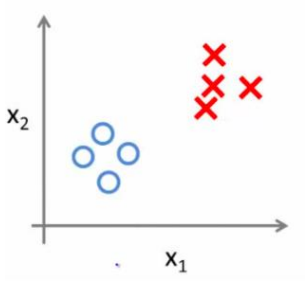
$$Repetir\ \left\{\theta_j := \theta_j - \frac{\alpha}{m}\Sigma_{i=1}^{m}\left(h_\Theta\left(x^{(i)}\right) - y^{(i)}\right).x_j^{(i)}\right\},$$

Se nos fixamos na función anterior de axuste dos parámetros, é igual que a de regresión lineal pero con:

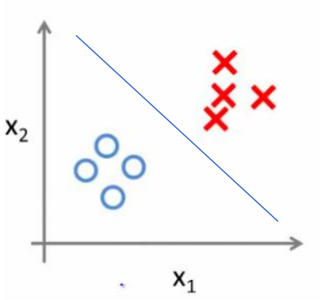
$$h_\Theta(X) = \frac{1}{1+e^{-\theta^T X}}$$

## b) División en clases

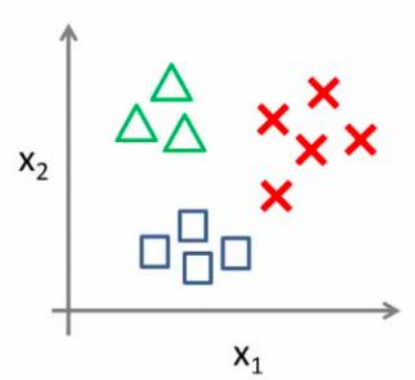
Discriminar  varias clases, como xa se dixo no apartado anterior, o único que habería que facer é producir e combinar máis regresores. Se eu teño que discriminar duas clases representadas no  seguinte plano:



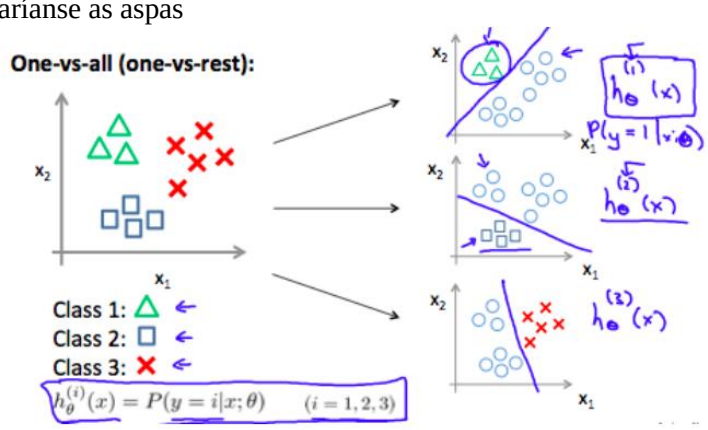

O que se fai é, a traves dunha función sigmoide darlle valores próximos a 0 ou próximos a 1 a cada unha desas clases de tal xeito que quedaría da seguinte maneira:




Se se teñen tres clases faríase o seguinte:




1- Separaríanse os triángulos do resto das clases
2- Separaríanse os cadrados
3- Separaríanse as aspas




Se diseña un clasificador mediante regresión logística para cada clase, asignando cada nuevo *x* de entrada a la clase para la que $h_\Theta(x)$ es mayor

Para distinguir entre as clases necesitaríanse 3 funcións de aproximación. Cando se remate co conxunto de entrenamento, entón unha vez se metan datos a resposta óptima será o máximo valor entre as tres funcións (o valor máis próximo a 1).

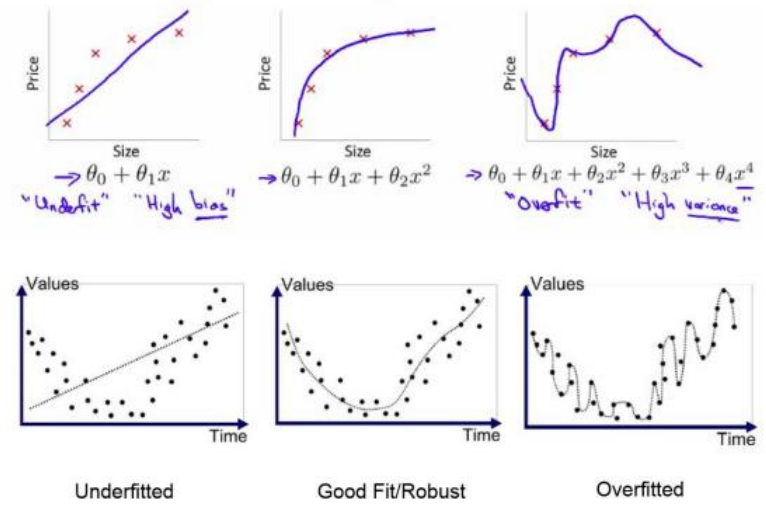
En canto ao deseño para problemas xenerais neceistaranse tantas funcións como clases se queiran discriminar.

## c) Sobreaprendizaxe

É un dos problemas comúns cando se aprende de forma automática, é dicir, cando utilizamos "*matemáticas que aprenden"*.

A sobreaprendizaxe é facelo tan ben que se aprende con moita fidelidade o conxunto de entrenamento pero ese conxunto de entrenamento é unha representación de

todo o conxunto problema que se supón que está suficientemente dimensionado en canto a número de exemplos, pero é uinha representación pequena da infinidade de casos que se poden dar. A sobreaprendizaxe é errónea xa que impide a xeralización.




Conclusión, non é bo infraaprender: primeira gráfica, móstrase que hai erros significativos de aproximación.

A segunda gráfica corresponde a unha boa aproximación. E a terceira é perfecta en canto ao erro overfit; non obstante cando hai que coller o exemplo é un erro enorme.

Sobreaprendizaxe: en polinomio é fantástica aproximación e terrible xeralización. Non representa ben o que é a evolución  dos puntos. É un grado de polinomio excesivo.

## d) ¿Como aborda-lo sobreaprendizaxe?

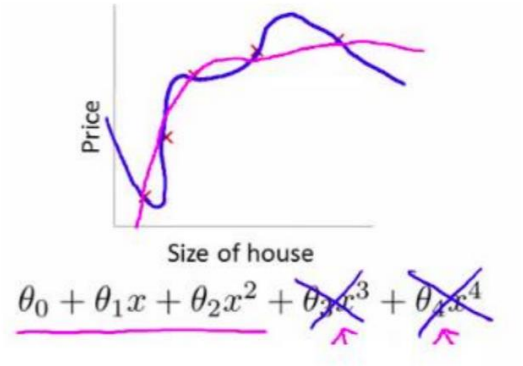
Hai múltiples formas de abordar e corrixir a sobreaprendizaxe. Algunhas formas:

- Reducindo o número de características (variabels) coas que estou operando, manualmente ou mediante métodos de selección das características menos relevantes. Saber por experiencia que é importante e que non. Información "adicional", etc.
- "Regularización" ou mantendo as características, pero reducindo o valor dos parámetros θ

## e) Regularización

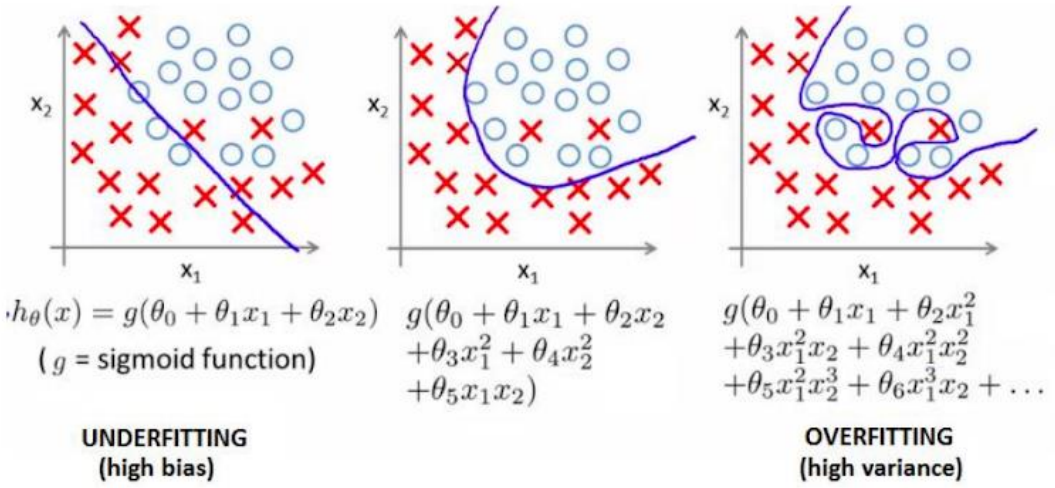
**Está na bitácora anterior e non explicou outra vez pero adxúntase a diapositiva correspondente para contextualizar**

Incluimos en la función de coste los valores de $\theta_j$ al cuadrado, ponderados por un factor λ, de valor alto. Esto hace que solo los valores de $\theta_j$ más relevantes prevalezcan




$$J(\theta) = \frac{1}{2m}\left[\Sigma_{i=1}^{m}\left(h_\Theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{m}\theta_j^2\right], con\ \lambda \gg$$

f) Figura que se saltara de Regresión Loxística




É a mesma idea de sobreaprendizaxe pero nun problema de regresión loxística (clasificación).

Na primeira gráfica: hai unha mala división entre dúas clases (lineal).

Segunda: hai unha boa división. Aprendin algo mais complexo que a sigmoide da primeira.

Terceira: é tan complexo e reversado que realmente da un clasificador que vai funcionar mal sobretodo na fronteira ao discriminar novos datos.

# 3. Aprendizaxe non supervisada

A maior parte dos problemas prácticos que se abordan na aprendizaxe automático pertecen á categoría de aprendizaxe supervisada. Eu teño un exemplo suficientemente representativo do tipo de datos cos que me vou atopar cando ese sistema que estou diseñando contén condicións normais. Ese conxunto de datos representativo además conteñen etiquetado, por tanto para eses

datos teño perfectamente identificado co que me gustaría que ese sistema vaia aprender a facer, predecir, estimar.

Hai problemas onde isto non o podo facer. Podo ter datos representativos pero non os teño preetiquetados.

Enton: ¿Que busco?

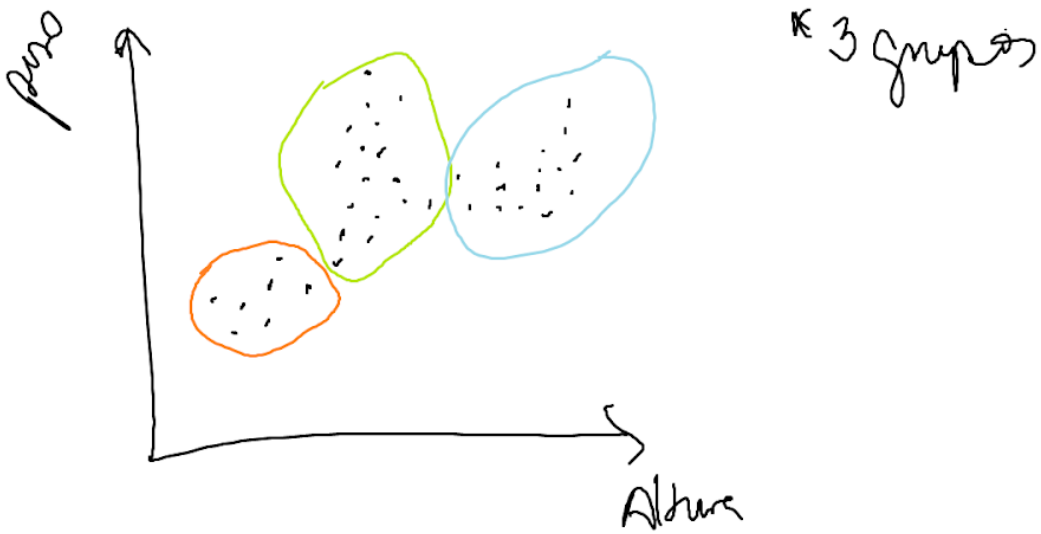
Eu busco agrupalos pero sen etiquetas predefinidas. Creo grupos distintos, agrupo os que se parecen entre si e diferencioos.

## g) Exemplo

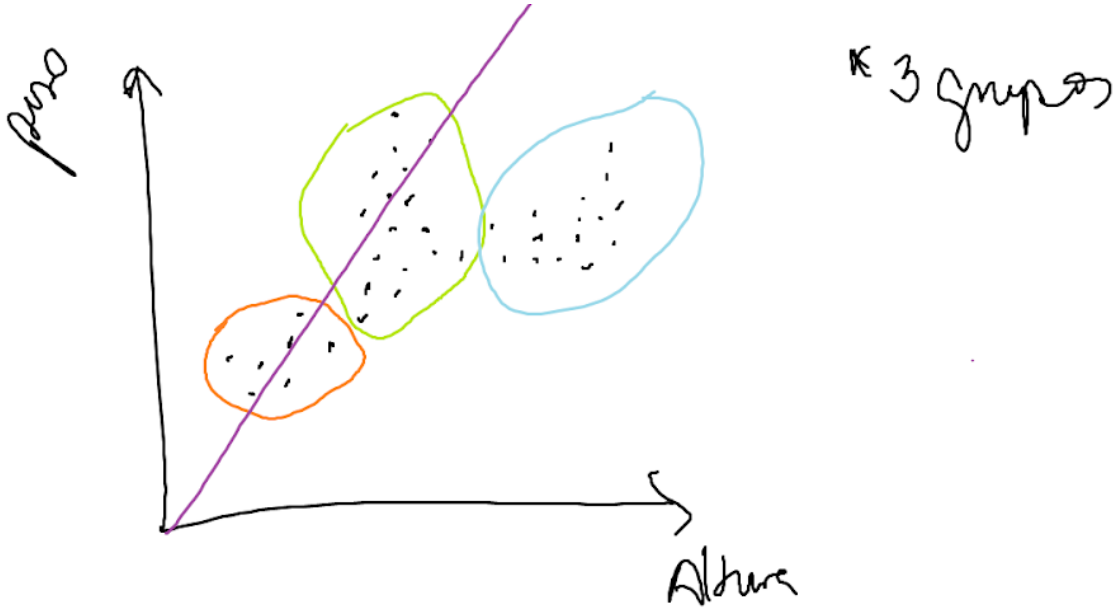
Nos medimos todos os da aula: (debuxos de Senén no encerado)




Intentar agrupalos a ollo:



Logo de agrupar: buscar interpretación.




Reflexión de dividir en tres grupos: hai xente que é alta e pesa pouco para a súa altura.

Poden haber excepcións e que esas excepcións poden ser interesantes para algún tipo de análise.

Funcionan con criterios matemaáticos, este exemplo foi con empírico, de algún modo coa nosa experiencia e co noso criterio categorizamos nos.

Entender que o que se parece, un mesmo elemento, comparte características, funcionalidades, a forma na que están feitos.

Nos non resolvemos matemáticas no sentido estricto cando dicimos “vemos tres grupos”.

Para levalo ao terreo computacional hai que buscar o equivalente a: como é discriminar cousas que se parecen entre si.

Isto é o que busca a aprendizaxe non supervisada.

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grado en Ingeniería Informática

3º curso – 1º cuatrimestre

# Inteligencia Artificial

27-10-2021

Clase 14

Temas trabajados: Aprendizaje no supervisado y aprendizaje automático por retropropagación

# Índice

1. Aprendizaje no supervisado ..... 3
2. Método K-medias (K-Means) ..... 3
3. Aprendizaje automático en RNA por retropropagación ..... 7
   a) Necesidad de redes multicapa ..... 8
   b) Ajuste de los pesos ..... 9

# 1. Aprendizaje no supervisado

Los datos (conjunto de entrenamiento) al final son los que nos aportan información para poder construir una solución al problema. La diferencia es que en este caso los datos no están etiquetados, por lo tanto, no tenemos a priori identificado como debería responder el sistema, sino que al final todo se basa en buscar proximidad, en buscar una forma de medir o cuantificar la similitud entre los datos, para poder agrupar aquellos que se parezcan entre sí de acuerdo con ese criterio. El criterio empleado debe tener sentido dado el problema a abordar.

Es el diseñador el que le atribuye significado (pertinencia) y valor (utilidad) al resultado. La máquina aprende a agrupar los datos, quien les asocia un significado es el diseñador. En el aprendizaje supervisado ya había claramente una proyección sobre el conjunto de datos de lo que era el significado de estos dentro del problema a resolver. Si partimos de un conjunto de datos y un conjunto de resultados que deberían de estar asociados a ellos, estos resultados ya son el significado de estos datos. Por lo tanto, en estos conjuntos de entrenamiento ya va una buena parte de información significativa desde el punto de vista de la salida de vuelta por un sistema de aprendizaje supervisado.

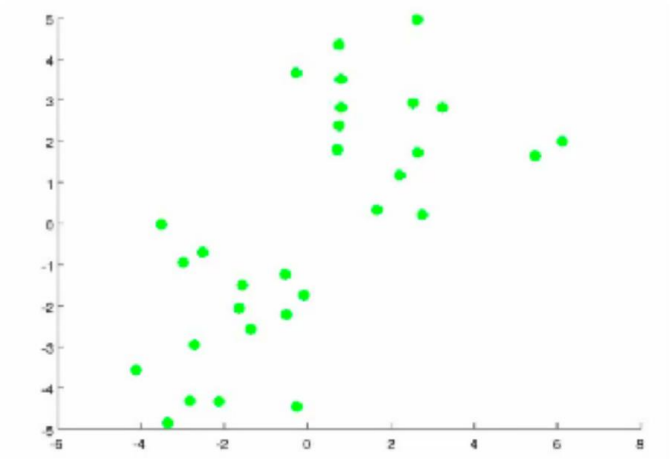
En aprendizaje no supervisado, no existe esta información. El sistema únicamente creará los agrupamientos y tendrá que ser el diseñador el que les dé significado.

Después del supervisado es el tipo de aprendizaje más utilizado

# 2. Método K-medias (K-Means)

Es el método de aprendizaje no supervisado más intuitivo y simple. Tiene multitud de variantes, pero vamos a ver el más elemental.

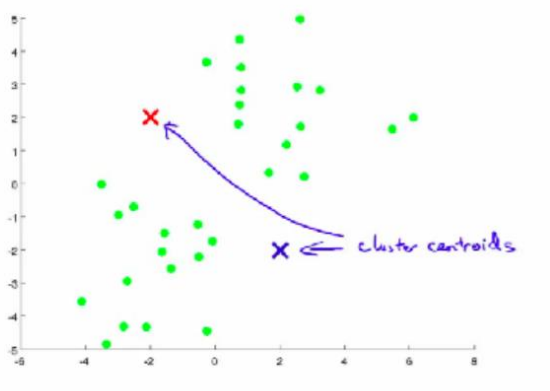
Suponemos que tenemos un conjunto de datos como el de la foto caracterizados a partir de dos variables, como altura y peso de un conjunto de personas.



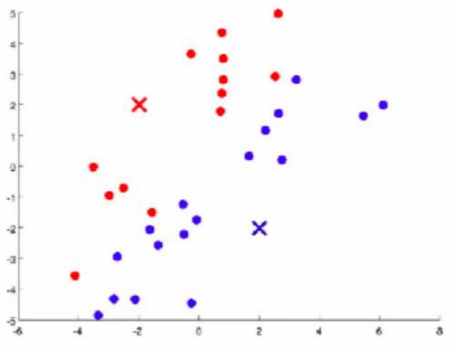
A ojo podemos distinguir agrupaciones en la imagen. Es un criterio empírico, no sabemos cómo realizamos estas agrupaciones en nuestra cabeza.

¿Cómo lo haría el método k-medias?

Suponemos que queremos identificar 2 grupos de datos. Partimos de los llamados centroides de cada uno de los grupos y los situamos aleatoriamente.




A partir de estos puntos de referencia, vemos que puntos están más cerca de uno y cuales están más cerca del otro, identificando cada punto con un centroide. Depende del tipo de distancia que usemos y esta es una cuestión muy relevante que dependerá, en la mayoría de los casos, del problema que queremos resolver.



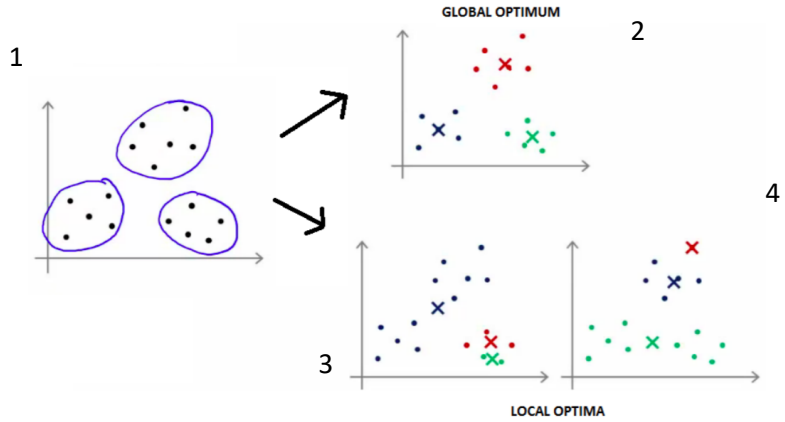
La imagen anterior sería una primera división del conjunto de puntos de partida en los dos grupos que tratamos de identificar utilizando distancia euclídea. El proceso lógicamente no termina aquí, va iterando:

- Ahora que ya tenemos distinguidos dos conjuntos, vamos a calcular sus nuevos centroides: buscamos el punto medio de cada conjunto.
- Al calcularlo, se van moviendo los centroides y puede ser que algún dato cambie de conjunto al estar más cerca del nuevo centroide del otro grupo.
- Se repite el proceso hasta la convergencia.

La base de operaciones de este algoritmo es muy simple, si se analiza en detalle y se utilizan otro tipo de distancias, este puede llegar a ser más complejo. Algunas cuestiones:

- Numero de agrupamientos en los que dividimos, nos lo pueden dar predefinido o no. Si no nos lo dan, veremos un criterio para elegirlo.
- Donde inicializamos los centroides, ya que podrían variar el resultado. Se suele hacer aleatoriamente e influirá en la velocidad del algoritmo (número de iteraciones necesarias para converger). Por ejemplo, en la imagen:
  (1) A ojo, podemos distinguir 3 agrupamientos

(2) Si tenemos la suerte de inicializar bien los centroides desde un principio, el proceso será óptimo y rápido.
(3) Si inicializamos así los centroides, el rojo y el verde se repartirán esos 5 puntos y el resto se quedarán siempre en el azul. Mala distribución.
(4) Si partimos de esos centroides, no sería correcta la agrupación porque el cluster rojo quedaría sin puntos asociados.




Para evitar problemas respecto a la inicialización de los clusters, podemos repetir el proceso muchas (cientos o miles, considerando el coste computacional que esto puede suponer) veces partiendo de distintos centroides y quedarnos con la mejor solución.

¿Cuál es esta mejor solución? A ojo, podemos ver en la imagen anterior cual es la mejor solución, la 2. Pero esto no nos vale computacionalmente. Deberemos tener un criterio de coste o rendimiento. Podemos suponer que este criterio normalmente es la solución que mejora una función de coste: por ejemplo, podemos decir que el coste o error será mayor cuanto más separados estén los puntos de un agrupamiento respecto a su centroide. Sumando las distancias euclídeas, en este caso, de los puntos a su centroide, podemos tener una medida del coste de la solución. Así también podríamos ver computacionalmente que la mejor solución es la 2, ya que es la que minimiza la función de error en este caso.

- $c^{(i)}$ = índice del agrupamiento (1,2,...,K) al cual el valor de entrada $x^{(i)}$ se ha asignado [temporalmente]
- $\mu_K$ = centroide del agrupamiento k ($\mu_K \in \Re^n$)
- $\mu_{c(i)}$ = centroide del agrupamiento al que se asignó [temporalmente] $x^{(i)}$

Objetivo a optimizar:

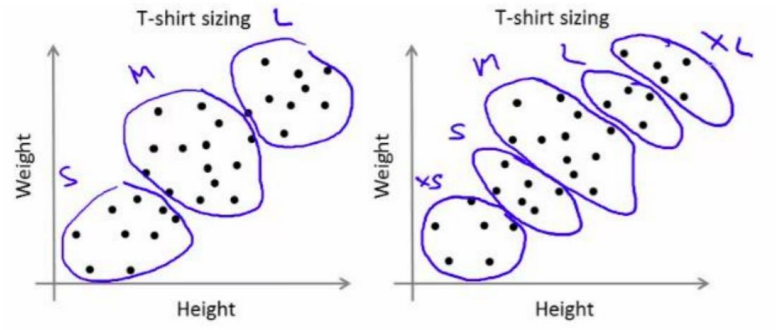
$$J(c^{(1)},\dots,c^{(m)},\mu_1,\dots,\mu_K) = \frac{1}{m}\sum_{i=1}^{m}||x^{(i)} - \mu_{c^{(i)}}||^2$$

$$\min_{\substack{c^{(1)},\dots,c^{(m)},\\ \mu_1,\dots,\mu_K}} J(c^{(1)},\dots,c^{(m)},\mu_1,\dots,\mu_K)$$

Tenemos un proceso de agrupamiento cuyo objetivo es maximizar la semejanza entre sí de los puntos pertenecientes a un agrupamiento, es decir, minimizar la distancia entre los puntos asociados en un agrupamiento y maximizarla con los de otros agrupamientos.

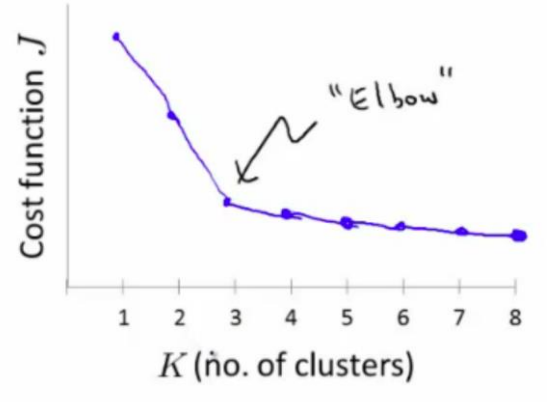
Pensando así podríamos decir que cada punto sea un agrupamiento distinto y por lo tanto sería optimo, ya que no habría distancia entre los puntos del agrupamiento, al ser solo un punto. Evidentemente, no diseñamos un sistema para que al final nos diga que tenemos tantos agrupamientos como puntos del conjunto de datos. Habrá que llegar a una solución de compromiso. ¿Cuántos clusters vamos a diferenciar? Podemos tener un número predefinido o obtener dicho número utilizando un criterio, que minimice o maximice, dependiendo de cada caso, el criterio establecido.

Ejemplo: no podemos diseñar camisetas a medida de cada consumidor. Debemos tener un cierto número de tallas y dicho número de tallas debe ser adecuado para ajustarse al público. No podemos tener demasiadas tallas, ya que esto supondría un gasto para el comerciante, pero tampoco demasiado pocas ya que debemos adaptarnos a los consumidores. Se podría coger una representación de la población a la que se dirige para la solución de este problema.




NOTA: En la imagen anterior podemos ver reflejada la idea de la que hablábamos al principio: el diseñador es el que le da sentido a los clusters, asignando una talla a cada uno y decidiendo que dicha talla debe cubrir a todas las personas incluidas en el cluster.

Si no nos dan un número predefinido de clusters, deberemos buscar el mínimo número que represente bien el conjunto de datos. ¿Qué criterio usamos? Tenemos una función de coste, como la anterior, la que minimizaba la distancia euclídea de cada punto respecto a su centroide. ¿Dónde estaría una solución de compromiso entre el número de agrupamientos y la reducción de la función de coste? Se puede ir viendo, representando el coste respecto al número de clusters utilizado y nos va a quedar una gráfica así:



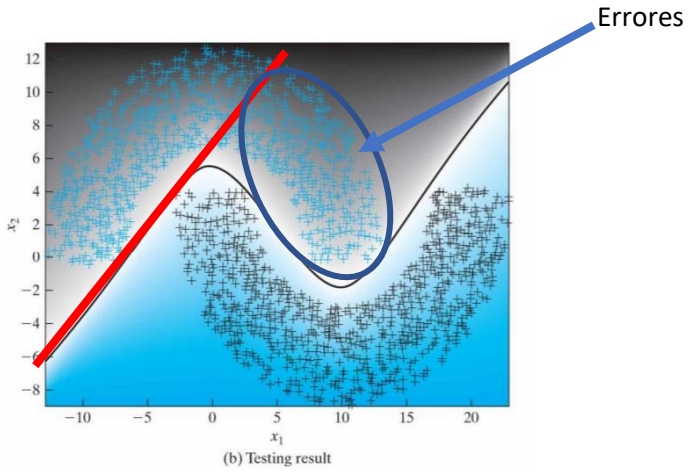
Para pocos clusters, el coste será alto. Según vamos añadiendo clusters, el coste cae muy rápidamente hasta llegar a un punto de inflexión donde ya, aunque vaya añadiendo más y más clusters, prácticamente ya no se reduce el coste o se reduce muy lentamente. Ese punto de inflexión o "codo" es el que se suele usar muchas veces como criterio práctico, se dice que esa solución de compromiso que buscábamos está ahí. A partir del codo, consideramos que el coste computacional adicional no vale la pena teniendo en cuenta la pequeña mejora del coste. En este caso, el número óptimo de clusters es 3.

Es bastante fácil que existan outliers o puntos atípicos en el conjunto de datos, debidos o no a errores de medida. Quedarán fuera de rango y distorsionarán la solución que obtendremos. Estos puntos se pueden identificar antes o durante el proceso de agrupamiento y descartarlos.

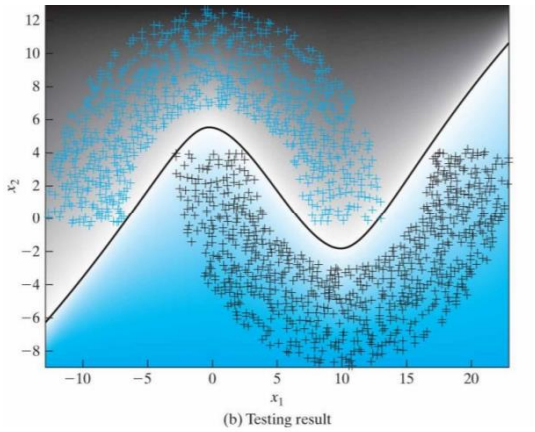
# 3. Aprendizaje automático en RNA por retropropagación

Este capítulo analiza el algoritmo de retropropagación del error. Es un algoritmo o un método que se asocia al aprendizaje supervisado y computación neuronal. Hasta ahora vimos que las redes neuronales se entrenaban con ciertos algoritmos, pero estos no permitían operar con estructuras multicapa, suponiendo una limitación clara respecto a la funcionalidad de estas redes neuronales y a como podían tratar los conjuntos de datos. Fundamentalmente el problema es que las redes monocapa no permiten discriminar problemas que no sean linealmente discriminables.

Por ejemplo, en la imagen siguiente no se podrían clasificar los puntos azules y los negros con una frontera de tipo lineal, como la linea roja, ya que, si usasemos una recta, resolveriamos el problema con muchos errores.




La alternativa sería aprender a construir una frontera de discriminación no lineal, como la de la siguiente imagen. De esta manera, como mínimo para el conjunto de entrenamiento, la clasificación es perfecta. Aunque, como ya sabemos, puede que para el conjunto de test o cualquier otro dato no lo sea, no es completamente infalible.



(b) Testing result

Las redes neuronales utilizadas para reconocimiento de patrones, análisis de textos… Son redes que usan la retopropagación y que están formadas por múltiples neuronas, capas, conexiones.. Es lo que se conoce como aprendizaje profundo (“deep learning”). Esta es muy buena aproximación de fuerza bruta.

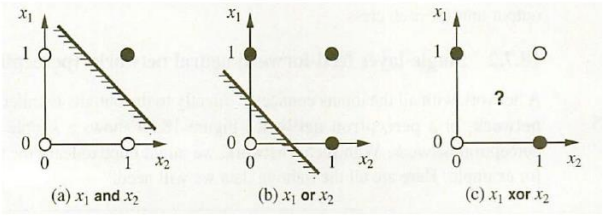
Ejemplo: Discriminación de animales sobre imágenes. Se debería tener una BD con múltiples imágenes, poniendo el resultado que debería obtener (el nombre de cada animal). Al principio funcionaría mal, pero finalmente solo cometen un 5% de errores. Si se tiene que extraer el aprendizaje de la red, no se puede. Es el principal problema del aprendizaje profundo, es increíblemente difícil de saber cual fue el factor decisivo en la clasificación, ya que esta clasificación está diluida en cientos de miles de datos y cada uno de ellos a priori no tiene un significado asociado, es igual el peso 1 que el peso 500. Por ello, a este tipo de computación se le denomina cajas negras. Se tienen cientos de miles de parámetros para aprender lo que se necesita y no se sabe cómo el sistema lo aprende, hay una falta de explicabilidad. Muchas veces los criterios que aprenden estas redes son criterios que a nosotros no nos resultarían útiles, pero que la red correlaciona. Por ejemplo, para la discriminación de animales, la red puede aprender a distinguir los lobos de los perros no por las características del propio animal sino por su hábitat que aprecia en la fotografía.

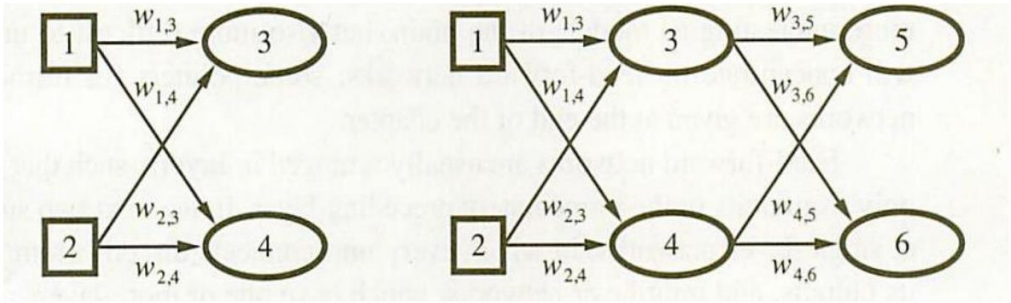
## a) Necesidad de redes multicapa

Se debe recordar una red monocapa que puede resolver ciertas cosas pero hay otras que necesitan un sistema multicapa.

Ejemplo: semisumador con acarreo.

| $x_1$ | $x_2$ | $y_{acarreo}$ | $y_{xuma}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |



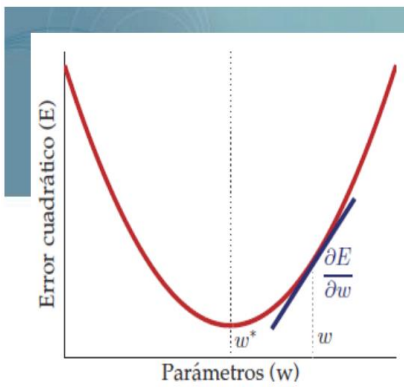




Esta función tiene dos salidas: una de suma y otra de acarreo. Esta es linealmente tratable, fijarse que si las dos variables de entrada ($x_1$ y $x_2$) con dos ejes y se tienen 4 puntos con 4 valores que definen la función de acarreo (a) se debe discriminar el punto negro que representa un 1 respecto a los otros que representan un 0 y, evidentemente eso se puede hacer con una frontera de discriminación con una línea. Lo mismo con el or (b).

El problema se encuentra en la función de suma (c), los puntos negros deberían tener un sistema con valor 1 y los blancos con valor 0. Con una única neurona o una única capa de neuronas, no hay forma de realizar esta discriminación porque siempre generaría errores, ya que la frontera de discriminación debe ser una línea, clasificando mal alguno de los puntos pongamos donde la pongamos. Con una red de 2 capas, sí que se puede resolver este problema. Sabiendo el error cometido, en cuanto a la diferencia entre la salida obtenida y la que se debería obtener, se sabe cómo se deben cambiar los pesos para hacer que esta discrepancia sea menor y, por tanto, hacer que la red neuronal tenga el funcionamiento deseado. Hay un proceso de evolución desde la entrada hasta la salida dónde todos los pesos del camino afectan, por lo que se deberían ajustar todos estos. Esto es lo que hace el algoritmo de retropropagación.

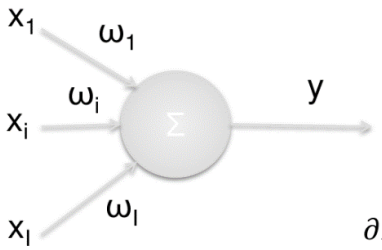
## b) Ajuste de los pesos

Este algoritmo sigue con el mismo criterio de siempre. Se ajustan los pesos proporcionalmente y en sentido contrario a cómo el error que comete la red se ve influido por ese peso. Si el error de la red fuese la curva roja del siguiente gráfico, y se representa el error sobre ese peso, me interesaría reducir el peso y con él, el error cometido. Por eso, utilizamos la derivada negativa.




$$\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$$

La ecuación de la imagen anterior es la ecuación de ajuste de pesos utilizada para TODOS LOS PESOS, independientemente de que sean los últimos o los primeros. El problema está en que cómo resolver la derivada parcial para ciertos pesos. Si por el medio existe una neurona no lineal entre dos elementos, no se podría realizar la derivada. No sería tan directa la solución. Tomando una neurona normal (sin función de activación), donde calculamos la salida (y) en función del sumatorio producto ponderado de las entradas ($x_i$) por los pesos correspondientes ($w_i$).




$$y = \sum_{i=1}^{I} x_i w_i$$

$$E = (t - y)^2$$

$$\Delta w_i = -\mu \frac{\partial E}{\partial w_i}$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial y}{\partial w_i}\frac{\partial E}{\partial y} = x_i.(-2(t - y))$$

$$w_i(t + 1) = w_i(t) + 2\mu.x_i.(t - y)$$

En la fórmula del error cuadrático: $E=(t-y)^2$ t sería la salida deseada y la variable y sería la salida obtenida.

Se tienen que ajustar los pesos de acuerdo con ese criterio y, si se hace a través de un proceso iterativo, se va reduciendo el error cometido a medida que se ajusten los pesos.

¿En qué medida el error E depende del peso? Se puede calcular de la siguiente manera: la derivada parcial de E con respecto $w_i$ se hace en descomponiéndola: La derivada parcial de E respecto a Y por la derivada parcial de Y con respecto a $w_i$. Es decir, ¿cuál es la derivada del error respecto al parámetro Y? Pues sería en qué medida ese error depende de Y por en qué medida Y depende de $w_i$.

$$y = \sum_{i=1}^{I} x_i w_i$$

$$E = (t - y)^2$$

$$\Delta w_i = -\mu \frac{\partial E}{\partial w_i}$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial y}{\partial w_i}\frac{\partial E}{\partial y} = x_i.(-2(t - y))$$

$$w_i(t + 1) = w_i(t) + 2\mu.x_i.(t - y)$$

Si se sigue esta ecuación de cambio para ajuste de pesos de forma iterativa, y se aplican los $x_i$, de algún modo converge reproduciendo las salidas con bastante fidelidad a los datos reales.

Las funciones anteriores son para un único punto, pero si se tienen un conjunto de datos con N puntos se utilizarían las siguientes, que son las anteriores pero adaptadas a N valores. El superíndice n quiere decir que se trata del dato de entrada n-ésimo.

$$E = \frac{1}{2}\sum_{n=1}^{N}(t^n - y^n)^2$$

$$\frac{\partial E}{\partial w_i} = \frac{\partial y^n}{\partial w_i}\frac{\partial E}{\partial y^n}$$

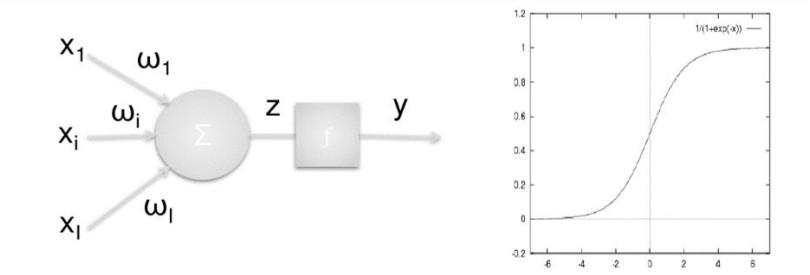
$$\frac{\partial E}{\partial w_i} = \frac{\partial\left(\frac{1}{2}\sum_{n=1}^{N}(t^n - y^n)^2\right)}{\partial w_i} = \frac{1}{2}\sum_{n=1}^{N}\frac{\partial y^n}{\partial w_i}\frac{\partial (t^n - y^n)^2}{\partial y^n} =$$

$$\frac{1}{2}\sum_{n=1}^{N} x_i^n.\left(-2(t^n - y^n)\right) = -\sum_{n=1}^{N} x_i^n (t^n - y^n)$$

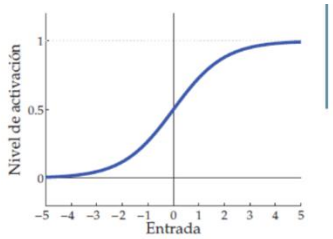
$$w_i(t + 1) = w_i(t) + \mu\sum_{n=1}^{N} x_i^n\,(t^n - y^n)$$

**AVISO: EN TODOS LOS SUMATORIOS QUE TIENEN ½, EN REALIDAD SERÍA 1/2N, LAS FÓRMULAS ESTÁN MAL. Están bien en el documento del cv en el que explica en profundidad esto.**

Esto se puede complicar si se utiliza una función no lineal, por ejemplo, una función sigmoide.




Básicamente servirían las ecuaciones usadas hasta ahora, sólo que se deben tener en cuenta algunas consideraciones. Interesa saber la derivada de la función sigmoide respecto a z:




$$z = \sum_{i=1}^{I} x_i\, w_i \qquad y = \frac{1}{1+e^{-z}}$$

$$\frac{\partial y}{\partial z} = \frac{0-(-e^{-z})}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = y(1-y)$$

**Regra Delta** para ***Perceptróns*** con NA **sigmoidais**

¿Cómo sería ahora, una vez introducida la función de activación sigmoide en la salida de la neurona, la expresión de cambio de los pesos? Pues del mismo modo que en una función lineal, se cambian los pesos en la dirección contraria de la derivada del error respecto al peso para la reducción del error.

Para un conjunto de datos, se utilizarían las siguientes funciones:

$$\Delta w_i = -\mu \frac{\partial E}{\partial w_i} = -\mu \frac{1}{2} \sum_{n=1}^{N} \frac{\partial y^n}{\partial w_i} \frac{\partial (t^n - y^n)^2}{\partial y^n}$$

$$= -\mu \frac{1}{2} \sum_{n=1}^{N} \frac{\partial z^n}{\partial w_i} \frac{\partial y^n}{\partial z^n} \frac{\partial (t^n - y^n)^2}{\partial y^n} =$$

$$-\mu \frac{1}{2} \sum_{n=1}^{N} x_i^n \left(y^n(1-y^n)\right).\left(-2(t^n - y^n)\right)$$

$$= \mu \sum_{n=1}^{N} x_i^n \left(y^n(1-y^n)\right).(t^n - y^n)$$

Escola Técnica Superior de Enxeñaría

USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Grao en Enxeñaría Informática

3º curso – 1º cuadrimestre

# Intelixencia artificial

03-11-2021

Clase 16

Temas traballados:

Aprendizaxe automática en RNA por retropropagación

# Índice

1. Introdución ..... 3
2. RNA multicapa ..... 3

# 1. Introdución

En primeiro lugar, comezouse facendo un recordatorio das exposicións, que comezarán o mércores 10 de novembro. Realizouse unha revisión dos grupos que xa están anotados e preguntouse se faltaba algún por anotarse. Efectivamente si faltaba, as nosas queridísimas Diana Mascareñas Sande e Teresa Gutiérrez Blanco faltaban por apuntarse.

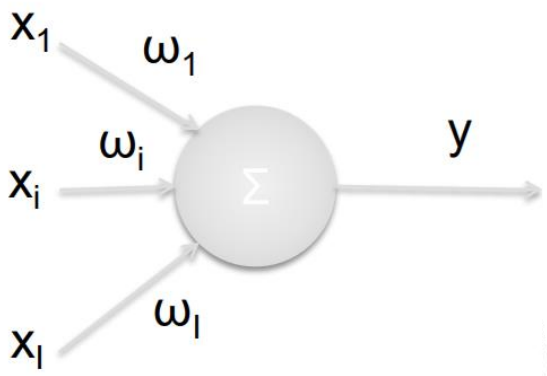
Logo o profesor fixo spam dos seus seminarios, destacando un sobre como comunicar ben.

Antes de dar lugar ao comezo da clase, tamén destacou algúns erros que teñen moitas veces os alumnos á hora de expor e que debemos evitar, como meter as mans nos petos (a xesticulación é moi importante, recálcase), non ver ó público mentres se expón... Non se poden descoidar este tipo de cousas.

*Cabe destacar que para a realización da bitácora utilizouse o PDF que está no campus virtual onde se desenvolve este apartado e non as diapositivas, por se alguén se fixa nas diapositivas que saiba que a notación utilizada aquí non vai coincidir.*

# 2. RNA multicapa

Xa vimos anteriormente como utilizar a regresión lineal para entrenar unha rede perceptrón monocapa, mais **facelo no caso dunha rede multicapa é máis complexo**. Para explicalo imos comezar co caso de ter unha única neurona:




Esta neurona ten I entradas, as cales pondera cos valores de ω. Por outro lado temos conxuntos de adestramento ($X^n$, $t^n$) onde $X^n$ representa o conxunto de entradas ($x_1^n$, $x_2^n$, $x_3^n$..., $x_I^n$) e $t^n$ é a saída desexada. Podemos definir o erro como:

$$E = \frac{1}{2N}\sum_{n=1}^{N}(t^n - y^n)^2$$

$$z = \sum_{i=1}^{I} x_i\, w_i \qquad y = \frac{1}{1+e^{-z}}$$

$$\frac{\partial y}{\partial z} = \frac{0-(-e^{-z})}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}}\cdot\frac{e^{-z}}{(1+e^{-z})} = y(1-y)$$

$$\Delta w_i = -\mu\frac{\partial E}{\partial w_i} = -\mu\frac{\partial\left(\frac{1}{2N}\sum_{n=1}^{N}(t^n-y^n)^2\right)}{\partial w_i} = -\mu\frac{1}{2N}\sum_{n=1}^{N}\frac{\partial y^n}{\partial w_i}\frac{\partial(t^n-y^n)^2}{\partial y^n}$$

$$= -\mu\frac{1}{2N}\sum_{n=1}^{N}\frac{\partial z^n}{\partial w_i}\frac{\partial y^n}{\partial z^n}\frac{\partial(t^n-y^n)^2}{\partial y^n} =$$

$$-\mu\frac{1}{2N}\sum_{n=1}^{N}x_i^n\left(y^n(1-y^n)\right).\left(-2(t^n-y^n)\right)$$

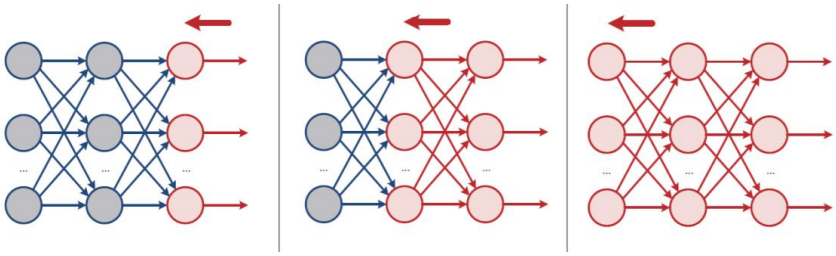
$$= \mu\frac{1}{N}\sum_{n=1}^{N}x_i^n\left(y^n(1-y^n)\right).(t^n-y^n)$$

Xa que:

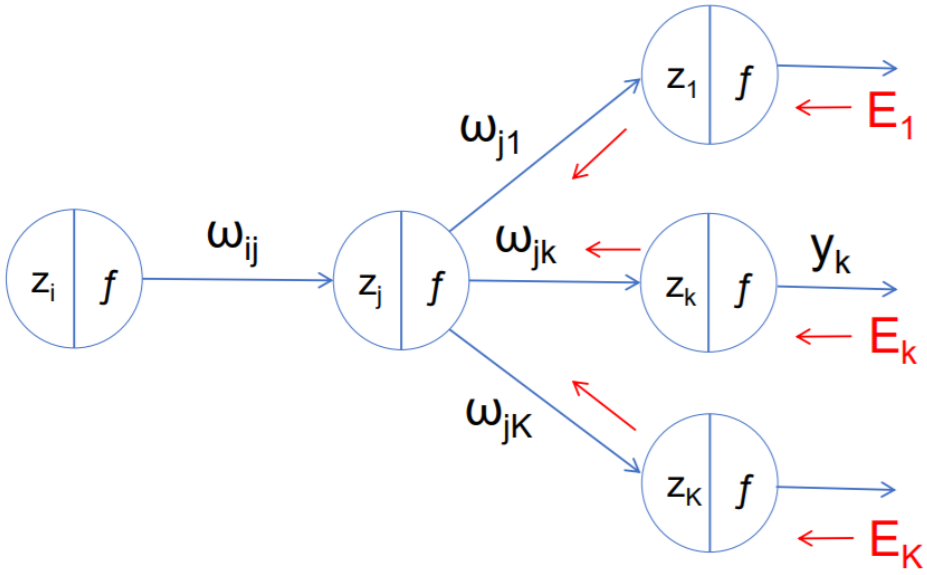
$$\frac{\partial y^n}{\partial w_i} = \frac{\partial z^n}{\partial w_i}\frac{\partial y^n}{\partial z^n}$$

$$\frac{\partial z^n}{\partial w_i} = x_i^n, \qquad \frac{\partial y^n}{\partial z^n} = y^n(1-y^n), \qquad \frac{\partial(t^n-y^n)^2}{\partial y^n} = -2(t^n-y^n)$$

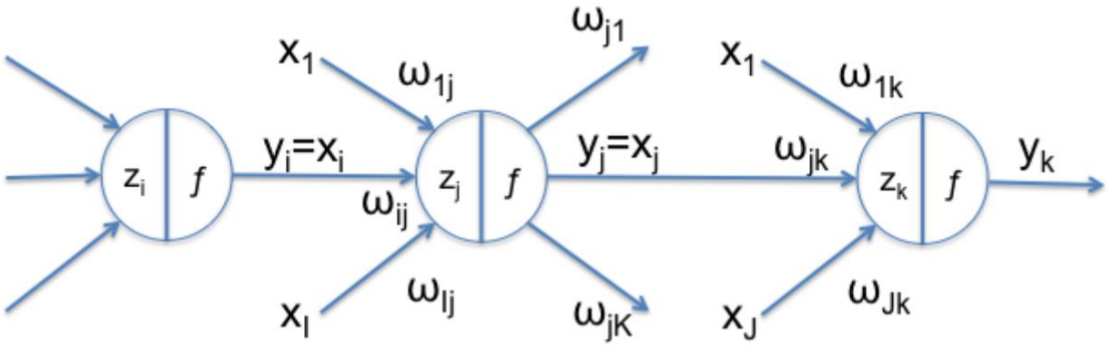
Con este método podemos resolver o caso dunha capa, dá igual o número de neuronas. Mais se queremos adestrar a **estruturas multicapa**, as cales poden superar as limitacións das monocapa, como podemos facelo? Mediante a retropropagación de erros.



**Nunha RNA multicapa, as neuronas das capas ocultas (as que producen saídas intemedias) afectan á saída da rede, e polo tanto, ó erros cometidos por esta**. Deste modo, os pesos das dúas conexións afectarán a ditos erros e, polo tanto, deberán ser axustados para minimizalos.




A imaxe poderíase xeralizar a calquera número de capas e neuronas por capa. Mais imos fixarnos nunha única neurona de saída, a k-ésima, xa que unha vez feita a análise de como inflúen os pesos da rede neuronal nela, o resto de neuronas trataríanse de igual maneira.




En primeiro lugar destacar que **é importante que todas as funcións intermedias sexan derivables para a realización dos cálculos**. Comezamos definindo o erro asociado á neurona k. Neste caso utilizamos o erro cuadrático pero poderíamos medilo doutra maneira.

$$E_k = (t_k - y_k)^2 \qquad \frac{\partial E_k}{\partial w_{jk}} = \frac{\partial y_k}{\partial w_{jk}} \frac{\partial E_k}{\partial y_k}$$

$$\frac{\partial E_k}{\partial y_k} = -2(t_k - y_k)$$

$$\frac{\partial y_k}{\partial w_{jk}} = \frac{\partial f(z_k)}{\partial w_{jk}} = f'(z_k).\frac{\partial z_k}{\partial w_{jk}} = f'(z_k).\frac{\partial(\sum_j y_j w_{jk})}{\partial w_{jk}} = f'(z_k).y_j$$

$$\frac{\partial E}{\partial w_{jk}} = -2(t_k - y_k).f'(z_k).y_j$$

$$\Delta w_{jk} = -\mu\frac{\partial E_k}{\partial w_{jk}} = \mu(t_k - y_k).f'(z_k).y_j = \mu\Delta_k.y_j$$

Onde consideramos que o 2 xa está incorporado ó valor do coeficiente $\mu$ e $\Delta_k = (t_k - y_k).f'(z_k)$, é un valor que logo utilizaremos e que é dependente do erro cometido pola neurona k-ésima de saída. Como era de esperar, se non hai discrepancia entre a saída dada pola neurona k-ésima, non vai cambiar o peso. Tampouco o fará se $y_j$=0, xa que se a saída da neurona é cero, a súa conexión coa neurona seguinte non vai influír no erro desta.

**Ata este punto só reconsideramos o que xa sabíamos**, inda non actuamos sobre as neuronas de niveis inferiores.

Agora a través da expresión $\Delta w_{jk} = w_{jk}(t+1) - w_{jk}(t)$ obtemos os cambios que lle temos que aplicar aos pesos das conexións, entre as neuronas da penúltima capa da rede e a neurona k-ésima da saída. Este cálculo é semellante ao que xa vimos para axustar os pesos dunha única neurona, mais precisamos un método para axustar calquera outro peso da rede. Vexamos, por exemplo, como podemos axustar o valor de $\omega_{ij}$, asociado á conexión entre a neurona i-ésima da antepenúltima capa e a neurona j-ésima da penúltima capa**. Agora complícase xa que $y_j$ contribúe co seu valor ó erro de todas as neuronas de saída e non só á k-ésima**. Polo tanto, os cambios en $\omega_{ij}$ van depender dos erros cometidos en todas as neuronas da capa de saída. É dicir:

$$\Delta w_{ij} = -\mu\sum_k \frac{\partial E_k}{\partial w_{ij}}$$

$$\frac{\partial E_k}{\partial w_{ij}} = \frac{\partial y_k}{\partial w_{ij}}\frac{\partial E_k}{\partial y_k}$$

$$\frac{\partial E_k}{\partial y_k} = -2(t_k - y_k)$$

$$\frac{\partial y_k}{\partial w_{ij}} = \frac{\partial f(z_k)}{\partial w_{ij}} = f'(z_k).\frac{\partial z_k}{\partial w_{ij}} = f'(z_k).\frac{\partial(\sum_j y_j w_{jk})}{\partial w_{ij}} =$$

$$= f'(z_k).\frac{\partial y_j w_{jk}}{\partial w_{ij}} = f'(z_k).w_{jk}\frac{\partial y_j}{\partial w_{ij}} = f'(z_k).w_{jk}\frac{\partial f(z_j)}{\partial w_{ij}} =$$

$$f'(z_k).w_{jk}.f'(z_j).\frac{\partial z_j}{\partial w_{ij}} = f'(z_k).w_{jk}.f'(z_j).\frac{\partial(\sum_i y_i w_{ij})}{\partial w_{ij}} = f'(z_k).w_{jk}.f'(z_j).y_i$$

$$\frac{\partial E_k}{\partial w_{ij}} = -2(t_k - y_k).f'(z_k).w_{jk}f'(z_j).y_i = -2\Delta_k.w_{jk}.f'(z_j).y_i$$

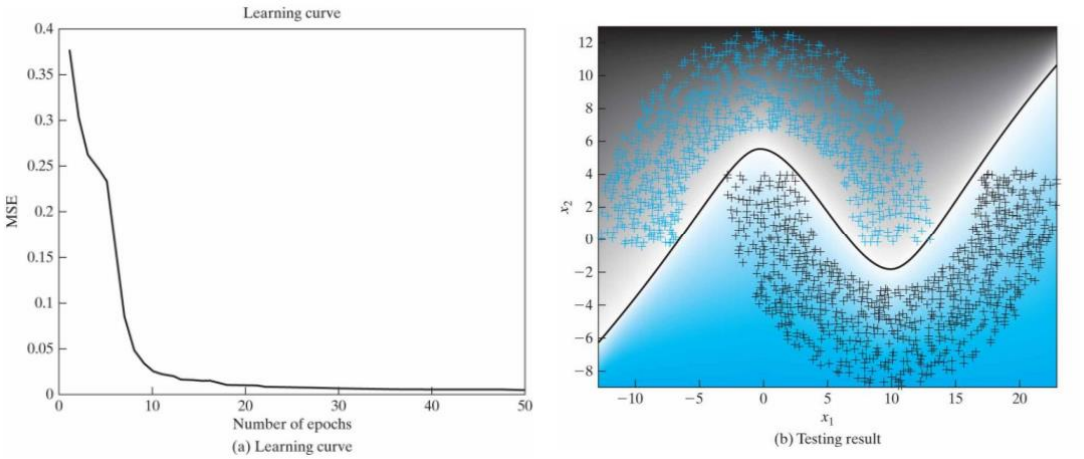
Esta é a proxección do erro cometido pola neurona k-ésima de saída sobre $\omega_{ij}$. Agora habería que sumar todas as contribucións a este peso derivadas do erro que se produce no conxunto de neuronas de saída, tal e como dixemos antes:

$$\Delta w_{ij} = -\mu \sum_k \frac{\partial E_k}{\partial w_{ij}} = \mu y_i . f'(z_j) . \sum_k (w_{jk}\Delta_k) = \mu y_i . \Delta_j$$

Finalmente, xeralizando o proceso para todas as capas, dende a de saída ata a de entrada, teremos as expresións de axuste de todos os pesos da rede. Podemos ver o pseudocódigo do algoritmo completo aquí:

**Function** *RETROPROPAGACIÓN* (*ejemplos*, *red_neuronal*) **returns** *red_neuronal*
**inputs**:
    *ejemplos*  /* conjunto de valores (*X*, *T*) del conjunto de entrenamiento, donde X es la entrada a la red y T la salida deseada
    *red_neuronal*  /* red neuronal con *M* capas, función de activación *f* y pesos $w_{ij}$
**variables locales**: Δ  /* vector de errores; cada valor está asociado a un nodo o neurona
**repeat**
    **for each** $w_{ij}$ **in** red_neuronal **do**  /* inicializa los pesos asociados a las conexiones
        $w_{ij} \leftarrow$ número pequeño aleatorio
    **for each** (*X*, *T*) **in** *ejemplos* **do**
        **for each** nodo *i* **in** capa 1 **do**  /* la 1 no es realmente una capa de neuronas sino que a ella se asocian las entradas a la red
            $y_i \leftarrow x_i$  /* asigna valores de entrada a la red
        **for** *m*=2 to *M* **do**  /* propaga las entradas hacia la capa de salida
            **for each** node *j* **in** capa *m* **do**
                $z_j \leftarrow \sum_i w_{ij} y_i$
                $y_j \leftarrow f(z_j)$
        **for each** nodo *k* **in** capa M **do**
            $\Delta_k \leftarrow f'(z_k).(t_k - y_k)$
        **for** *m*=*M*-1 to 1 do
            **for each** nodo j **in** capa m **do**
                $\Delta_j \leftarrow f'(z_j).\sum_k w_{jk}\Delta_k$
        **for each** $w_{ij}$ **in** red_neuronal **do**  /* actualiza los pesos de la red
            $w_{ij} \leftarrow w_{ij} + \mu y_i . \Delta_j$
**until** criterio de finalización
**return** *red_neuronal*

O proceso aquí pode resultar algo ambigüo, xa que para precisar o peso do que falamos en cada momento, ademais dos subíndices habería que asociarlle a este a capa coa que se corresponde.




Neste exemplo que vemos aquí, vemos un problema que é irresoluble de xeito lineal, e as estruturas monocapa só poden traballar con fronteiras lineais, polo que están bastante limitadas.

Destácase o feito de que o conxunto de parámetros cos que unha IA pode levar a cabo o seu funcionamento é denominado caixa negra, inda que o termo fai referencia xusto ó contrario da caixa negra dun avión, pois esta última serve xustamente para extraer os datos e interpretalos. Neste instante é moi difícil (practicamente imposible) saber como unha intelixencia artificial clasifica e aproxima os datos unha vez esta aprendeu.